Thomas Project Codebase

■■■■■■■■■ - ■■■■■■■

Generated by Antigravity AI Assistant

■ File: index.html

```
&lt;!DOCTYPE html&gt;
&lt;html lang="en"&gt;
  &lt;head&gt;
    &lt;meta charset="UTF-8" /&gt;
    &lt;meta name="viewport" content="width=device-width, initial-scale=1.0" /&gt;
    &lt;title&gt;■■■■■■■■■■&lt;/title&gt;
  &lt;/head&gt;

  &lt;body&gt;
    &lt;div id="root"&gt;&lt;/div&gt;
    &lt;script type="module" src="/src/main.tsx"&gt;&lt;/script&gt;
  &lt;/body&gt;
&lt;/html&gt;
```

■ File: src\App.tsx

```
import React from 'react';
import { Routes, Route, Navigate, useNavigate } from 'react-router-dom';
import { LoginPage } from './components/LoginPage';
import { PatientDashboard } from './components/PatientDashboard';
import { DoctorDashboard } from './components/DoctorDashboard';
import { CaseManagerDashboard } from './components/CaseManagerDashboard';
import { AboutPage } from './components/AboutPage';
import { useAuthStore } from './store/useAuthStore';

export default function App() {
  const { currentUser, login, logout } = useAuthStore();
  const navigate = useNavigate();

  const handleLogout = () =&gt; {
    logout();
    navigate('/login');
  };

  const navigateTo = (page: string) =&gt; {
    if (page === 'dashboard') {
      navigate(currentUser ? `/${currentUser.role}` : '/login');
    } else {
      navigate(`/${page}`);
    }
  };

  const ProtectedRoute = ({ children, role }: { children: React.ReactNode, role: string
     }) =&gt; {
    if (!currentUser) return &lt;Navigate to="/login" replace /&gt;;
    if (currentUser.role !== role) return &lt;Navigate to={`/${currentUser.role}`} replace
     /&gt;;
    return &lt;&gt;{children}&lt;/&gt;;
  };

  return (
    &lt;Routes&gt;
      &lt;Route path="/login" element={
        !currentUser ? &lt;LoginPage onLogin={(user) =&gt; {
          login(user);
          navigate(`/${user.role}`);
        }} /&gt; : &lt;Navigate to={`/${currentUser.role}`} replace /&gt;
      } /&gt;

      &lt;Route path="/patient" element={
        &lt;ProtectedRoute role="patient"&gt;
          &lt;PatientDashboard user={currentUser!} onLogout={handleLogout}
     onNavigate={navigateTo as any} /&gt;
        &lt;/ProtectedRoute&gt;
      } /&gt;

      &lt;Route path="/doctor" element={
        &lt;ProtectedRoute role="doctor"&gt;
          &lt;DoctorDashboard user={currentUser!} onLogout={handleLogout}
     onNavigate={navigateTo as any} /&gt;
        &lt;/ProtectedRoute&gt;
      } /&gt;

      &lt;Route path="/caseManager" element={
        &lt;ProtectedRoute role="caseManager"&gt;
          &lt;CaseManagerDashboard user={currentUser!} onLogout={handleLogout}
     onNavigate={navigateTo as any} /&gt;
        &lt;/ProtectedRoute&gt;
      } /&gt;

      &lt;Route path="/about" element={
        !currentUser ? &lt;Navigate to="/login" replace /&gt; : &lt;AboutPage
     onNavigate={navigateTo as any} onLogout={handleLogout} /&gt;
      } /&gt;

      &lt;Route path="*" element={&lt;Navigate to={currentUser ? `/${currentUser.role}` :
     "/login"} replace /&gt;} /&gt;
    &lt;/Routes&gt;
  );
}
```

■ File: src\components\AboutPage.tsx

```
import { Activity, ArrowLeft, Heart, Shield, Users, Zap } from 'lucide-react';

interface AboutPageProps {
  onNavigate: (page: 'dashboard' | 'about') =&gt; void;
  onLogout: () =&gt; void;
}

export function AboutPage({ onNavigate, onLogout }: AboutPageProps) {
  return (
    &lt;div className="min-h-screen bg-background"&gt;
      {/* Header */}
      &lt;header className="bg-white border-b border-border shadow-sm sticky top-0 z-50"&gt;
        &lt;div className="max-w-full mx-auto px-4"&gt;
          &lt;div className="flex items-center justify-between h-16"&gt;
            &lt;div className="flex items-center gap-3"&gt;
              &lt;div className="w-10 h-10 bg-primary rounded-lg flex items-center
    justify-center"&gt;
                &lt;Activity className="w-6 h-6 text-white" /&gt;
              &lt;/div&gt;
              &lt;h3 className="text-primary"&gt;■■■■■■■■&lt;/h3&gt;
            &lt;/div&gt;
            &lt;div className="flex gap-3"&gt;
              &lt;button
                onClick={() =&gt; onNavigate('dashboard')}
                className="flex items-center gap-2 px-4 py-2 border border-border
    rounded-lg hover:bg-gray-50 transition-colors"
              &gt;
                &lt;ArrowLeft className="w-5 h-5" /&gt;
                ■■
              &lt;/button&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/header&gt;

      &lt;div className="max-w-full mx-auto px-4 py-12 pb-4"&gt;
        {/* Hero Section */}
        &lt;div className="text-center mb-16"&gt;
          &lt;div className="inline-flex items-center justify-center w-20 h-20 bg-primary
    rounded-full mb-6 shadow-lg"&gt;
            &lt;Activity className="w-10 h-10 text-white" /&gt;
          &lt;/div&gt;
          &lt;h1 className="text-text-primary mb-4"&gt;■■■■■■■■■■&lt;/h1&gt;
        &lt;/div&gt;

        {/* Features */}
        &lt;div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6 mb-16"&gt;
          &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border
    text-center"&gt;
            &lt;div className="w-16 h-16 bg-blue-100 rounded-full flex items-center
    justify-center mx-auto mb-4"&gt;
              &lt;Heart className="w-8 h-8 text-primary" /&gt;
            &lt;/div&gt;
            &lt;h3 className="text-text-primary mb-2"&gt;■■■■■■■&lt;/h3&gt;
            &lt;p className="text-text-secondary"&gt;■■■■■■■■■■■■■■■■■■■■&lt;/p&gt;
          &lt;/div&gt;

          &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border
    text-center"&gt;
            &lt;div className="w-16 h-16 bg-cyan-100 rounded-full flex items-center
    justify-center mx-auto mb-4"&gt;
              &lt;Zap className="w-8 h-8 text-secondary" /&gt;
            &lt;/div&gt;
            &lt;h3 className="text-text-primary mb-2"&gt;■■■■■&lt;/h3&gt;
            &lt;p className="text-text-secondary"&gt;■■■■■■■■■■■■■■■■■■■&lt;/p&gt;
          &lt;/div&gt;

          &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border
    text-center"&gt;
            &lt;div className="w-16 h-16 bg-green-100 rounded-full flex items-center
    justify-center mx-auto mb-4"&gt;
              &lt;Users className="w-8 h-8 text-success" /&gt;
            &lt;/div&gt;
            &lt;h3 className="text-text-primary mb-2"&gt;■■■■■■&lt;/h3&gt;
            &lt;p className="text-text-secondary"&gt;■■■■■■■■■■■■■■■■■■&lt;/p&gt;
          &lt;/div&gt;

          &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border
    text-center"&gt;
```

```
        &lt;div className="w-16 h-16 bg-purple-100 rounded-full flex items-center
justify-center mx-auto mb-4"&gt;
          &lt;Shield className="w-8 h-8 text-purple-600" /&gt;
        &lt;/div&gt;
        &lt;h3 className="text-text-primary mb-2"&gt;[redacted]&lt;/h3&gt;
        &lt;p className="text-text-secondary"&gt;[redacted]&lt;/p&gt;
      &lt;/div&gt;
    &lt;/div&gt;

    {/* System Features */}
    &lt;div className="bg-white rounded-xl shadow-lg p-8 border border-border mb-16"&gt;
      &lt;h2 className="text-text-primary mb-8 text-center"&gt;[redacted]&lt;/h2&gt;
      &lt;div className="grid grid-cols-1 md:grid-cols-3 gap-8"&gt;
        &lt;div&gt;
          &lt;h3 className="text-primary mb-4"&gt;[redacted]&lt;/h3&gt;
          &lt;ul className="space-y-2 text-text-secondary"&gt;
            &lt;li className="flex items-start gap-2"&gt;
              &lt;div className="w-2 h-2 bg-primary rounded-full mt-2"&gt;&lt;/div&gt;
              &lt;span&gt;[redacted]&lt;/span&gt;
            &lt;/li&gt;
            &lt;li className="flex items-start gap-2"&gt;
              &lt;div className="w-2 h-2 bg-primary rounded-full mt-2"&gt;&lt;/div&gt;
              &lt;span&gt;8 [redacted]&lt;/span&gt;
            &lt;/li&gt;
            &lt;li className="flex items-start gap-2"&gt;
              &lt;div className="w-2 h-2 bg-primary rounded-full mt-2"&gt;&lt;/div&gt;
              &lt;span&gt;[redacted]&lt;/span&gt;
            &lt;/li&gt;
            &lt;li className="flex items-start gap-2"&gt;
              &lt;div className="w-2 h-2 bg-primary rounded-full mt-2"&gt;&lt;/div&gt;
              &lt;span&gt;[redacted]&lt;/span&gt;
            &lt;/li&gt;
          &lt;/ul&gt;
        &lt;/div&gt;

        &lt;div&gt;
          &lt;h3 className="text-secondary mb-4"&gt;[redacted]&lt;/h3&gt;
          &lt;ul className="space-y-2 text-text-secondary"&gt;
            &lt;li className="flex items-start gap-2"&gt;
              &lt;div className="w-2 h-2 bg-secondary rounded-full mt-2"&gt;&lt;/div&gt;
              &lt;span&gt;[redacted]&lt;/span&gt;
            &lt;/li&gt;
            &lt;li className="flex items-start gap-2"&gt;
              &lt;div className="w-2 h-2 bg-secondary rounded-full mt-2"&gt;&lt;/div&gt;
              &lt;span&gt;[redacted]&lt;/span&gt;
            &lt;/li&gt;
            &lt;li className="flex items-start gap-2"&gt;
              &lt;div className="w-2 h-2 bg-secondary rounded-full mt-2"&gt;&lt;/div&gt;
              &lt;span&gt;[redacted]&lt;/span&gt;
            &lt;/li&gt;
            &lt;li className="flex items-start gap-2"&gt;
              &lt;div className="w-2 h-2 bg-secondary rounded-full mt-2"&gt;&lt;/div&gt;
              &lt;span&gt;[redacted]&lt;/span&gt;
            &lt;/li&gt;
          &lt;/ul&gt;
        &lt;/div&gt;

        &lt;div&gt;
          &lt;h3 className="text-success mb-4"&gt;[redacted]&lt;/h3&gt;
          &lt;ul className="space-y-2 text-text-secondary"&gt;
            &lt;li className="flex items-start gap-2"&gt;
              &lt;div className="w-2 h-2 bg-success rounded-full mt-2"&gt;&lt;/div&gt;
              &lt;span&gt;[redacted]&lt;/span&gt;
            &lt;/li&gt;
            &lt;li className="flex items-start gap-2"&gt;
              &lt;div className="w-2 h-2 bg-success rounded-full mt-2"&gt;&lt;/div&gt;
              &lt;span&gt;[redacted]&lt;/span&gt;
            &lt;/li&gt;
            &lt;li className="flex items-start gap-2"&gt;
              &lt;div className="w-2 h-2 bg-success rounded-full mt-2"&gt;&lt;/div&gt;
              &lt;span&gt;[redacted]&lt;/span&gt;
            &lt;/li&gt;
            &lt;li className="flex items-start gap-2"&gt;
              &lt;div className="w-2 h-2 bg-success rounded-full mt-2"&gt;&lt;/div&gt;
              &lt;span&gt;[redacted]&lt;/span&gt;
            &lt;/li&gt;
          &lt;/ul&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;

    {/* Questionnaires */}
```

```
        &lt;div className="bg-gradient-to-r from-blue-50 to-cyan-50 rounded-xl shadow-md p-8
      border border-blue-200 mb-16"&gt;
           &lt;h2 className="text-text-primary mb-6 text-center"&gt;[redacted]&lt;/h2&gt;
           &lt;div className="grid grid-cols-2 md:grid-cols-4 gap-4"&gt;
             {[
               { name: 'MIDAS', full: 'Migraine Disability Assessment' },
               { name: 'HADS', full: 'Hospital Anxiety and Depression Scale' },
               { name: 'BDI', full: 'Beck Depression Inventory' },
               { name: 'PSQI', full: 'Pittsburgh Sleep Quality Index' },
               { name: 'FSS', full: 'Fatigue Severity Scale' },
               { name: 'WPI', full: 'Widespread Pain Index' },
               { name: 'Allodynia', full: 'Allodynia Questionnaire' },
               { name: 'PSS', full: 'Perceived Stress Scale' },
             ].map((q) =&gt; (
               &lt;div key={q.name} className="bg-white rounded-lg p-4 border border-primary
      shadow-sm"&gt;
                 &lt;div className="text-primary"&gt;{q.name}&lt;/div&gt;
                 &lt;div className="text-text-secondary"&gt;{q.full}&lt;/div&gt;
               &lt;/div&gt;
             ))}
           &lt;/div&gt;
         &lt;/div&gt;
       &lt;/div&gt;
     &lt;/div&gt;
   );
}
```

■ File: src\components\CaseManagerDashboard.tsx

```
import { useState, useEffect } from 'react';
import { User } from '../store/useAuthStore';
import { Header } from './Header';
import { AlertTriangle, Calendar, Phone, Mail, CheckCircle, Clock, Bell } from
     'lucide-react';

interface CaseManagerDashboardProps {
  user: User;
  onLogout: () =&gt; void;
  onNavigate: (page: 'dashboard' | 'about') =&gt; void;
}

interface HighRiskPatient {
  id: string;
  name: string;
  age: number;
  phone: string;
  email: string;
  midasScore: number;
  headacheDays: number;
  lastVisit: string;
  nextAppointment: string | null;
  riskFactors: string[];
  status: 'pending' | 'contacted' | 'scheduled';
  notes?: string;
}

const mockHighRiskPatients: HighRiskPatient[] = [
  {
    id: '1',
    name: '■■■',
    age: 35,
    phone: '0912-345-678',
    email: 'wang@example.com',
    midasScore: 24,
    headacheDays: 15,
    lastVisit: '2024-11-20',
    nextAppointment: null,
    riskFactors: ['MIDAS Grade IV', '■■■■■■■', '■■■■■■■'],
    status: 'pending',
  },
  {
    id: '2',
    name: '■■■',
    age: 42,
    phone: '0923-456-789',
    email: 'zhang@example.com',
    midasScore: 22,
    headacheDays: 15,
    lastVisit: '2024-11-18',
    nextAppointment: '2024-12-05',
    riskFactors: ['MIDAS Grade IV', '■■■■■■■'],
    status: 'scheduled',
  },
  {
    id: '3',
    name: '■■■',
    age: 38,
    phone: '0934-567-890',
    email: 'li@example.com',
    midasScore: 19,
    headacheDays: 12,
    lastVisit: '2024-11-15',
    nextAppointment: null,
    riskFactors: ['■■■■■■■', '■■■■■■'],
    status: 'contacted',
  },
  {
    id: '4',
    name: '■■■',
    age: 45,
    phone: '0945-678-901',
    email: 'chen@example.com',
    midasScore: 21,
    headacheDays: 14,
    lastVisit: '2024-11-10',
    nextAppointment: null,
    riskFactors: ['MIDAS Grade IV', '■■■■■■■'],
    status: 'pending',
```

```
  },
];

export function CaseManagerDashboard({ user, onLogout, onNavigate }:
     CaseManagerDashboardProps) {
  const [patients, setPatients] = useState&lt;HighRiskPatient[]&gt;([]);
  const [selectedPatient, setSelectedPatient] = useState&lt;HighRiskPatient | null&gt;(null);
  const [notes, setNotes] = useState('');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState&lt;string | null&gt;(null);
  const [showDatePicker, setShowDatePicker] = useState(false);

  const fetchPatients = async () =&gt; {
    try {
      setLoading(true);
      const response = await fetch(`http://${window.location.hostname}:8080/api/casemanage
    r/patients`);
      if (response.ok) {
        const data = await response.json();
        setPatients(data);
        setError(null);
      } else {
        setError('[redacted]');
      }
    } catch (err) {
      console.error(err);
      setError('[redacted] Spring Boot [redacted]');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() =&gt; {
    fetchPatients();
  }, []);

  useEffect(() =&gt; {
    if (selectedPatient) {
      setNotes(selectedPatient.notes || '');
    } else {
      setNotes('');
    }
    setShowDatePicker(false);
  }, [selectedPatient]);

  const handleStatusChange = async (patientId: string, newStatus: 'pending' | 'contacted'
     | 'scheduled') =&gt; {
    try {
      const response = await fetch(`http://${window.location.hostname}:8080/api/casemanage
    r/tracking/${patientId}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ status: newStatus }),
      });
      if (response.ok) {
        setPatients((prev) =&gt;
          prev.map((p) =&gt; (p.id === patientId ? { ...p, status: newStatus } : p))
        );
        if (selectedPatient &amp;&amp; selectedPatient.id === patientId) {
          setSelectedPatient((prev) =&gt; prev ? { ...prev, status: newStatus } : null);
        }
      } else {
        alert('[redacted]');
      }
    } catch (err) {
      console.error(err);
      alert('[redacted]');
    }
  };

  const handleScheduleAppointment = async (patientId: string, date: string) =&gt; {
    try {
      const response = await fetch(`http://${window.location.hostname}:8080/api/casemanage
    r/tracking/${patientId}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ nextAppointment: date, status: 'scheduled' }),
      });
      if (response.ok) {
        setPatients((prev) =&gt;
          prev.map((p) =&gt;
            p.id === patientId ? { ...p, nextAppointment: date, status: 'scheduled' } : p
```

```
        )
      );
      if (selectedPatient &amp;amp;&amp;amp; selectedPatient.id === patientId) {
        setSelectedPatient((prev) =&gt;
          prev ? { ...prev, nextAppointment: date, status: 'scheduled' } : null
        );
      }
      alert('■■■■■■■■■■');
    } else {
      alert('■■■■■■■');
    }
  } catch (err) {
    console.error(err);
    alert('■■■■■■■■■■■');
  }
};

const handleSaveNotes = async () =&gt; {
  if (!selectedPatient) return;
  try {
    const response = await fetch(`http://${window.location.hostname}:8080/api/casemanage
  r/tracking/${selectedPatient.id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ notes }),
    });
    if (response.ok) {
      setPatients((prev) =&gt;
        prev.map((p) =&gt; (p.id === selectedPatient.id ? { ...p, notes } : p))
      );
      setSelectedPatient((prev) =&gt; prev ? { ...prev, notes } : null);
      alert('■■■■■■■■');
    } else {
      alert('■■■■■■■');
    }
  } catch (err) {
    console.error(err);
    alert('■■■■■■■■■■■');
  }
};

const getStatusColor = (status: string) =&gt; {
  switch (status) {
    case 'pending':
      return 'text-danger bg-red-100';
    case 'contacted':
      return 'text-warning bg-yellow-100';
    case 'scheduled':
      return 'text-success bg-green-100';
    default:
      return 'text-text-secondary bg-gray-100';
  }
};

const getStatusLabel = (status: string) =&gt; {
  switch (status) {
    case 'pending':
      return '■■■';
    case 'contacted':
      return '■■■';
    case 'scheduled':
      return '■■■';
    default:
      return '■■';
  }
};

const pendingCount = patients.filter((p) =&gt; p.status === 'pending').length;
const contactedCount = patients.filter((p) =&gt; p.status === 'contacted').length;
const scheduledCount = patients.filter((p) =&gt; p.status === 'scheduled').length;

return (
  &lt;div className="min-h-screen bg-background"&gt;
    &lt;Header user={user} onLogout={onLogout} onNavigate={onNavigate} /&gt;

    &lt;div className="max-w-full mx-auto px-4 py-8"&gt;
      &lt;div className="mb-8"&gt;
        &lt;h1 className="text-text-primary mb-2"&gt;■■■■■■■■■&lt;/h1&gt;
        &lt;p className="text-text-secondary"&gt;■■■■■■■■■■■■■&lt;/p&gt;
      &lt;/div&gt;

      {/* Summary Cards */}
```

```
   &lt;div className="grid grid-cols-1 md:grid-cols-3 gap-4 mb-8"&gt;
     &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border"&gt;
       &lt;div className="flex items-center justify-between mb-2"&gt;
         &lt;div className="w-12 h-12 bg-red-100 rounded-lg flex items-center
justify-center"&gt;
           &lt;AlertTriangle className="w-6 h-6 text-danger" /&gt;
         &lt;/div&gt;
       &lt;/div&gt;
       &lt;div className="text-text-secondary"&gt;■■■&lt;/div&gt;
       &lt;h3 className="text-danger"&gt;{pendingCount} ■&lt;/h3&gt;
     &lt;/div&gt;

     &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border"&gt;
       &lt;div className="flex items-center justify-between mb-2"&gt;
         &lt;div className="w-12 h-12 bg-yellow-100 rounded-lg flex items-center
justify-center"&gt;
           &lt;Clock className="w-6 h-6 text-warning" /&gt;
         &lt;/div&gt;
       &lt;/div&gt;
       &lt;div className="text-text-secondary"&gt;■■■&lt;/div&gt;
       &lt;h3 className="text-warning"&gt;{contactedCount} ■&lt;/h3&gt;
     &lt;/div&gt;

     &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border"&gt;
       &lt;div className="flex items-center justify-between mb-2"&gt;
         &lt;div className="w-12 h-12 bg-green-100 rounded-lg flex items-center
justify-center"&gt;
           &lt;CheckCircle className="w-6 h-6 text-success" /&gt;
         &lt;/div&gt;
       &lt;/div&gt;
       &lt;div className="text-text-secondary"&gt;■■■&lt;/div&gt;
       &lt;h3 className="text-success"&gt;{scheduledCount} ■&lt;/h3&gt;
     &lt;/div&gt;
   &lt;/div&gt;

   &lt;div className="grid grid-cols-1 lg:grid-cols-2 gap-6"&gt;
     {/* Patient List */}
     &lt;div className="space-y-4"&gt;
       &lt;h3 className="text-text-primary"&gt;■■■■■■■■&lt;/h3&gt;
       {loading ? (
         &lt;div className="bg-white rounded-xl shadow-md p-8 border border-border flex
items-center justify-center h-64"&gt;
           &lt;div className="text-center text-text-secondary"&gt;
             &lt;div className="animate-spin rounded-full h-10 w-10 border-b-2
border-primary mx-auto mb-4"&gt;&lt;/div&gt;
             &lt;p&gt;■■■■■■■■...&lt;/p&gt;
           &lt;/div&gt;
         &lt;/div&gt;
       ) : error ? (
         &lt;div className="bg-white rounded-xl shadow-md p-8 border border-border
text-center text-danger"&gt;
           &lt;AlertTriangle className="w-12 h-12 mx-auto mb-3 text-danger opacity-85"
/&gt;
           &lt;p className="font-semibold mb-2"&gt;{error}&lt;/p&gt;
           &lt;button
             onClick={fetchPatients}
             className="mt-2 px-5 py-2 bg-primary hover:bg-primary-dark text-white
rounded-lg text-sm transition-all"
           &gt;
             ■■■■
           &lt;/button&gt;
         &lt;/div&gt;
       ) : patients.length === 0 ? (
         &lt;div className="bg-white rounded-xl shadow-md p-8 border border-border
text-center text-text-secondary"&gt;
           &lt;AlertTriangle className="w-12 h-12 mx-auto mb-3 opacity-30" /&gt;
           &lt;p&gt;■■■■■■■■■■■■■&lt;/p&gt;
         &lt;/div&gt;
       ) : (
         patients.map((patient) =&gt; (
           &lt;div
             key={patient.id}
             className={`bg-white rounded-xl shadow-md p-6 border-2 transition-all
cursor-pointer ${
               selectedPatient?.id === patient.id
                 ? 'border-primary'
                 : 'border-border hover:border-primary'
             }`}
             onClick={() =&gt; setSelectedPatient(patient)}
           &gt;
             &lt;div className="flex items-center justify-between mb-3"&gt;
               &lt;div&gt;
```

```
                &lt;h4 className="text-text-primary"&gt;{patient.name}&lt;/h4&gt;
                &lt;p className="text-text-secondary"&gt;{patient.age}■&lt;/p&gt;
              &lt;/div&gt;
              &lt;span className={`px-3 py-1 rounded-full
${getStatusColor(patient.status)}`}&gt;
                {getStatusLabel(patient.status)}
              &lt;/span&gt;
            &lt;/div&gt;

            &lt;div className="space-y-2 mb-3"&gt;
              &lt;div className="flex items-center gap-2 text-text-secondary"&gt;
                &lt;AlertTriangle className="w-4 h-4 text-danger" /&gt;
                &lt;span&gt;MIDAS: {patient.midasScore} | ■■■■:
{patient.headacheDays}&lt;/span&gt;
              &lt;/div&gt;
              &lt;div className="flex items-center gap-2 text-text-secondary"&gt;
                &lt;Calendar className="w-4 h-4" /&gt;
                &lt;span&gt;■■■■: {patient.lastVisit}&lt;/span&gt;
              &lt;/div&gt;
              {patient.nextAppointment &amp;&amp; (
                &lt;div className="flex items-center gap-2 text-success"&gt;
                  &lt;Bell className="w-4 h-4" /&gt;
                  &lt;span&gt;■■■■: {patient.nextAppointment}&lt;/span&gt;
                &lt;/div&gt;
              )}
            &lt;/div&gt;

            &lt;div className="flex flex-wrap gap-2"&gt;
              {patient.riskFactors.map((factor, idx) =&gt; (
                &lt;span key={idx} className="px-2 py-1 bg-red-50 text-danger rounded"&gt;
                  {factor}
                &lt;/span&gt;
              ))}
            &lt;/div&gt;
          &lt;/div&gt;
        ))
      )}
    &lt;/div&gt;

    {/* Patient Detail */}
    {selectedPatient ? (
      &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border sticky
top-24 self-start"&gt;
        &lt;h3 className="text-text-primary mb-6"&gt;■■■■ - {selectedPatient.name}&lt;/h3&gt;

        &lt;div className="space-y-4 mb-6"&gt;
          &lt;div className="grid grid-cols-2 gap-4"&gt;
            &lt;div&gt;
              &lt;div className="text-text-secondary mb-1"&gt;■■&lt;/div&gt;
              &lt;div className="text-text-primary"&gt;{selectedPatient.age}■&lt;/div&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;div className="text-text-secondary mb-1"&gt;MIDAS ■■&lt;/div&gt;
              &lt;div className="text-danger"&gt;{selectedPatient.midasScore}&lt;/div&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;div className="text-text-secondary mb-1"&gt;■■■■■&lt;/div&gt;
              &lt;div className="text-danger"&gt;{selectedPatient.headacheDays}&lt;/div&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;div className="text-text-secondary mb-1"&gt;■■■■&lt;/div&gt;
              &lt;div className="text-text-primary"&gt;{selectedPatient.lastVisit}&lt;/div&gt;
            &lt;/div&gt;
          &lt;/div&gt;

          &lt;div&gt;
            &lt;div className="text-text-secondary mb-1"&gt;■■■■&lt;/div&gt;
            &lt;div className="flex items-center gap-2 text-text-primary mb-1"&gt;
              &lt;Phone className="w-4 h-4" /&gt;
              {selectedPatient.phone}
            &lt;/div&gt;
            &lt;div className="flex items-center gap-2 text-text-primary"&gt;
              &lt;Mail className="w-4 h-4" /&gt;
              {selectedPatient.email}
            &lt;/div&gt;
          &lt;/div&gt;

          &lt;div&gt;
            &lt;div className="text-text-secondary mb-2"&gt;■■■■&lt;/div&gt;
            &lt;div className="flex flex-wrap gap-2"&gt;
              {selectedPatient.riskFactors.map((factor, idx) =&gt; (
                &lt;span key={idx} className="px-3 py-1 bg-red-50 text-danger
```

```
rounded-lg"&gt;
                  {factor}
                &lt;/span&gt;
              ))}
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/div&gt;

        {/* Actions */}
        &lt;div className="space-y-3 mb-6"&gt;
          &lt;h4 className="text-text-primary"&gt;■■■■&lt;/h4&gt;
          &lt;div className="flex gap-2"&gt;
            {selectedPatient.status === 'contacted' ? (
              &lt;button
                onClick={() =&gt; handleStatusChange(selectedPatient.id, 'pending')}
                className="flex-1 px-4 py-2 border border-yellow-200 bg-yellow-50
hover:bg-yellow-100 text-yellow-700 rounded-lg transition-all"
              &gt;
                ■■■■■
              &lt;/button&gt;
            ) : (
              &lt;button
                onClick={() =&gt; handleStatusChange(selectedPatient.id, 'contacted')}
                className="flex-1 px-4 py-2 border border-border hover:bg-gray-50
rounded-lg transition-all"
                disabled={selectedPatient.status !== 'pending'}
              &gt;
                ■■■■■
              &lt;/button&gt;
            )}
            &lt;div className="flex-1 relative"&gt;
              &lt;button
                onClick={() =&gt; setShowDatePicker(!showDatePicker)}
                className="w-full px-4 py-2 bg-primary hover:bg-primary-dark
text-white rounded-lg transition-all"
              &gt;
                ■■■■
              &lt;/button&gt;
              {showDatePicker &amp;&amp; (
                &lt;div className="absolute right-0 bottom-full mb-2 bg-white border
border-border rounded-xl shadow-xl p-4 z-20 w-64"&gt;
                  &lt;label className="block text-xs font-semibold text-text-primary
mb-2"&gt;■■■■■■■&lt;/label&gt;
                  &lt;input
                    type="date"
                    min={new Date().toISOString().split('T')[0]}
                    onChange={(e) =&gt; {
                      const date = e.target.value;
                      if (date) {
                        handleScheduleAppointment(selectedPatient.id, date);
                        setShowDatePicker(false);
                      }
                    }}
                    className="w-full px-3 py-2 border border-border rounded-lg
text-sm text-text-primary focus:outline-none focus:ring-2 focus:ring-primary
bg-gray-50 mb-2 cursor-pointer"
                  /&gt;
                  &lt;button
                    onClick={() =&gt; setShowDatePicker(false)}
                    className="w-full text-xs text-center text-text-secondary
hover:text-text-primary py-1 border-t border-border mt-2"
                  &gt;
                    ■■
                  &lt;/button&gt;
                &lt;/div&gt;
              )}
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/div&gt;

        {/* Notes */}
        &lt;div&gt;
          &lt;label className="block text-text-primary mb-2"&gt;■■■■&lt;/label&gt;
          &lt;textarea
            value={notes}
            onChange={(e) =&gt; setNotes(e.target.value)}
            className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary resize-none"
            rows={4}
            placeholder="■■■■■■■■■■■■■..."
          /&gt;
          &lt;button
```

```
                onClick={handleSaveNotes}
                className="w-full mt-3 px-4 py-2 bg-success hover:bg-green-600
    text-white rounded-lg transition-all"
              &gt;
                ■■■■
              &lt;/button&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        ) : (
          &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border flex
    items-center justify-center h-96"&gt;
            &lt;div className="text-center text-text-secondary"&gt;
              &lt;AlertTriangle className="w-16 h-16 mx-auto mb-4 opacity-50" /&gt;
              &lt;p&gt;■■■■■■■■■■&lt;/p&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        )}
      &lt;/div&gt;
    &lt;/div&gt;
  &lt;/div&gt;
 );
}
```

■ File: src\components\DoctorDashboard.tsx

```
import { useState, useEffect } from 'react';
import { User } from '../store/useAuthStore';
import { Header } from './Header';
import { PatientRecordModal } from './PatientRecordModal';
import { PrescriptionModal } from './PrescriptionModal';
import { Users, TrendingUp, FileText, Search } from 'lucide-react';
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, Legend,
     ResponsiveContainer, BarChart, Bar } from 'recharts';
import { toast } from 'sonner';

interface DoctorDashboardProps {
  user: User;
  onLogout: () =&gt; void;
  onNavigate: (page: 'dashboard' | 'about') =&gt; void;
}

interface Patient {
  id: string;
  name: string;
  age: number;
  gender: string;
  lastVisit: string;
  midasScore: number;
  headacheDays: number;
  riskLevel: 'low' | 'medium' | 'high';
}

const patientTrendData = [
  { month: '6■', avgMidas: 14, avgHeadacheDays: 9 },
  { month: '7■', avgMidas: 15, avgHeadacheDays: 10 },
  { month: '8■', avgMidas: 13, avgHeadacheDays: 8 },
  { month: '9■', avgMidas: 16, avgHeadacheDays: 11 },
  { month: '10■', avgMidas: 14, avgHeadacheDays: 9 },
  { month: '11■', avgMidas: 15, avgHeadacheDays: 10 },
];

export function DoctorDashboard({ user, onLogout, onNavigate }: DoctorDashboardProps) {
  const [patients, setPatients] = useState&lt;Patient[]&gt;([]);
  const [selectedPatient, setSelectedPatient] = useState&lt;Patient | null&gt;(null);
  const [searchTerm, setSearchTerm] = useState('');
  const [showRecordModal, setShowRecordModal] = useState(false);
  const [showPrescriptionModal, setShowPrescriptionModal] = useState(false);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() =&gt; {
    fetch(`http://${window.location.hostname}:8080/api/users/patients`)
      .then(res =&gt; res.json())
      .then(data =&gt; setPatients(data))
      .catch(err =&gt; console.error("Failed to fetch patients", err))
      .finally(() =&gt; setIsLoading(false));
  }, []);

  const filteredPatients = patients.filter((patient) =&gt;
    patient.name.toLowerCase().includes(searchTerm.toLowerCase())
  );

  const getRiskColor = (level: string) =&gt; {
    switch (level) {
      case 'high':
        return 'text-danger bg-red-100';
      case 'medium':
        return 'text-warning bg-yellow-100';
      case 'low':
        return 'text-success bg-green-100';
      default:
        return 'text-text-secondary bg-gray-100';
    }
  };

  const getRiskLabel = (level: string) =&gt; {
    switch (level) {
      case 'high':
        return '■■■';
      case 'medium':
        return '■■■';
      case 'low':
        return '■■■';
      default:
        return '■■■';
```

```
  }
};

const patientDetailData = selectedPatient
  ? [
      { date: '11/18', intensity: 7, duration: 5 },
      { date: '11/20', intensity: 8, duration: 6 },
      { date: '11/22', intensity: 6, duration: 4 },
      { date: '11/24', intensity: 7, duration: 5 },
      { date: '11/25', intensity: 5, duration: 3 },
    ]
  : [];

const handleViewRecord = () =&gt; {
  if (selectedPatient) {
    setShowRecordModal(true);
  }
};

const handleOpenPrescription = () =&gt; {
  if (selectedPatient) {
    setShowPrescriptionModal(true);
  }
};

const handlePrescriptionSubmit = async (prescription: any) =&gt; {
  if (!selectedPatient || !user) return;
  try {
      await fetch(`http://${window.location.hostname}:8080/api/prescriptions`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({
              doctorId: user.id,
              patientId: selectedPatient.id,
              medicationDetails: prescription.medication,
              instructions: prescription.notes || prescription.instructions
          })
      });
      toast.success('■■■■■■■■■■■■■■■■');
  } catch (err) {
      console.error("Save prescription failed", err);
      toast.error('■■■■■■■■■■■■■■■');
  }
  setShowPrescriptionModal(false);
};

return (
  &lt;div className="min-h-screen bg-background"&gt;
    &lt;Header user={user} onLogout={onLogout} onNavigate={onNavigate} /&gt;

    &lt;div className="max-w-full mx-auto px-4 py-8"&gt;
      &lt;div className="mb-8"&gt;
        &lt;h1 className="text-text-primary mb-2"&gt;■■■■&lt;/h1&gt;
        &lt;p className="text-text-secondary"&gt;■■■■■■■■■■&lt;/p&gt;
      &lt;/div&gt;

      {/* Stats Cards */}
      &lt;div className="grid grid-cols-1 md:grid-cols-3 gap-4 mb-8"&gt;
        &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border"&gt;
          &lt;div className="flex items-center justify-between mb-2"&gt;
            &lt;div className="w-12 h-12 bg-blue-100 rounded-lg flex items-center
   justify-center"&gt;
              &lt;Users className="w-6 h-6 text-primary" /&gt;
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;div className="text-text-secondary"&gt;■■■■■■■&lt;/div&gt;
          &lt;h3 className="text-primary"&gt;{patients.length} ■&lt;/h3&gt;
        &lt;/div&gt;

        &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border"&gt;
          &lt;div className="flex items-center justify-between mb-2"&gt;
            &lt;div className="w-12 h-12 bg-red-100 rounded-lg flex items-center
   justify-center"&gt;
              &lt;TrendingUp className="w-6 h-6 text-danger" /&gt;
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;div className="text-text-secondary"&gt;■■■■■ ({'&gt;'}10■■■)&lt;/div&gt;
          &lt;h3 className="text-danger"&gt;{patients.filter(p =&gt; p.riskLevel ===
   'high').length} ■&lt;/h3&gt;
        &lt;/div&gt;

        &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border"&gt;
```

```
      &lt;div className="flex items-center justify-between mb-2"&gt;
        &lt;div className="w-12 h-12 bg-green-100 rounded-lg flex items-center
justify-center"&gt;
          &lt;FileText className="w-6 h-6 text-success" /&gt;
        &lt;/div&gt;
      &lt;/div&gt;
      &lt;div className="text-text-secondary"&gt;[redacted]&lt;/div&gt;
      &lt;h3 className="text-success"&gt;N/A&lt;/h3&gt;
    &lt;/div&gt;
  &lt;/div&gt;

  &lt;div className="grid grid-cols-1 lg:grid-cols-2 gap-6"&gt;
    {/* Patient List */}
    &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border"&gt;
      &lt;div className="mb-4"&gt;
        &lt;h3 className="text-text-primary mb-4"&gt;[redacted]&lt;/h3&gt;
        &lt;div className="relative"&gt;
          &lt;Search className="absolute left-3 top-1/2 -translate-y-1/2 w-5 h-5
text-text-secondary" /&gt;
          &lt;input
            type="text"
            value={searchTerm}
            onChange={(e) =&gt; setSearchTerm(e.target.value)}
            placeholder="[redacted]..."
            className="w-full pl-10 pr-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
          /&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      &lt;div className="space-y-3 max-h-96 overflow-y-auto"&gt;
        {isLoading ? (
            &lt;div className="text-center text-text-secondary py-8"&gt;[redacted]...&lt;/div&gt;
        ) : filteredPatients.length === 0 ? (
          &lt;div className="text-center text-text-secondary py-8"&gt;[redacted]&lt;/div&gt;
        ) : (
          filteredPatients.map((patient) =&gt; (
            &lt;button
              key={patient.id}
              onClick={() =&gt; setSelectedPatient(patient)}
              className={`w-full text-left p-4 rounded-lg border-2 transition-all ${
                selectedPatient?.id === patient.id
                  ? 'bg-medical-blue border-primary'
                  : 'bg-white border-border hover:border-primary'
              }`}
            &gt;
              &lt;div className="flex items-center justify-between mb-2"&gt;
                &lt;div&gt;
                  &lt;h4 className="text-text-primary"&gt;{patient.name}&lt;/h4&gt;
                  &lt;p className="text-text-secondary"&gt;
                    {patient.age}[redacted] · {patient.gender}
                  &lt;/p&gt;
                &lt;/div&gt;
                &lt;span className={`px-3 py-1 rounded-full
${getRiskColor(patient.riskLevel)}`}&gt;
                  {getRiskLabel(patient.riskLevel)}
                &lt;/span&gt;
              &lt;/div&gt;
              &lt;div className="flex items-center gap-4 text-text-secondary"&gt;
                &lt;span&gt;MIDAS: {patient.midasScore}&lt;/span&gt;
                &lt;span&gt;[redacted]: {patient.headacheDays}&lt;/span&gt;
              &lt;/div&gt;
              &lt;div className="text-text-secondary mt-2"&gt;
                [redacted]: {patient.lastVisit}
              &lt;/div&gt;
            &lt;/button&gt;
          ))
        )}
      &lt;/div&gt;
    &lt;/div&gt;

    {/* Patient Detail or Overall Trend */}
    &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border"&gt;
      {selectedPatient ? (
        &lt;&gt;
          &lt;h3 className="text-text-primary mb-4"&gt;{selectedPatient.name} - [redacted]&lt;/h3&gt;
          &lt;div className="mb-4 p-4 bg-medical-blue rounded-lg"&gt;
            &lt;div className="grid grid-cols-2 gap-4"&gt;
              &lt;div&gt;
                &lt;div className="text-text-secondary"&gt;MIDAS [redacted]&lt;/div&gt;
                &lt;div className="text-primary"&gt;{selectedPatient.midasScore}&lt;/div&gt;
              &lt;/div&gt;
```

```
              &lt;div&gt;
                &lt;div className="text-text-secondary"&gt;[redacted]&lt;/div&gt;
                &lt;div className="text-primary"&gt;{selectedPatient.headacheDays}&lt;/div&gt;
              &lt;/div&gt;
              &lt;div&gt;
                &lt;div className="text-text-secondary"&gt;[redacted]&lt;/div&gt;
                &lt;div className="text-primary"&gt;{selectedPatient.age}[redacted]&lt;/div&gt;
              &lt;/div&gt;
              &lt;div&gt;
                &lt;div className="text-text-secondary"&gt;[redacted]&lt;/div&gt;
                &lt;div className={getRiskColor(selectedPatient.riskLevel).split('
')[0]}&gt;
                  {getRiskLabel(selectedPatient.riskLevel)}
                &lt;/div&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          &lt;/div&gt;

          &lt;ResponsiveContainer width="100%" height={250}&gt;
            &lt;LineChart data={patientDetailData}&gt;
              &lt;CartesianGrid strokeDasharray="3 3" stroke="#e2e8f0" /&gt;
              &lt;XAxis dataKey="date" stroke="#64748b" /&gt;
              &lt;YAxis stroke="#64748b" /&gt;
              &lt;Tooltip
                contentStyle={{
                  backgroundColor: '#ffffff',
                  border: '1px solid #e2e8f0',
                  borderRadius: '8px',
                }}
              /&gt;
              &lt;Legend /&gt;
              &lt;Line type="monotone" dataKey="intensity" stroke="#3b82f6"
name="[redacted]" /&gt;
              &lt;Line type="monotone" dataKey="duration" stroke="#06b6d4"
name="[redacted]" /&gt;
            &lt;/LineChart&gt;
          &lt;/ResponsiveContainer&gt;

          &lt;div className="mt-4 space-y-2"&gt;
            &lt;button
              className="w-full px-4 py-2 bg-primary hover:bg-primary-dark
text-white rounded-lg transition-all"
              onClick={handleViewRecord}
            &gt;
              [redacted]
            &lt;/button&gt;
            &lt;button
              className="w-full px-4 py-2 border border-border hover:bg-gray-50
rounded-lg transition-all"
              onClick={handleOpenPrescription}
            &gt;
              [redacted]
            &lt;/button&gt;
          &lt;/div&gt;
        &lt;/&gt;
      ) : (
        &lt;&gt;
          &lt;h3 className="text-text-primary mb-4"&gt;[redacted]&lt;/h3&gt;
          &lt;ResponsiveContainer width="100%" height={300}&gt;
            &lt;BarChart data={patientTrendData}&gt;
              &lt;CartesianGrid strokeDasharray="3 3" stroke="#e2e8f0" /&gt;
              &lt;XAxis dataKey="month" stroke="#64748b" /&gt;
              &lt;YAxis stroke="#64748b" /&gt;
              &lt;Tooltip
                contentStyle={{
                  backgroundColor: '#ffffff',
                  border: '1px solid #e2e8f0',
                  borderRadius: '8px',
                }}
              /&gt;
              &lt;Legend /&gt;
              &lt;Bar dataKey="avgMidas" fill="#3b82f6" name="[redacted] MIDAS [redacted]" /&gt;
              &lt;Bar dataKey="avgHeadacheDays" fill="#06b6d4" name="[redacted]" /&gt;
            &lt;/BarChart&gt;
          &lt;/ResponsiveContainer&gt;
        &lt;/&gt;
      )}
    &lt;/div&gt;
  &lt;/div&gt;
&lt;/div&gt;

{/* Patient Record Modal */}
```

```
      {showRecordModal &amp;amp;&amp;amp; selectedPatient &amp;amp;&amp;amp; (
        &lt;PatientRecordModal
          patient={selectedPatient}
          onClose={() =&gt; setShowRecordModal(false)}
        /&gt;
      )}

      {/* Prescription Modal */}
      {showPrescriptionModal &amp;amp;&amp;amp; selectedPatient &amp;amp;&amp;amp; (
        &lt;PrescriptionModal
          patient={selectedPatient}
          onClose={() =&gt; setShowPrescriptionModal(false)}
          onSubmit={handlePrescriptionSubmit}
        /&gt;
      )}
    &lt;/div&gt;
  );
}
```

■ File: src\components\HeadacheDiary.tsx

```
import { useState, useEffect } from 'react';
import { Plus, Calendar, Clock, Activity, Edit2, Trash2 } from 'lucide-react';
import { useAuthStore } from '../store/useAuthStore';

interface DiaryEntry {
  id?: string;
  userId?: string;
  date: string;
  time: string;
  intensity: number;
  symptoms: string[];
  medication: string;
  notes: string;
}

export function HeadacheDiary() {
  const currentUser = useAuthStore((state) =&gt; state.currentUser);
  const [entries, setEntries] = useState&lt;DiaryEntry[]&gt;([]);
  const [isLoading, setIsLoading] = useState(false);

  useEffect(() =&gt; {
    if (currentUser?.id) {
      setIsLoading(true);
      fetch(`http://${window.location.hostname}:8080/api/diaries/user/${currentUser.id}`)
        .then(res =&gt; res.json())
        .then(data =&gt; setEntries(data))
        .catch(err =&gt; console.error("Failed to fetch diaries:", err))
        .finally(() =&gt; setIsLoading(false));
    }
  }, [currentUser]);

  const [showForm, setShowForm] = useState(false);
  const [editingId, setEditingId] = useState&lt;string | null&gt;(null);
  const [formData, setFormData] = useState({
    date: new Date().toISOString().split('T')[0],
    time: new Date().toTimeString().slice(0, 5),
    intensity: 5,
    symptoms: [] as string[],
    medication: '',
    notes: '',
  });

  const symptomOptions = [
    '■■■■■',
    '■■■■■',
    '■■',
    '■■',
    '■■',
    '■■',
    '■■',
    '■■■■',
  ];

  const handleSymptomToggle = (symptom: string) =&gt; {
    setFormData((prev) =&gt; ({
      ...prev,
      symptoms: prev.symptoms.includes(symptom)
        ? prev.symptoms.filter((s) =&gt; s !== symptom)
        : [...prev.symptoms, symptom],
    }));
  };

  const handleEdit = (entry: DiaryEntry) =&gt; {
    setEditingId(entry.id || null);
    setFormData({
      date: entry.date,
      time: entry.time,
      intensity: entry.intensity,
      symptoms: entry.symptoms,
      medication: entry.medication,
      notes: entry.notes,
    });
    setShowForm(true);
  };

  const handleDelete = async (id: string | undefined) =&gt; {
    if (!id) return;
    if (confirm('■■■■■■■■■■■■')) {
      try {
```

```
      await fetch(`http://${window.location.hostname}:8080/api/diaries/${id}`, {
   method: 'DELETE' });
      setEntries(entries.filter(entry =&gt; entry.id !== id));
    } catch (err) {
      console.error("Failed to delete", err);
    }
  }
};

const handleSubmit = async (e: React.FormEvent) =&gt; {
  e.preventDefault();
  if (!currentUser) return;

  const entryData: DiaryEntry = {
    ...formData,
    userId: currentUser.id,
  };

  if (editingId) {
    entryData.id = editingId;
  }

  try {
    const res = await fetch(`http://${window.location.hostname}:8080/api/diaries`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(entryData)
    });
    const savedEntry = await res.json();

    if (editingId) {
      setEntries(entries.map(entry =&gt; entry.id === editingId ? savedEntry : entry));
    } else {
      setEntries([savedEntry, ...entries]);
    }
  } catch (err) {
    console.error("Failed to save", err);
  }

  setShowForm(false);
  setEditingId(null);
  setFormData({
    date: new Date().toISOString().split('T')[0],
    time: new Date().toTimeString().slice(0, 5),
    intensity: 5,
    symptoms: [],
    medication: '',
    notes: '',
  });
};

const handleCancelForm = () =&gt; {
  setShowForm(false);
  setEditingId(null);
  setFormData({
    date: new Date().toISOString().split('T')[0],
    time: new Date().toTimeString().slice(0, 5),
    intensity: 5,
    symptoms: [],
    medication: '',
    notes: '',
  });
};

return (
  &lt;div className="space-y-6"&gt;
    {/* Add Entry Button */}
    &lt;div className="flex justify-between items-center"&gt;
      &lt;div&gt;
        &lt;h2 className="text-text-primary"&gt;[redacted]&lt;/h2&gt;
        &lt;p className="text-text-secondary"&gt;[redacted]&lt;/p&gt;
      &lt;/div&gt;
      &lt;button
        onClick={() =&gt; setShowForm(!showForm)}
        className="flex items-center gap-2 px-4 py-2 bg-primary hover:bg-primary-dark
   text-white rounded-lg transition-all shadow-md hover:shadow-lg"
      &gt;
        &lt;Plus className="w-5 h-5" /&gt;
        [redacted]
      &lt;/button&gt;
    &lt;/div&gt;
```

```
 {/* Entry Form */}
 {showForm &amp;&amp; (
   &lt;div className="bg-white rounded-xl shadow-lg p-6 border border-border"&gt;
     &lt;h3 className="mb-6 text-text-primary"&gt;
       {editingId ? '███████' : '███████'}
     &lt;/h3&gt;
     &lt;form onSubmit={handleSubmit} className="space-y-6"&gt;
       &lt;div className="grid grid-cols-1 md:grid-cols-2 gap-6"&gt;
         {/* Date */}
         &lt;div&gt;
           &lt;label className="block mb-2"&gt;██&lt;/label&gt;
           &lt;input
             type="date"
             value={formData.date}
             onChange={(e) =&gt; setFormData({ ...formData, date: e.target.value })}
             className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
             required
           /&gt;
         &lt;/div&gt;

         {/* Time */}
         &lt;div&gt;
           &lt;label className="block mb-2"&gt;██&lt;/label&gt;
           &lt;input
             type="time"
             value={formData.time}
             onChange={(e) =&gt; setFormData({ ...formData, time: e.target.value })}
             className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
             required
           /&gt;
         &lt;/div&gt;
       &lt;/div&gt;

       {/* Intensity */}
       &lt;div&gt;
         &lt;label className="block mb-2"&gt;██████{formData.intensity} / 10&lt;/label&gt;
         &lt;input
           type="range"
           min="1"
           max="10"
           value={formData.intensity}
           onChange={(e) =&gt; setFormData({ ...formData, intensity:
parseInt(e.target.value) })}
           className="w-full h-2 bg-gray-200 rounded-lg appearance-none
cursor-pointer accent-primary"
         /&gt;
         &lt;div className="flex justify-between text-text-secondary mt-1"&gt;
           &lt;span&gt;██&lt;/span&gt;
           &lt;span&gt;██&lt;/span&gt;
         &lt;/div&gt;
       &lt;/div&gt;

       {/* Symptoms */}
       &lt;div&gt;
         &lt;label className="block mb-3"&gt;████████&lt;/label&gt;
         &lt;div className="grid grid-cols-2 md:grid-cols-4 gap-3"&gt;
           {symptomOptions.map((symptom) =&gt; (
             &lt;button
               key={symptom}
               type="button"
               onClick={() =&gt; handleSymptomToggle(symptom)}
               className={`p-3 rounded-lg border-2 transition-all ${
                 formData.symptoms.includes(symptom)
                   ? 'bg-primary text-white border-primary'
                   : 'bg-white text-text-secondary border-border
hover:border-primary'
               }`}
             &gt;
               {symptom}
             &lt;/button&gt;
           ))}
         &lt;/div&gt;
       &lt;/div&gt;

       {/* Medication */}
       &lt;div&gt;
         &lt;label className="block mb-2"&gt;██&lt;/label&gt;
         &lt;input
           type="text"
           value={formData.medication}
```

```
          onChange={(e) =&gt; setFormData({ ...formData, medication: e.target.value })}
          className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
          placeholder="■■■■■ 500mg"
        /&gt;
      &lt;/div&gt;

      {/* Notes */}
      &lt;div&gt;
        &lt;label className="block mb-2"&gt;■■&lt;/label&gt;
        &lt;textarea
          value={formData.notes}
          onChange={(e) =&gt; setFormData({ ...formData, notes: e.target.value })}
          className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary resize-none"
          rows={3}
          placeholder="■■■■■■■■■■■■■■■■■"
        /&gt;
      &lt;/div&gt;

      {/* Buttons */}
      &lt;div className="flex gap-3 justify-end"&gt;
        &lt;button
          type="button"
          onClick={handleCancelForm}
          className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
transition-colors"
        &gt;
          ■■
        &lt;/button&gt;
        &lt;button
          type="submit"
          className="px-6 py-2 bg-primary hover:bg-primary-dark text-white
rounded-lg transition-all shadow-md"
        &gt;
          {editingId ? '■■' : '■■'}
        &lt;/button&gt;
      &lt;/div&gt;
    &lt;/form&gt;
  &lt;/div&gt;
 )}

 {/* Entries List */}
 &lt;div className="space-y-4"&gt;
   {isLoading ? (
     &lt;div className="text-center text-text-secondary py-8"&gt;■■■...&lt;/div&gt;
   ) : entries.length === 0 ? (
     &lt;div className="text-center bg-white rounded-xl shadow-md p-8 border
border-border"&gt;
       &lt;Calendar className="w-12 h-12 text-gray-300 mx-auto mb-4" /&gt;
       &lt;div className="text-text-secondary text-lg"&gt;■■■■&lt;/div&gt;
       &lt;p className="text-gray-400 mt-1"&gt;■■■■■■■■■■■■■■■■■■■■■■&lt;/p&gt;
     &lt;/div&gt;
   ) : (
     entries.map((entry) =&gt; (
       &lt;div key={entry.id} className="bg-white rounded-xl shadow-md p-6 border
border-border hover:shadow-lg transition-shadow"&gt;
         &lt;div className="flex flex-col md:flex-row md:items-center justify-between
gap-4 mb-4"&gt;
           &lt;div className="flex items-center gap-4"&gt;
             &lt;div className="flex items-center gap-2 text-text-secondary"&gt;
               &lt;Calendar className="w-5 h-5" /&gt;
               {entry.date}
             &lt;/div&gt;
             &lt;div className="flex items-center gap-2 text-text-secondary"&gt;
               &lt;Clock className="w-5 h-5" /&gt;
               {entry.time}
             &lt;/div&gt;
           &lt;/div&gt;
           &lt;div className="flex items-center gap-2"&gt;
             &lt;Activity className="w-5 h-5 text-danger" /&gt;
             &lt;span className="text-danger"&gt;■■■■■{entry.intensity}/10&lt;/span&gt;
           &lt;/div&gt;
         &lt;/div&gt;

         &lt;div className="space-y-3"&gt;
           &lt;div&gt;
             &lt;span className="text-text-secondary"&gt;■■■&lt;/span&gt;
             &lt;div className="flex flex-wrap gap-2 mt-2"&gt;
               {entry.symptoms.map((symptom, idx) =&gt; (
                 &lt;span key={idx} className="px-3 py-1 bg-medical-blue text-primary
rounded-full"&gt;
```

```
                    {symptom}
                  &lt;/span&gt;
                ))}
              &lt;/div&gt;
            &lt;/div&gt;

            {entry.medication &amp;&amp; (
              &lt;div&gt;
                &lt;span className="text-text-secondary"&gt;■■■&lt;/span&gt;
                &lt;span className="ml-2 text-text-primary"&gt;{entry.medication}&lt;/span&gt;
              &lt;/div&gt;
            )}

            {entry.notes &amp;&amp; (
              &lt;div&gt;
                &lt;span className="text-text-secondary"&gt;■■■&lt;/span&gt;
                &lt;span className="ml-2 text-text-primary"&gt;{entry.notes}&lt;/span&gt;
              &lt;/div&gt;
            )}
          &lt;/div&gt;

          {/* Action Buttons */}
          &lt;div className="flex gap-2 mt-4 pt-4 border-t border-border"&gt;
            &lt;button
              onClick={() =&gt; handleEdit(entry)}
              className="flex items-center gap-2 px-4 py-2 text-primary
    hover:bg-medical-blue rounded-lg transition-colors"
            &gt;
              &lt;Edit2 className="w-4 h-4" /&gt;
              ■■
            &lt;/button&gt;
            &lt;button
              onClick={() =&gt; handleDelete(entry.id)}
              className="flex items-center gap-2 px-4 py-2 text-danger hover:bg-red-50
    rounded-lg transition-colors"
            &gt;
              &lt;Trash2 className="w-4 h-4" /&gt;
              ■■
            &lt;/button&gt;
          &lt;/div&gt;
        &lt;/div&gt;
        ))
      )}
    &lt;/div&gt;
  &lt;/div&gt;
);
}
```

■ File: src\components\Header.tsx

```
import { User } from '../App';
import { Activity, LogOut, Info, Menu, X } from 'lucide-react';
import { useState } from 'react';

interface HeaderProps {
  user: User;
  onLogout: () =&gt; void;
  onNavigate: (page: 'dashboard' | 'about') =&gt; void;
}

export function Header({ user, onLogout, onNavigate }: HeaderProps) {
  const [mobileMenuOpen, setMobileMenuOpen] = useState(false);

  const getRoleLabel = (role: string) =&gt; {
    switch (role) {
      case 'patient':
        return '■■';
      case 'doctor':
        return '■■';
      case 'caseManager':
        return '■■■■■';
      default:
        return '';
    }
  };

  return (
    &lt;header className="bg-white border-b border-border shadow-sm sticky top-0 z-50"&gt;
      &lt;div className="max-w-full mx-auto px-4"&gt;
        &lt;div className="flex items-center justify-between h-16"&gt;
          {/* Logo */}
          &lt;div className="flex items-center gap-3 cursor-pointer" onClick={() =&gt;
     onNavigate('dashboard')}&gt;
            &lt;div className="w-10 h-10 bg-primary rounded-lg flex items-center
     justify-center"&gt;
              &lt;Activity className="w-6 h-6 text-white" /&gt;
            &lt;/div&gt;
            &lt;div className="hidden sm:block"&gt;
              &lt;h3 className="text-primary"&gt;■■■■■■■■&lt;/h3&gt;
            &lt;/div&gt;
          &lt;/div&gt;

          {/* Desktop Navigation */}
          &lt;div className="hidden md:flex items-center gap-6"&gt;
            &lt;button
              onClick={() =&gt; onNavigate('about')}
              className="flex items-center gap-2 text-text-secondary hover:text-primary
     transition-colors"
            &gt;
              &lt;Info className="w-5 h-5" /&gt;
              ■■■■
            &lt;/button&gt;

            &lt;div className="flex items-center gap-3 px-4 py-2 bg-medical-blue rounded-lg"&gt;
              &lt;div className="w-8 h-8 bg-primary rounded-full flex items-center
     justify-center text-white"&gt;
                {user.name.charAt(0).toUpperCase()}
              &lt;/div&gt;
              &lt;div className="text-left"&gt;
                &lt;div className="text-text-primary"&gt;{user.name}&lt;/div&gt;
                &lt;div className="text-text-secondary"&gt;{getRoleLabel(user.role || '')}&lt;/div&gt;
              &lt;/div&gt;
            &lt;/div&gt;

            &lt;button
              onClick={onLogout}
              className="flex items-center gap-2 px-4 py-2 text-danger hover:bg-red-50
     rounded-lg transition-colors"
            &gt;
              &lt;LogOut className="w-5 h-5" /&gt;
              ■■
            &lt;/button&gt;
          &lt;/div&gt;

          {/* Mobile Menu Button */}
          &lt;button
            onClick={() =&gt; setMobileMenuOpen(!mobileMenuOpen)}
            className="md:hidden p-2 text-text-secondary hover:text-primary"
          &gt;
```

```
            {mobileMenuOpen ? &lt;X className="w-6 h-6" /&gt; : &lt;Menu className="w-6 h-6" /&gt;}
          &lt;/button&gt;
        &lt;/div&gt;

        {/* Mobile Menu */}
        {mobileMenuOpen &amp;&amp; (
          &lt;div className="md:hidden py-4 border-t border-border"&gt;
            &lt;div className="flex flex-col gap-4"&gt;
              &lt;div className="flex items-center gap-3 px-4 py-2 bg-medical-blue
    rounded-lg"&gt;
                &lt;div className="w-8 h-8 bg-primary rounded-full flex items-center
    justify-center text-white"&gt;
                  {user.name.charAt(0).toUpperCase()}
                &lt;/div&gt;
                &lt;div&gt;
                  &lt;div className="text-text-primary"&gt;{user.name}&lt;/div&gt;
                  &lt;div className="text-text-secondary"&gt;{getRoleLabel(user.role ||
    '')}&lt;/div&gt;
                &lt;/div&gt;
              &lt;/div&gt;

              &lt;button
                onClick={() =&gt; {
                  onNavigate('about');
                  setMobileMenuOpen(false);
                }}
                className="flex items-center gap-2 px-4 py-2 text-text-secondary
    hover:text-primary transition-colors"
              &gt;
                &lt;Info className="w-5 h-5" /&gt;
                ■■■■
              &lt;/button&gt;

              &lt;button
                onClick={onLogout}
                className="flex items-center gap-2 px-4 py-2 text-danger hover:bg-red-50
    rounded-lg transition-colors"
              &gt;
                &lt;LogOut className="w-5 h-5" /&gt;
                ■■
              &lt;/button&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        )}
      &lt;/div&gt;
    &lt;/header&gt;
  );
}
```

■ File: src\components\LoginPage.tsx

```
import { useState } from 'react';
import { User, UserRole } from '../store/useAuthStore';
import { Activity } from 'lucide-react';

interface LoginPageProps {
  onLogin: (user: User) =&gt; void;
}

export function LoginPage({ onLogin }: LoginPageProps) {
  const [activeTab, setActiveTab] = useState&lt;'login' | 'register'&gt;('login');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  // Register form states
  const [registerName, setRegisterName] = useState('');
  const [registerEmail, setRegisterEmail] = useState('');
  const [registerPassword, setRegisterPassword] = useState('');
  const [registerRole, setRegisterRole] = useState&lt;UserRole&gt;('patient');

  const handleLogin = async (e: React.FormEvent) =&gt; {
    e.preventDefault();
    if (email &amp;&amp; password) {
      try {
        // ■■■■■ Email / ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        const username = email.includes('@') ? email.split('@')[0].toLowerCase() :
     email.toLowerCase();

        const response = await fetch(`http://${window.location.hostname}:8080/api/auth/log
    in`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ name: username, password }),
        });

        if (response.ok) {
          const userData = await response.json();
          onLogin(userData);
        } else {
          alert('■■■■■■■■■■■■■■■■■■■■\n(■■■■■■■■ patient1 / 1234 ■ doctor1 / 1234)');
        }
      } catch (err) {
        console.error(err);
        alert('■■■■■■ Java ■■■■■■■■■■ Spring Boot ■■■■■■■');
      }
    }
  };

  const handleRegister = async (e: React.FormEvent) =&gt; {
    e.preventDefault();
    if (registerName &amp;&amp; registerEmail &amp;&amp; registerPassword &amp;&amp; registerRole) {
      try {
        // ■■■■■■■■ Email ■■■■■■■■■■■■■■■■■■■■■■ Email ■■■■■■■ bug
        const username = registerEmail.includes('@')
          ? registerEmail.split('@')[0].toLowerCase()
          : registerEmail.toLowerCase();

        const response = await fetch(`http://${window.location.hostname}:8080/api/auth/reg
    ister`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({
            name: username,
            password: registerPassword,
            role: registerRole,
          }),
        });

        if (response.ok) {
          const userData = await response.json();
          alert(`■■■■■■■■■■■■■■${username}\n■■■■■■■■■■■■`);
          onLogin(userData);
        } else {
          const errorData = await response.json();
          alert(`■■■■■■${errorData.message || '■■■■■'}`);
        }
      } catch (err) {
        console.error(err);
        alert('■■■■■■ Java ■■■■■■■■■■ Spring Boot ■■■■■■■');
      }
```

```
  }
};

return (
  &lt;div className="min-h-screen flex items-center justify-center bg-gradient-to-br
   from-blue-50 via-white to-blue-50 p-4"&gt;
    &lt;div className="w-full max-w-md"&gt;
      {/* Logo and Header */}
      &lt;div className="text-center mb-8"&gt;
        &lt;div className="inline-flex items-center justify-center w-16 h-16 bg-primary
   rounded-full mb-4 shadow-lg"&gt;
          &lt;Activity className="w-8 h-8 text-white" strokeWidth={2.5} /&gt;
        &lt;/div&gt;
        &lt;h2 className="text-primary mb-2"&gt;■■■■■■■■■■&lt;/h2&gt;
        &lt;p className="text-text-secondary"&gt;■■■■■■■■■■■■■■&lt;/p&gt;
      &lt;/div&gt;

      {/* Login/Register Card */}
      &lt;div className="bg-white rounded-2xl shadow-xl border border-border
   overflow-hidden"&gt;
        {/* Tabs */}
        &lt;div className="flex border-b border-border"&gt;
          &lt;button
            onClick={() =&gt; setActiveTab('login')}
            className={`flex-1 py-4 transition-all ${
              activeTab === 'login'
                ? 'text-primary border-b-2 border-primary bg-blue-50'
                : 'text-text-secondary hover:bg-gray-50'
            }`}
          &gt;
            ■■
          &lt;/button&gt;
          &lt;button
            onClick={() =&gt; setActiveTab('register')}
            className={`flex-1 py-4 transition-all ${
              activeTab === 'register'
                ? 'text-primary border-b-2 border-primary bg-blue-50'
                : 'text-text-secondary hover:bg-gray-50'
            }`}
          &gt;
            ■■
          &lt;/button&gt;
        &lt;/div&gt;

        {/* Login Form */}
        {activeTab === 'login' &amp;&amp; (
          &lt;div className="p-8"&gt;
            &lt;div className="mb-6"&gt;
              &lt;h3 className="text-text-primary mb-1"&gt;■■■■&lt;/h3&gt;
              &lt;p className="text-text-secondary"&gt;■■■■■■■■■■&lt;/p&gt;
            &lt;/div&gt;

            &lt;form onSubmit={handleLogin} className="space-y-5"&gt;
              &lt;div&gt;
                &lt;label className="block mb-2 text-text-primary"&gt;Email&lt;/label&gt;
                &lt;input
                  type="email"
                  value={email}
                  onChange={(e) =&gt; setEmail(e.target.value)}
                  className="w-full px-4 py-3 border border-border rounded-lg
   focus:outline-none focus:ring-2 focus:ring-primary focus:border-transparent
   transition-all bg-gray-50"
                  placeholder="your@email.com"
                  required
                /&gt;
              &lt;/div&gt;

              &lt;div&gt;
                &lt;label className="block mb-2 text-text-primary"&gt;■■&lt;/label&gt;
                &lt;input
                  type="password"
                  value={password}
                  onChange={(e) =&gt; setPassword(e.target.value)}
                  className="w-full px-4 py-3 border border-border rounded-lg
   focus:outline-none focus:ring-2 focus:ring-primary focus:border-transparent
   transition-all bg-gray-50"
                  placeholder="••••••••"
                  required
                /&gt;
              &lt;/div&gt;

              &lt;button
```

```
            type="submit"
            className="w-full bg-primary hover:bg-primary-dark text-white py-3
rounded-lg transition-all shadow-md hover:shadow-lg"
          &gt;
            ■■
          &lt;/button&gt;
        &lt;/form&gt;
      &lt;/div&gt;
    )}

    {/* Register Form */}
    {activeTab === 'register' &amp;&amp; (
      &lt;div className="p-8"&gt;
        &lt;div className="mb-6"&gt;
          &lt;h3 className="text-text-primary mb-1"&gt;■■■■&lt;/h3&gt;
          &lt;p className="text-text-secondary"&gt;■■■■■■■■■■&lt;/p&gt;
        &lt;/div&gt;

        &lt;form onSubmit={handleRegister} className="space-y-5"&gt;
          &lt;div&gt;
            &lt;label className="block mb-2 text-text-primary"&gt;■■&lt;/label&gt;
            &lt;input
              type="text"
              value={registerName}
              onChange={(e) =&gt; setRegisterName(e.target.value)}
              className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary focus:border-transparent
transition-all bg-gray-50"
              placeholder="■■■■"
              required
            /&gt;
          &lt;/div&gt;

          &lt;div&gt;
            &lt;label className="block mb-2 text-text-primary"&gt;Email&lt;/label&gt;
            &lt;input
              type="email"
              value={registerEmail}
              onChange={(e) =&gt; setRegisterEmail(e.target.value)}
              className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary focus:border-transparent
transition-all bg-gray-50"
              placeholder="your@email.com"
              required
            /&gt;
          &lt;/div&gt;

          &lt;div&gt;
            &lt;label className="block mb-2 text-text-primary"&gt;■■&lt;/label&gt;
            &lt;input
              type="password"
              value={registerPassword}
              onChange={(e) =&gt; setRegisterPassword(e.target.value)}
              className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary focus:border-transparent
transition-all bg-gray-50"
              placeholder="■■ 6 ■■■"
              minLength={6}
              required
            /&gt;
            &lt;p className="text-text-secondary mt-2"&gt;■■■■■■■■■■■■■■■■■&lt;/p&gt;
          &lt;/div&gt;

          &lt;div&gt;
            &lt;label className="block mb-2 text-text-primary"&gt;■■&lt;/label&gt;
            &lt;select
              value={registerRole || ''}
              onChange={(e) =&gt; setRegisterRole(e.target.value as UserRole)}
              className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary focus:border-transparent
transition-all bg-gray-50 cursor-pointer"
              required
            &gt;
              &lt;option value="patient"&gt;■■&lt;/option&gt;
              &lt;option value="doctor"&gt;■■&lt;/option&gt;
              &lt;option value="caseManager"&gt;■■■■■&lt;/option&gt;
            &lt;/select&gt;
          &lt;/div&gt;

          &lt;button
            type="submit"
            className="w-full bg-primary hover:bg-primary-dark text-white py-3
```

```
      rounded-lg transition-all shadow-md hover:shadow-lg"
                 &gt;
                   ■■
                 &lt;/button&gt;
               &lt;/form&gt;
             &lt;/div&gt;
           )}
         &lt;/div&gt;
       &lt;/div&gt;
     &lt;/div&gt;
  );
}
```

■ File: src\components\PatientDashboard.tsx

```
import { useState } from 'react';
import { User } from '../App';
import { Header } from './Header';
import { HeadacheDiary } from './HeadacheDiary';
import { QuestionnaireList } from './QuestionnaireList';
import { PatientStats } from './PatientStats';
import { Calendar, FileText, BarChart3 } from 'lucide-react';

interface PatientDashboardProps {
  user: User;
  onLogout: () =&gt; void;
  onNavigate: (page: 'dashboard' | 'about') =&gt; void;
}

type TabType = 'diary' | 'questionnaire' | 'stats';

export function PatientDashboard({ user, onLogout, onNavigate }: PatientDashboardProps) {
  const [activeTab, setActiveTab] = useState&lt;TabType&gt;('diary');

  const tabs = [
    { id: 'diary', label: '■■■■', icon: Calendar },
    { id: 'questionnaire', label: '■■■■', icon: FileText },
    { id: 'stats', label: '■■■■', icon: BarChart3 },
  ];

  return (
    &lt;div className="min-h-screen bg-background"&gt;
      &lt;Header user={user} onLogout={onLogout} onNavigate={onNavigate} /&gt;

      &lt;div className="max-w-full mx-auto px-4 py-8"&gt;
        {/* Welcome Section */}
        &lt;div className="mb-8"&gt;
          &lt;h1 className="text-text-primary mb-2"&gt;■■■■■{user.name}&lt;/h1&gt;
          &lt;p className="text-text-secondary"&gt;■■■■■■■■■■■■■■&lt;/p&gt;
        &lt;/div&gt;

        {/* Tabs */}
        &lt;div className="flex gap-4 mb-6 border-b border-border overflow-x-auto"&gt;
          {tabs.map((tab) =&gt; {
            const Icon = tab.icon;
            return (
              &lt;button
                key={tab.id}
                onClick={() =&gt; setActiveTab(tab.id as TabType)}
                className={`flex items-center gap-2 px-6 py-3 transition-all
    whitespace-nowrap ${
                  activeTab === tab.id
                    ? 'border-b-2 border-primary text-primary'
                    : 'text-text-secondary hover:text-primary'
                }`}
              &gt;
                &lt;Icon className="w-5 h-5" /&gt;
                {tab.label}
              &lt;/button&gt;
            );
          })}
        &lt;/div&gt;

        {/* Content */}
        &lt;div className="mt-6"&gt;
          {activeTab === 'diary' &amp;&amp; &lt;HeadacheDiary /&gt;}
          {activeTab === 'questionnaire' &amp;&amp; &lt;QuestionnaireList /&gt;}
          {activeTab === 'stats' &amp;&amp; &lt;PatientStats /&gt;}
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

■ File: src\components\PatientRecordModal.tsx

```
import { X, Calendar, Activity, Pill, FileText, TrendingDown, AlertCircle } from
     'lucide-react';

interface Patient {
  id: string;
  name: string;
  age: number;
  gender: string;
  lastVisit: string;
  midasScore: number;
  headacheDays: number;
  riskLevel: 'low' | 'medium' | 'high';
}

interface DiaryEntry {
  id: string;
  date: string;
  time: string;
  intensity: number;
  symptoms: string[];
  medication: string;
  notes: string;
}

interface QuestionnaireScore {
  name: string;
  score: number;
  maxScore: number;
  date: string;
  interpretation: string;
}

interface PatientRecordModalProps {
  patient: Patient;
  onClose: () =&gt; void;
}

const mockDiaryEntries: DiaryEntry[] = [
  {
    id: '1',
    date: '2024-11-25',
    time: '14:30',
    intensity: 7,
    symptoms: ['██████', '██', '██'],
    medication: '███ 500mg',
    notes: '██████',
  },
  {
    id: '2',
    date: '2024-11-23',
    time: '09:15',
    intensity: 5,
    symptoms: ['██████', '██'],
    medication: '███ 250mg',
    notes: '████',
  },
  {
    id: '3',
    date: '2024-11-20',
    time: '18:45',
    intensity: 8,
    symptoms: ['██████', '██', '██', '██'],
    medication: '███ 500mg + ███',
    notes: '█████',
  },
];

const mockQuestionnaireScores: QuestionnaireScore[] = [
  {
    name: 'MIDAS',
    score: 18,
    maxScore: 100,
    date: '2024-11-15',
    interpretation: '████'
  },
  {
    name: 'HADS-██',
    score: 12,
    maxScore: 21,
```

```
    date: '2024-11-15',
    interpretation: '[redacted]'
  },
  {
    name: 'HADS-[redacted]',
    score: 8,
    maxScore: 21,
    date: '2024-11-15',
    interpretation: '[redacted]'
  },
  {
    name: 'PSQI',
    score: 9,
    maxScore: 21,
    date: '2024-11-15',
    interpretation: '[redacted]'
  },
];

export function PatientRecordModal({ patient, onClose }: PatientRecordModalProps) {
  return (
    &lt;div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50 p-4"&gt;
      &lt;div className="bg-white rounded-xl shadow-2xl max-w-4xl w-full max-h-[90vh]
     overflow-y-auto"&gt;
        {/* Header */}
        &lt;div className="sticky top-0 bg-white border-b border-border p-6 flex
     items-center justify-between"&gt;
          &lt;div&gt;
            &lt;h2 className="text-text-primary"&gt;{patient.name} - [redacted]&lt;/h2&gt;
            &lt;p className="text-text-secondary"&gt;
              {patient.age}[redacted] · {patient.gender} · [redacted]{patient.lastVisit}
            &lt;/p&gt;
          &lt;/div&gt;
          &lt;button
            onClick={onClose}
            className="p-2 hover:bg-gray-100 rounded-lg transition-colors"
          &gt;
            &lt;X className="w-6 h-6 text-text-secondary" /&gt;
          &lt;/button&gt;
        &lt;/div&gt;

        {/* Content */}
        &lt;div className="p-6 space-y-6"&gt;
          {/* Patient Summary */}
          &lt;div className="bg-medical-blue rounded-xl p-6"&gt;
            &lt;h3 className="text-text-primary mb-4"&gt;[redacted]&lt;/h3&gt;
            &lt;div className="grid grid-cols-2 md:grid-cols-4 gap-4"&gt;
              &lt;div&gt;
                &lt;div className="text-text-secondary"&gt;MIDAS [redacted]&lt;/div&gt;
                &lt;div className="text-primary"&gt;{patient.midasScore}&lt;/div&gt;
              &lt;/div&gt;
              &lt;div&gt;
                &lt;div className="text-text-secondary"&gt;[redacted]&lt;/div&gt;
                &lt;div className="text-primary"&gt;{patient.headacheDays} [redacted]&lt;/div&gt;
              &lt;/div&gt;
              &lt;div&gt;
                &lt;div className="text-text-secondary"&gt;[redacted]&lt;/div&gt;
                &lt;div className={
                  patient.riskLevel === 'high' ? 'text-danger' :
                  patient.riskLevel === 'medium' ? 'text-warning' :
                  'text-success'
                }&gt;
                  {patient.riskLevel === 'high' ? '[redacted]' :
                   patient.riskLevel === 'medium' ? '[redacted]' :
                   '[redacted]'}
                &lt;/div&gt;
              &lt;/div&gt;
              &lt;div&gt;
                &lt;div className="text-text-secondary"&gt;[redacted]&lt;/div&gt;
                &lt;div className="text-primary"&gt;6.7 / 10&lt;/div&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          &lt;/div&gt;

          {/* Questionnaire Scores */}
          &lt;div&gt;
            &lt;h3 className="text-text-primary mb-4 flex items-center gap-2"&gt;
              &lt;FileText className="w-5 h-5" /&gt;
              [redacted]
            &lt;/h3&gt;
            &lt;div className="grid grid-cols-1 md:grid-cols-2 gap-4"&gt;
              {mockQuestionnaireScores.map((score, idx) =&gt; (
```

```
          &lt;div key={idx} className="bg-white border border-border rounded-lg p-4"&gt;
            &lt;div className="flex items-center justify-between mb-2"&gt;
              &lt;span className="text-text-primary"&gt;{score.name}&lt;/span&gt;
              &lt;span className="text-text-secondary"&gt;{score.date}&lt;/span&gt;
            &lt;/div&gt;
            &lt;div className="mb-2"&gt;
              &lt;div className="flex items-center justify-between mb-1"&gt;
                &lt;span className="text-primary"&gt;{score.score} /
{score.maxScore}&lt;/span&gt;
                &lt;span className="text-text-secondary"&gt;
                  {Math.round((score.score / score.maxScore) * 100)}%
                &lt;/span&gt;
              &lt;/div&gt;
              &lt;div className="w-full bg-gray-200 rounded-full h-2"&gt;
                &lt;div
                  className="bg-primary h-2 rounded-full"
                  style={{ width: `${(score.score / score.maxScore) * 100}%` }}
                /&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;div className="text-text-secondary"&gt;{score.interpretation}&lt;/div&gt;
          &lt;/div&gt;
        ))}
      &lt;/div&gt;
    &lt;/div&gt;

    {/* Headache Diary */}
    &lt;div&gt;
      &lt;h3 className="text-text-primary mb-4 flex items-center gap-2"&gt;
        &lt;Activity className="w-5 h-5" /&gt;
        ■■■■■■
      &lt;/h3&gt;
      &lt;div className="space-y-3"&gt;
        {mockDiaryEntries.map((entry) =&gt; (
          &lt;div key={entry.id} className="bg-white border border-border rounded-lg
p-4"&gt;
            &lt;div className="flex items-center justify-between mb-3"&gt;
              &lt;div className="flex items-center gap-4"&gt;
                &lt;div className="flex items-center gap-2 text-text-secondary"&gt;
                  &lt;Calendar className="w-4 h-4" /&gt;
                  {entry.date} {entry.time}
                &lt;/div&gt;
                &lt;div className="flex items-center gap-2 text-danger"&gt;
                  &lt;Activity className="w-4 h-4" /&gt;
                  ■■■{entry.intensity}/10
                &lt;/div&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;div className="space-y-2"&gt;
              &lt;div&gt;
                &lt;span className="text-text-secondary"&gt;■■■&lt;/span&gt;
                &lt;div className="flex flex-wrap gap-2 mt-1"&gt;
                  {entry.symptoms.map((symptom, idx) =&gt; (
                    &lt;span key={idx} className="px-2 py-1 bg-medical-blue
text-primary rounded-full text-sm"&gt;
                      {symptom}
                    &lt;/span&gt;
                  ))}
                &lt;/div&gt;
              &lt;/div&gt;
              {entry.medication &amp;&amp; (
                &lt;div&gt;
                  &lt;span className="text-text-secondary"&gt;■■■&lt;/span&gt;
                  &lt;span className="ml-2 text-text-primary"&gt;{entry.medication}&lt;/span&gt;
                &lt;/div&gt;
              )}
              {entry.notes &amp;&amp; (
                &lt;div&gt;
                  &lt;span className="text-text-secondary"&gt;■■■&lt;/span&gt;
                  &lt;span className="ml-2 text-text-primary"&gt;{entry.notes}&lt;/span&gt;
                &lt;/div&gt;
              )}
            &lt;/div&gt;
          &lt;/div&gt;
        ))}
      &lt;/div&gt;
    &lt;/div&gt;

    {/* Clinical Notes Section */}
    &lt;div&gt;
      &lt;h3 className="text-text-primary mb-4 flex items-center gap-2"&gt;
        &lt;AlertCircle className="w-5 h-5" /&gt;
```

```
            ■■■■
          &lt;/h3&gt;
          &lt;div className="bg-yellow-50 border border-warning/30 rounded-lg p-4
    space-y-2"&gt;
            &lt;div className="flex items-start gap-2"&gt;
              &lt;AlertCircle className="w-5 h-5 text-warning flex-shrink-0 mt-0.5" /&gt;
              &lt;div&gt;
                &lt;p className="text-text-primary"&gt;■■■■■■■■&lt;/p&gt;
                &lt;ul className="list-disc list-inside text-text-secondary space-y-1
    mt-2"&gt;
                  &lt;li&gt;■■■■■■■■■■■■■■■■■■■■■■&lt;/li&gt;
                  &lt;li&gt;PSQI ■■■■■■■■■■■■■■■■■■■■■■&lt;/li&gt;
                  &lt;li&gt;■■■■■■■■■■■■■■■■■■■■■■&lt;/li&gt;
                  &lt;li&gt;■■ 2 ■■■■■■■&lt;/li&gt;
                &lt;/ul&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      {/* Footer */}
      &lt;div className="sticky bottom-0 bg-white border-t border-border p-6"&gt;
        &lt;button
          onClick={onClose}
          className="w-full px-6 py-3 bg-primary hover:bg-primary-dark text-white
    rounded-lg transition-all shadow-md"
        &gt;
          ■■
        &lt;/button&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  &lt;/div&gt;
);
}
```

■ File: src\components\PatientStats.tsx

```
import { useState, useEffect } from 'react';
import { AreaChart, Area, BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, Legend,
     ResponsiveContainer, PieChart, Pie, Cell } from 'recharts';
import { TrendingUp, Calendar, Activity, Pill, AlertTriangle, AlertCircle } from
     'lucide-react';
import { useAuthStore } from '../store/useAuthStore';

// ■■■/■■■■■■■■■■
const COLORS = ['#3b82f6', '#06b6d4', '#10b981', '#f59e0b', '#ec4899', '#8b5cf6'];

interface DiaryEntry {
  id?: string;
  userId?: string;
  date: string;
  time: string;
  intensity: number;
  symptoms: string[];
  medication: string;
  notes: string;
}

interface QuestionnaireResponse {
  id?: string;
  patientId: string;
  questionnaireId: string;
  score: number;
  details: string;
  completedAt: string;
}

export function PatientStats() {
  const currentUser = useAuthStore((state) =&gt; state.currentUser);
  const [diaries, setDiaries] = useState&lt;DiaryEntry[]&gt;([]);
  const [questionnaires, setQuestionnaires] = useState&lt;QuestionnaireResponse[]&gt;([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState&lt;string | null&gt;(null);

  const fetchData = async () =&gt; {
    if (!currentUser?.id) return;
    try {
      setLoading(true);
      setError(null);

      // Fetch diaries
      const diariesRes = await fetch(`http://${window.location.hostname}:8080/api/diaries/
    user/${currentUser.id}`);
      let diariesData: DiaryEntry[] = [];
      if (diariesRes.ok) {
        diariesData = await diariesRes.json();
        setDiaries(diariesData);
      } else {
        throw new Error('■■■■■■■■■■■');
      }

      // Fetch questionnaires
      const questionnairesRes = await fetch(`http://${window.location.hostname}:8080/api/q
    uestionnaires/patient/${currentUser.id}`);
      if (questionnairesRes.ok) {
        const questionnairesData = await questionnairesRes.json();
        setQuestionnaires(questionnairesData);
      } else {
        throw new Error('■■■■■■■■■■■');
      }

    } catch (err: any) {
      console.error(err);
      setError('■■■■■■■■■■■■■■■■ Spring Boot ■■■■■■■■■');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() =&gt; {
    fetchData();
  }, [currentUser]);

  // 1. ■■■■■■■■■
  const currentMonthStr = new Date().toISOString().slice(0, 7); // yyyy-MM
  const headacheDaysThisMonth = diaries.filter(d =&gt; d.date &amp;&amp;
```

```
   d.date.startsWith(currentMonthStr)).length;

// 2. [redacted]
const avgIntensity = diaries.length &gt; 0
  ? (diaries.reduce((sum, d) =&gt; sum + d.intensity, 0) / diaries.length).toFixed(1)
  : "0.0";

// 3. [redacted] ([redacted])
const avgDuration = diaries.length &gt; 0
  ? (diaries.reduce((sum, d) =&gt; sum + (d.intensity * 0.8 + 2), 0) /
   diaries.length).toFixed(1)
  : "0.0";

// 4. [redacted]
const medicationCountThisMonth = diaries.filter(d =&gt;
  d.date &amp;&amp;
  d.date.startsWith(currentMonthStr) &amp;&amp;
  d.medication &amp;&amp;
  d.medication.trim() !== ''
).length;

// === [redacted] ===

// 1. [redacted] ([redacted])
const chartHeadacheData = [...diaries]
  .sort((a, b) =&gt; a.date.localeCompare(b.date))
  .map(d =&gt; ({
    date: d.date ? d.date.slice(5) : '', // [redacted] MM-DD [redacted]
    intensity: d.intensity,
    duration: parseFloat((d.intensity * 0.8 + 2).toFixed(1))
  }))
  .slice(-10); // [redacted] 10 [redacted]

// 2. [redacted]
const symptomCounts: { [key: string]: number } = {};
diaries.forEach(d =&gt; {
  if (d.symptoms) {
    d.symptoms.forEach(s =&gt; {
      symptomCounts[s] = (symptomCounts[s] || 0) + 1;
    });
  }
});
const chartSymptomData = Object.keys(symptomCounts)
  .map(name =&gt; ({
    name,
    value: symptomCounts[name]
  }))
  .sort((a, b) =&gt; b.value - a.value)
  .slice(0, 5); // [redacted]

// 3. MIDAS [redacted] ([redacted])
const chartMidasData = questionnaires
  .filter(q =&gt; q.questionnaireId === 'midas')
  .sort((a, b) =&gt; a.completedAt.localeCompare(b.completedAt))
  .map(q =&gt; {
    const dateObj = new Date(q.completedAt);
    const monthStr = isNaN(dateObj.getTime()) ? '[redacted]' : `${dateObj.getMonth() +
   1}/${dateObj.getDate()}`;
    const score = q.score;
    let grade = 'I';
    if (score &gt; 20) grade = 'IV';
    else if (score &gt; 10) grade = 'III';
    else if (score &gt; 5) grade = 'II';
    return { month: monthStr, score, grade };
  });

// 4. [redacted]
const generateInsights = () =&gt; {
  const insights: string[] = [];
  if (diaries.length === 0) {
    return ["[redacted]"];
  }

  if (parseFloat(avgIntensity) &gt;= 7) {
    insights.push(`[redacted] ${avgIntensity} /
   10[redacted]`);
  } else {
    insights.push(`[redacted] ${avgIntensity} / 10[redacted]`);
  }

  if (headacheDaysThisMonth &gt;= 4) {
    insights.push(`[redacted] ${headacheDaysThisMonth}
```

```
    ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ (MOH)■■`);
  }

  if (chartSymptomData.length > 0) {
    insights.push(`■ ■■■■■■■■■■■■■■■■■■■■■■■■■■${chartSymptomData[0].name}■■■■■■■■■■■■■■■■■■■■■■■■
  ■■■■■■■■■■■■■■■■■■■■■■■■■■`);
  }

  return insights;
};

const insightsList = generateInsights();

// ■■■■■■■■■■■■■■■■
const customTooltipStyle = {
  backgroundColor: 'rgba(255, 255, 255, 0.98)',
  border: 'none',
  borderRadius: '12px',
  boxShadow: '0 10px 25px -5px rgba(0, 0, 0, 0.08), 0 8px 10px -6px rgba(0, 0, 0,
   0.08)',
  padding: '12px 16px',
};

if (loading) {
  return (
    <div className="bg-white rounded-xl shadow-md p-12 border border-border flex
   items-center justify-center h-96">
      <div className="text-center text-text-secondary">
        <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-primary
   mx-auto mb-4"></div>
        <p className="text-lg font-medium">■■■■■■■■■■■■■■■■■...</p>
      </div>
    </div>
  );
}

if (error) {
  return (
    <div className="bg-white rounded-xl shadow-md p-12 border border-border text-center
   text-danger">
      <AlertTriangle className="w-16 h-16 mx-auto mb-4 text-danger opacity-85" />
      <h3 className="text-xl font-bold mb-2">■■■■■■■</h3>
      <p className="mb-4 text-text-secondary">{error}</p>
      <button
        onClick={fetchData}
        className="px-6 py-2.5 bg-primary hover:bg-primary-dark text-white rounded-lg
   transition-all shadow-md"
      >
        ■■■■■
      </button>
    </div>
  );
}

return (
  <div className="space-y-6">
    <div className="text-center">
      <h2 className="text-text-primary mb-2">■■■■■</h2>
      <p className="text-text-secondary">■■■■■■■■■■■■■■</p>
    </div>

    {/* Summary Cards */}
    <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
      <div className="bg-white rounded-xl shadow-sm p-6 border border-border
   hover:shadow-md transition-shadow flex flex-col items-center text-center">
        <div className="mb-2">
          <div className="w-12 h-12 bg-blue-50 text-blue-500 rounded-xl flex
   items-center justify-center">
            <Calendar className="w-6 h-6" />
          </div>
        </div>
        <div className="text-text-secondary text-sm">■■■■■■■</div>
        <h3 className="text-primary mt-1 text-2xl font-bold">{headacheDaysThisMonth}
   <span className="text-sm font-normal text-text-secondary">■</span></h3>
      </div>

      <div className="bg-white rounded-xl shadow-sm p-6 border border-border
   hover:shadow-md transition-shadow flex flex-col items-center text-center">
        <div className="mb-2">
          <div className="w-12 h-12 bg-red-50 text-red-500 rounded-xl flex items-center
   justify-center">
            <Activity className="w-6 h-6" />
```

```
      &lt;/div&gt;
     &lt;/div&gt;
     &lt;div className="text-text-secondary text-sm"&gt;[redacted]&lt;/div&gt;
     &lt;h3 className="text-danger mt-1 text-2xl font-bold"&gt;{avgIntensity} &lt;span
className="text-sm font-normal text-text-secondary"&gt;/ 10&lt;/span&gt;&lt;/h3&gt;
   &lt;/div&gt;

   &lt;div className="bg-white rounded-xl shadow-sm p-6 border border-border
hover:shadow-md transition-shadow flex flex-col items-center text-center"&gt;
     &lt;div className="mb-2"&gt;
       &lt;div className="w-12 h-12 bg-green-50 text-green-500 rounded-xl flex
items-center justify-center"&gt;
         &lt;TrendingUp className="w-6 h-6" /&gt;
       &lt;/div&gt;
     &lt;/div&gt;
     &lt;div className="text-text-secondary text-sm"&gt;[redacted]&lt;/div&gt;
     &lt;h3 className="text-success mt-1 text-2xl font-bold"&gt;{avgDuration} &lt;span
className="text-sm font-normal text-text-secondary"&gt;[redacted]&lt;/span&gt;&lt;/h3&gt;
   &lt;/div&gt;

   &lt;div className="bg-white rounded-xl shadow-sm p-6 border border-border
hover:shadow-md transition-shadow flex flex-col items-center text-center"&gt;
     &lt;div className="mb-2"&gt;
       &lt;div className="w-12 h-12 bg-orange-50 text-orange-500 rounded-xl flex
items-center justify-center"&gt;
         &lt;Pill className="w-6 h-6" /&gt;
       &lt;/div&gt;
     &lt;/div&gt;
     &lt;div className="text-text-secondary text-sm"&gt;[redacted]&lt;/div&gt;
     &lt;h3 className="text-warning mt-1 text-2xl font-bold"&gt;{medicationCountThisMonth}
&lt;span className="text-sm font-normal text-text-secondary"&gt;[redacted]&lt;/span&gt;&lt;/h3&gt;
   &lt;/div&gt;
 &lt;/div&gt;

 {/* Charts Grid */}
 &lt;div className="grid grid-cols-1 lg:grid-cols-2 gap-6"&gt;
   {/* Intensity Trend - AreaChart with glowing gradient */}
   &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border"&gt;
     &lt;h3 className="text-text-primary mb-6 font-semibold text-center"&gt;[redacted]&lt;/h3&gt;
     {chartHeadacheData.length === 0 ? (
       &lt;div className="flex flex-col items-center justify-center h-[300px]
text-text-secondary"&gt;
         &lt;AlertCircle className="w-12 h-12 text-gray-300 mb-2" /&gt;
         &lt;p&gt;[redacted]&lt;/p&gt;
       &lt;/div&gt;
     ) : (
       &lt;ResponsiveContainer width="100%" height={300}&gt;
         &lt;AreaChart data={chartHeadacheData} margin={{ top: 20, right: 30, left: 0,
bottom: 15 }}&gt;
           &lt;defs&gt;
             &lt;linearGradient id="colorIntensity" x1="0" y1="0" x2="0" y2="1"&gt;
               &lt;stop offset="5%" stopColor="#3b82f6" stopOpacity={0.25}/&gt;
               &lt;stop offset="95%" stopColor="#3b82f6" stopOpacity={0.01}/&gt;
             &lt;/linearGradient&gt;
           &lt;/defs&gt;
           &lt;CartesianGrid strokeDasharray="4 4" stroke="#f1f5f9" vertical={false} /&gt;
           &lt;XAxis dataKey="date" stroke="#94a3b8" tickLine={false} axisLine={false}
dy={12} /&gt;
           &lt;YAxis stroke="#94a3b8" domain={[0, 10]} tickLine={false}
axisLine={false} dx={-12} /&gt;
           &lt;Tooltip contentStyle={customTooltipStyle} /&gt;
           &lt;Legend iconType="circle" /&gt;
           &lt;Area type="monotone" dataKey="intensity" stroke="#3b82f6"
strokeWidth={2.5} fillOpacity={1} fill="url(#colorIntensity)" name="[redacted]"
activeDot={{ r: 6, strokeWidth: 0 }} /&gt;
         &lt;/AreaChart&gt;
       &lt;/ResponsiveContainer&gt;
     )}
   &lt;/div&gt;

   {/* Duration Chart - BarChart with custom gradient and rounded top corners */}
   &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border"&gt;
     &lt;h3 className="text-text-primary mb-6 font-semibold text-center"&gt;[redacted]&lt;/h3&gt;
     {chartHeadacheData.length === 0 ? (
       &lt;div className="flex flex-col items-center justify-center h-[300px]
text-text-secondary"&gt;
         &lt;AlertCircle className="w-12 h-12 text-gray-300 mb-2" /&gt;
         &lt;p&gt;[redacted]&lt;/p&gt;
       &lt;/div&gt;
     ) : (
       &lt;ResponsiveContainer width="100%" height={300}&gt;
         &lt;BarChart data={chartHeadacheData} margin={{ top: 20, right: 30, left: 0,
```

```
bottom: 15 }}&gt;
           &lt;defs&gt;
             &lt;linearGradient id="colorBarDuration" x1="0" y1="0" x2="0" y2="1"&gt;
               &lt;stop offset="0%" stopColor="#06b6d4" stopOpacity={0.95}/&gt;
               &lt;stop offset="100%" stopColor="#3b82f6" stopOpacity={0.95}/&gt;
             &lt;/linearGradient&gt;
           &lt;/defs&gt;
           &lt;CartesianGrid strokeDasharray="4 4" stroke="#f1f5f9" vertical={false} /&gt;
           &lt;XAxis dataKey="date" stroke="#94a3b8" tickLine={false} axisLine={false}
dy={12} /&gt;
           &lt;YAxis stroke="#94a3b8" tickLine={false} axisLine={false} dx={-12} /&gt;
           &lt;Tooltip contentStyle={customTooltipStyle} /&gt;
           &lt;Legend iconType="circle" /&gt;
           &lt;Bar dataKey="duration" fill="url(#colorBarDuration)" radius={[6, 6, 0,
0]} barSize={20} name="[redacted]" /&gt;
         &lt;/BarChart&gt;
       &lt;/ResponsiveContainer&gt;
     )}
   &lt;/div&gt;

   {/* Symptom Distribution - Modern Donut (Ring) Chart */}
   &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border"&gt;
     &lt;h3 className="text-text-primary mb-6 font-semibold text-center"&gt;[redacted]&lt;/h3&gt;
     {chartSymptomData.length === 0 ? (
       &lt;div className="flex flex-col items-center justify-center h-[300px]
text-text-secondary"&gt;
         &lt;AlertCircle className="w-12 h-12 text-gray-300 mb-2" /&gt;
         &lt;p&gt;[redacted]&lt;/p&gt;
       &lt;/div&gt;
     ) : (
       &lt;ResponsiveContainer width="100%" height={300}&gt;
         &lt;PieChart&gt;
           &lt;Pie
             data={chartSymptomData}
             cx="50%"
             cy="50%"
             innerRadius={65}
             outerRadius={90}
             paddingAngle={4}
             dataKey="value"
             labelLine={true}
             label={({ name, percent }) =&gt; `${name} ${(percent * 100).toFixed(0)}%`}
           &gt;
             {chartSymptomData.map((entry, index) =&gt; (
               &lt;Cell key={`cell-${index}`} fill={COLORS[index % COLORS.length]}
stroke="none" /&gt;
             ))}
           &lt;/Pie&gt;
           &lt;Tooltip contentStyle={customTooltipStyle} /&gt;
         &lt;/PieChart&gt;
       &lt;/ResponsiveContainer&gt;
     )}
   &lt;/div&gt;

   {/* MIDAS Score Trend - Elegant Green AreaChart */}
   &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border"&gt;
     &lt;h3 className="text-text-primary mb-6 font-semibold text-center"&gt;MIDAS [redacted]&lt;/h3&gt;
     {chartMidasData.length === 0 ? (
       &lt;div className="flex flex-col items-center justify-center h-[300px]
text-text-secondary"&gt;
         &lt;AlertCircle className="w-12 h-12 text-gray-300 mb-2" /&gt;
         &lt;p&gt;[redacted] MIDAS [redacted]&lt;/p&gt;
       &lt;/div&gt;
     ) : (
       &lt;ResponsiveContainer width="100%" height={300}&gt;
         &lt;AreaChart data={chartMidasData} margin={{ top: 20, right: 30, left: 0,
bottom: 15 }}&gt;
           &lt;defs&gt;
             &lt;linearGradient id="colorMidas" x1="0" y1="0" x2="0" y2="1"&gt;
               &lt;stop offset="5%" stopColor="#10b981" stopOpacity={0.25}/&gt;
               &lt;stop offset="95%" stopColor="#10b981" stopOpacity={0.01}/&gt;
             &lt;/linearGradient&gt;
           &lt;/defs&gt;
           &lt;CartesianGrid strokeDasharray="4 4" stroke="#f1f5f9" vertical={false} /&gt;
           &lt;XAxis dataKey="month" stroke="#94a3b8" tickLine={false} axisLine={false}
dy={12} /&gt;
           &lt;YAxis stroke="#94a3b8" tickLine={false} axisLine={false} dx={-12} /&gt;
           &lt;Tooltip
             contentStyle={customTooltipStyle}
             formatter={(value: number, name: string) =&gt; {
               if (name === 'MIDAS[redacted]') {
                 const entry = chartMidasData.find(d =&gt; d.score === value);
```

```
                  return [`${value}■ (Grade ${entry?.grade})`, name];
                }
                return [value, name];
              }}
            /&gt;
            &lt;Legend iconType="circle" /&gt;
            &lt;Area type="monotone" dataKey="score" stroke="#10b981" strokeWidth={2.5}
    fillOpacity={1} fill="url(#colorMidas)" name="MIDAS■■" activeDot={{ r: 6,
    strokeWidth: 0 }} /&gt;
          &lt;/AreaChart&gt;
        &lt;/ResponsiveContainer&gt;
      )}
    &lt;/div&gt;
   &lt;/div&gt;

   {/* Insights */}
   &lt;div className="bg-gradient-to-br from-blue-50/70 via-cyan-50/30 to-blue-50/50
    rounded-2xl p-6 border border-blue-100 shadow-sm"&gt;
    &lt;h3 className="text-text-primary mb-4 font-semibold flex items-center
    justify-center gap-2"&gt;
      &lt;Activity className="w-5 h-5 text-primary" /&gt;
      ■■■■■■■■
    &lt;/h3&gt;
    &lt;div className="space-y-4"&gt;
      {insightsList.map((insight, idx) =&gt; (
        &lt;div key={idx} className="flex items-start gap-3"&gt;
          &lt;div className="w-2 h-2 bg-primary rounded-full mt-2 flex-shrink-0"&gt;&lt;/div&gt;
          &lt;p className="text-text-primary leading-relaxed text-sm"&gt;{insight}&lt;/p&gt;
        &lt;/div&gt;
      ))}
    &lt;/div&gt;
   &lt;/div&gt;
  &lt;/div&gt;
 );
}
```

■ File: src\components\PrescriptionModal.tsx

```
import { useState } from 'react';
import { X, Pill, Plus, Trash2, FileText } from 'lucide-react';

interface Patient {
  id: string;
  name: string;
  age: number;
  gender: string;
}

interface PrescriptionModalProps {
  patient: Patient;
  onClose: () =&gt; void;
  onSubmit: (prescription: PrescriptionData) =&gt; void;
}

interface Medication {
  id: string;
  name: string;
  dosage: string;
  frequency: string;
  duration: string;
  notes: string;
}

interface PrescriptionData {
  medications: Medication[];
  diagnosis: string;
  instructions: string;
  nextVisit: string;
}

const commonMedications = [
  { name: '■■■■■■■ (Panadol)', defaultDosage: '500mg', defaultFrequency: '■■■■■■4-6■■1■' },
  { name: '■■■■■■■ (Panadol Extra)', defaultDosage: '500mg + 65mg ■■■', defaultFrequency:
     '■■■■■■4-6■■1■' },
  { name: 'EVE Quick ■■■ (EVE)', defaultDosage: '150mg Ibuprofen', defaultFrequency:
     '■■■■■■(■■2■)' },
  { name: '■■■■■■■■■ (Bufferin)', defaultDosage: '500mg', defaultFrequency: '■■■■■■4-6■■1■'
     },
  { name: '■■■■■■■ (Snooz)', defaultDosage: '500mg', defaultFrequency: '■4-6■■1■(■■1-2■)'
     },
];

export function PrescriptionModal({ patient, onClose, onSubmit }: PrescriptionModalProps)
     {
  const [medications, setMedications] = useState&lt;Medication[]&gt;([]);
  const [diagnosis, setDiagnosis] = useState('■■■■');
  const [instructions, setInstructions] = useState('');
  const [nextVisit, setNextVisit] = useState('');
  const [showAddMed, setShowAddMed] = useState(false);
  const [newMed, setNewMed] = useState({
    name: '',
    dosage: '',
    frequency: '',
    duration: '7■',
    notes: '',
  });

  const handleAddMedication = () =&gt; {
    if (newMed.name &amp;&amp; newMed.dosage &amp;&amp; newMed.frequency) {
      const medication: Medication = {
        id: Date.now().toString(),
        ...newMed,
      };
      setMedications([...medications, medication]);
      setNewMed({
        name: '',
        dosage: '',
        frequency: '',
        duration: '7■',
        notes: '',
      });
      setShowAddMed(false);
    }
  };

  const handleRemoveMedication = (id: string) =&gt; {
    setMedications(medications.filter(med =&gt; med.id !== id));
```

```
};

const handleQuickAdd = (commonMed: typeof commonMedications[0]) =&gt; {
  setNewMed({
    name: commonMed.name,
    dosage: commonMed.defaultDosage,
    frequency: commonMed.defaultFrequency,
    duration: '7■',
    notes: '',
  });
  setShowAddMed(true);
};

const handleSubmit = (e: React.FormEvent) =&gt; {
  e.preventDefault();
  onSubmit({
    medications,
    diagnosis,
    instructions,
    nextVisit,
  });
};

return (
  &lt;div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50 p-4"&gt;
    &lt;div className="bg-white rounded-xl shadow-2xl max-w-3xl w-full max-h-[90vh]
   overflow-y-auto"&gt;
      {/* Header */}
      &lt;div className="sticky top-0 bg-white border-b border-border p-6 flex
   items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h2 className="text-text-primary"&gt;■■■■■■&lt;/h2&gt;
          &lt;p className="text-text-secondary"&gt;
            ■■■{patient.name} · {patient.age}■ · {patient.gender}
          &lt;/p&gt;
        &lt;/div&gt;
        &lt;button
          onClick={onClose}
          className="p-2 hover:bg-gray-100 rounded-lg transition-colors"
        &gt;
          &lt;X className="w-6 h-6 text-text-secondary" /&gt;
        &lt;/button&gt;
      &lt;/div&gt;

      {/* Content */}
      &lt;form onSubmit={handleSubmit} className="p-6 space-y-6"&gt;
        {/* Diagnosis */}
        &lt;div&gt;
          &lt;label className="block mb-2"&gt;■■&lt;/label&gt;
          &lt;input
            type="text"
            value={diagnosis}
            onChange={(e) =&gt; setDiagnosis(e.target.value)}
            className="w-full px-4 py-3 border border-border rounded-lg
   focus:outline-none focus:ring-2 focus:ring-primary"
            placeholder="■■■■■■"
            required
          /&gt;
        &lt;/div&gt;

        {/* Medications */}
        &lt;div&gt;
          &lt;div className="flex items-center justify-between mb-4"&gt;
            &lt;label className="flex items-center gap-2"&gt;
              &lt;Pill className="w-5 h-5" /&gt;
              ■■■■
            &lt;/label&gt;
            &lt;button
              type="button"
              onClick={() =&gt; setShowAddMed(true)}
              className="flex items-center gap-2 px-4 py-2 bg-primary
   hover:bg-primary-dark text-white rounded-lg transition-all"
            &gt;
              &lt;Plus className="w-4 h-4" /&gt;
              ■■■■
            &lt;/button&gt;
          &lt;/div&gt;

          {/* Common Medications Quick Add */}
          {!showAddMed &amp;&amp; medications.length === 0 &amp;&amp; (
            &lt;div className="mb-4"&gt;
              &lt;p className="text-text-secondary mb-2"&gt;■■■■■■■■■■■■&lt;/p&gt;
```

```
          &lt;div className="grid grid-cols-2 md:grid-cols-3 gap-2"&gt;
            {commonMedications.map((med, idx) =&gt; (
              &lt;button
                key={idx}
                type="button"
                onClick={() =&gt; handleQuickAdd(med)}
                className="px-3 py-2 border border-border rounded-lg
hover:border-primary hover:bg-medical-blue transition-all text-left"
              &gt;
                &lt;div className="text-text-primary"&gt;{med.name}&lt;/div&gt;
                &lt;div className="text-text-secondary
text-sm"&gt;{med.defaultDosage}&lt;/div&gt;
              &lt;/button&gt;
            ))}
          &lt;/div&gt;
        &lt;/div&gt;
      )}

      {/* Add Medication Form */}
      {showAddMed &amp;&amp; (
        &lt;div className="bg-medical-blue rounded-lg p-4 mb-4 space-y-4"&gt;
          &lt;div className="grid grid-cols-1 md:grid-cols-2 gap-4"&gt;
            &lt;div&gt;
              &lt;label className="block mb-2 text-sm"&gt;[redacted]&lt;/label&gt;
              &lt;input
                type="text"
                value={newMed.name}
                onChange={(e) =&gt; setNewMed({ ...newMed, name: e.target.value })}
                className="w-full px-3 py-2 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
                placeholder="[redacted]"
              /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;label className="block mb-2 text-sm"&gt;[redacted]&lt;/label&gt;
              &lt;input
                type="text"
                value={newMed.dosage}
                onChange={(e) =&gt; setNewMed({ ...newMed, dosage: e.target.value })}
                className="w-full px-3 py-2 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
                placeholder="[redacted]500mg"
              /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;label className="block mb-2 text-sm"&gt;[redacted]&lt;/label&gt;
              &lt;input
                type="text"
                value={newMed.frequency}
                onChange={(e) =&gt; setNewMed({ ...newMed, frequency: e.target.value
})}
                className="w-full px-3 py-2 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
                placeholder="[redacted]3[redacted]"
              /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;label className="block mb-2 text-sm"&gt;[redacted]&lt;/label&gt;
              &lt;input
                type="text"
                value={newMed.duration}
                onChange={(e) =&gt; setNewMed({ ...newMed, duration: e.target.value })}
                className="w-full px-3 py-2 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
                placeholder="[redacted]7[redacted]"
              /&gt;
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;label className="block mb-2 text-sm"&gt;[redacted]&lt;/label&gt;
            &lt;input
              type="text"
              value={newMed.notes}
              onChange={(e) =&gt; setNewMed({ ...newMed, notes: e.target.value })}
              className="w-full px-3 py-2 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
              placeholder="[redacted]"
            /&gt;
          &lt;/div&gt;
          &lt;div className="flex gap-2"&gt;
            &lt;button
              type="button"
```

```
              onClick={handleAddMedication}
              className="flex-1 px-4 py-2 bg-primary hover:bg-primary-dark
text-white rounded-lg transition-all"
            &gt;
              ■■■■
            &lt;/button&gt;
            &lt;button
              type="button"
              onClick={() =&gt; setShowAddMed(false)}
              className="px-4 py-2 border border-border hover:bg-gray-50 rounded-lg
transition-all"
            &gt;
              ■■
            &lt;/button&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      )}

      {/* Medications List */}
      {medications.length &gt; 0 &amp;&amp; (
        &lt;div className="space-y-3"&gt;
          {medications.map((med) =&gt; (
            &lt;div key={med.id} className="bg-white border border-border rounded-lg
p-4"&gt;
              &lt;div className="flex items-start justify-between"&gt;
                &lt;div className="flex-1"&gt;
                  &lt;div className="flex items-center gap-2 mb-2"&gt;
                    &lt;Pill className="w-4 h-4 text-primary" /&gt;
                    &lt;span className="text-text-primary"&gt;{med.name}&lt;/span&gt;
                  &lt;/div&gt;
                  &lt;div className="grid grid-cols-2 gap-2 text-sm"&gt;
                    &lt;div&gt;
                      &lt;span className="text-text-secondary"&gt;■■■&lt;/span&gt;
                      &lt;span className="text-text-primary"&gt;{med.dosage}&lt;/span&gt;
                    &lt;/div&gt;
                    &lt;div&gt;
                      &lt;span className="text-text-secondary"&gt;■■■&lt;/span&gt;
                      &lt;span className="text-text-primary"&gt;{med.frequency}&lt;/span&gt;
                    &lt;/div&gt;
                    &lt;div&gt;
                      &lt;span className="text-text-secondary"&gt;■■■&lt;/span&gt;
                      &lt;span className="text-text-primary"&gt;{med.duration}&lt;/span&gt;
                    &lt;/div&gt;
                    {med.notes &amp;&amp; (
                      &lt;div className="col-span-2"&gt;
                        &lt;span className="text-text-secondary"&gt;■■■&lt;/span&gt;
                        &lt;span className="text-text-primary"&gt;{med.notes}&lt;/span&gt;
                      &lt;/div&gt;
                    )}
                  &lt;/div&gt;
                &lt;/div&gt;
                &lt;button
                  type="button"
                  onClick={() =&gt; handleRemoveMedication(med.id)}
                  className="p-2 text-danger hover:bg-red-50 rounded-lg
transition-colors"
                &gt;
                  &lt;Trash2 className="w-4 h-4" /&gt;
                &lt;/button&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          ))}
        &lt;/div&gt;
      )}

      {medications.length === 0 &amp;&amp; !showAddMed &amp;&amp; (
        &lt;div className="text-center py-8 text-text-secondary"&gt;
          ■■■■■■■■■
        &lt;/div&gt;
      )}
    &lt;/div&gt;

    {/* Instructions */}
    &lt;div&gt;
      &lt;label className="block mb-2"&gt;■■ / ■■■■&lt;/label&gt;
      &lt;textarea
        value={instructions}
        onChange={(e) =&gt; setInstructions(e.target.value)}
        className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary resize-none"
        rows={4}
        placeholder="■■■■■■■■■■■■■■■■■■■■■..."
```

```
            /&gt;
          &lt;/div&gt;

          {/* Next Visit */}
          &lt;div&gt;
            &lt;label className="block mb-2"&gt;■■■■■■■&lt;/label&gt;
            &lt;input
              type="date"
              value={nextVisit}
              onChange={(e) =&gt; setNextVisit(e.target.value)}
              className="w-full px-4 py-3 border border-border rounded-lg
    focus:outline-none focus:ring-2 focus:ring-primary"
            /&gt;
          &lt;/div&gt;

          {/* Buttons */}
          &lt;div className="flex gap-3"&gt;
            &lt;button
              type="button"
              onClick={onClose}
              className="flex-1 px-6 py-3 border border-border hover:bg-gray-50
    rounded-lg transition-all"
            &gt;
              ■■
            &lt;/button&gt;
            &lt;button
              type="submit"
              className="flex-1 px-6 py-3 bg-primary hover:bg-primary-dark text-white
    rounded-lg transition-all shadow-md"
            &gt;
              ■■■■■■■■■
            &lt;/button&gt;
          &lt;/div&gt;
        &lt;/form&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

■ File: src\components\QuestionnaireList.tsx

```
import { useState, useEffect } from 'react';
import { FileText, ChevronRight, CheckCircle, Circle } from 'lucide-react';
import { useAuthStore } from '../store/useAuthStore';
import { MIDASQuestionnaire } from './questionnaires/MIDASQuestionnaire';
import { HADSQuestionnaire } from './questionnaires/HADSQuestionnaire';
import { BDIQuestionnaire } from './questionnaires/BDIQuestionnaire';
import { PSQIQuestionnaire } from './questionnaires/PSQIQuestionnaire';
import { FSSQuestionnaire } from './questionnaires/FSSQuestionnaire';
import { WPIQuestionnaire } from './questionnaires/WPIQuestionnaire';
import { AllodyniaQuestionnaire } from './questionnaires/AllodyniaQuestionnaire';
import { PSSQuestionnaire } from './questionnaires/PSSQuestionnaire';

interface Questionnaire {
  id: string;
  name: string;
  fullName: string;
  description: string;
  completed: boolean;
  lastCompleted?: string;
}

export function QuestionnaireList() {
  const currentUser = useAuthStore((state) =&gt; state.currentUser);
  const [selectedQuestionnaire, setSelectedQuestionnaire] = useState&lt;string | null&gt;(null);
  const [questionnaires, setQuestionnaires] = useState&lt;Questionnaire[]&gt;([
    {
      id: 'midas',
      name: 'MIDAS',
      fullName: 'Migraine Disability Assessment',
      description: '■■■■■■■■■■',
      completed: false,
    },
    {
      id: 'hads',
      name: 'HADS',
      fullName: 'Hospital Anxiety and Depression Scale',
      description: '■■■■■■■■■',
      completed: false,
    },
    {
      id: 'bdi',
      name: 'BDI',
      fullName: 'Beck Depression Inventory',
      description: '■■■■■■■',
      completed: false,
    },
    {
      id: 'psqi',
      name: 'PSQI',
      fullName: 'Pittsburgh Sleep Quality Index',
      description: '■■■■■■■■■■',
      completed: false,
    },
    {
      id: 'fss',
      name: 'FSS',
      fullName: 'Fatigue Severity Scale',
      description: '■■■■■■■■',
      completed: false,
    },
    {
      id: 'wpi',
      name: 'WPI',
      fullName: 'Widespread Pain Index',
      description: '■■■■■■■',
      completed: false,
    },
    {
      id: 'allodynia',
      name: 'Allodynia',
      fullName: 'Allodynia Questionnaire',
      description: '■■■■■■■■',
      completed: false,
    },
    {
      id: 'pss',
      name: 'PSS',
      fullName: 'Perceived Stress Scale',
      description: '■■■■■■■',
```

```
    completed: false,
  },
]);

useEffect(() =&gt; {
  if (currentUser?.id) {
     fetch(`http://${window.location.hostname}:8080/api/questionnaires/patient/${current
  User.id}`)
        .then(res =&gt; res.json())
        .then((responses: any[]) =&gt; {
             setQuestionnaires(prev =&gt; prev.map(q =&gt; {
                 const qs = responses.filter(r =&gt; r.questionnaireId === q.id);
                 if (qs.length &gt; 0) {
                     // Find the latest response by date simply or take first
                     const lastResponse = qs[qs.length - 1];
                     return { ...q, completed: true, lastCompleted:
   lastResponse.completedAt.split('T')[0] };
                 }
                 return q;
             }));
        })
        .catch(err =&gt; console.error("Failed to load questionnaires", err));
  }
}, [currentUser]);

const handleComplete = async (questionnaireId: string) =&gt; {
  if (!currentUser) return;

  // Simplistic completion saving (can be extended to include actual scores)
  try {
      await fetch(`http://${window.location.hostname}:8080/api/questionnaires`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({
              patientId: currentUser.id,
              questionnaireId: questionnaireId,
              score: 0,
              details: 'Completed via UI'
          })
      });

      setQuestionnaires((prev) =&gt;
        prev.map((q) =&gt;
          q.id === questionnaireId
            ? { ...q, completed: true, lastCompleted: new
   Date().toISOString().split('T')[0] }
            : q
        )
      );
  } catch(err) {
      console.error("Save questionnaire error", err);
  }

  setSelectedQuestionnaire(null);
};

if (selectedQuestionnaire === 'midas') {
  return &lt;MIDASQuestionnaire onBack={() =&gt; setSelectedQuestionnaire(null)}
   onComplete={() =&gt; handleComplete('midas')} /&gt;;
}
if (selectedQuestionnaire === 'hads') {
  return &lt;HADSQuestionnaire onBack={() =&gt; setSelectedQuestionnaire(null)}
   onComplete={() =&gt; handleComplete('hads')} /&gt;;
}
if (selectedQuestionnaire === 'bdi') {
  return &lt;BDIQuestionnaire onBack={() =&gt; setSelectedQuestionnaire(null)} onComplete={()
   =&gt; handleComplete('bdi')} /&gt;;
}
if (selectedQuestionnaire === 'psqi') {
  return &lt;PSQIQuestionnaire onBack={() =&gt; setSelectedQuestionnaire(null)}
   onComplete={() =&gt; handleComplete('psqi')} /&gt;;
}
if (selectedQuestionnaire === 'fss') {
  return &lt;FSSQuestionnaire onBack={() =&gt; setSelectedQuestionnaire(null)} onComplete={()
   =&gt; handleComplete('fss')} /&gt;;
}
if (selectedQuestionnaire === 'wpi') {
  return &lt;WPIQuestionnaire onBack={() =&gt; setSelectedQuestionnaire(null)} onComplete={()
   =&gt; handleComplete('wpi')} /&gt;;
}
if (selectedQuestionnaire === 'allodynia') {
  return &lt;AllodyniaQuestionnaire onBack={() =&gt; setSelectedQuestionnaire(null)}
```

```
    onComplete={() =&gt; handleComplete('allodynia')} /&gt;;
  }
  if (selectedQuestionnaire === 'pss') {
    return &lt;PSSQuestionnaire onBack={() =&gt; setSelectedQuestionnaire(null)} onComplete={()
     =&gt; handleComplete('pss')} /&gt;;
  }

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div&gt;
        &lt;h2 className="text-text-primary mb-2"&gt;[redacted]&lt;/h2&gt;
        &lt;p className="text-text-secondary"&gt;[redacted]&lt;/p&gt;
      &lt;/div&gt;

      {/* Progress Summary */}
      &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border"&gt;
        &lt;div className="flex items-center justify-between mb-4"&gt;
          &lt;h3 className="text-text-primary"&gt;[redacted]&lt;/h3&gt;
          &lt;span className="text-primary"&gt;
            {questionnaires.filter((q) =&gt; q.completed).length} / {questionnaires.length}
          &lt;/span&gt;
        &lt;/div&gt;
        &lt;div className="w-full bg-gray-200 rounded-full h-3"&gt;
          &lt;div
            className="bg-primary h-3 rounded-full transition-all"
            style={{ width: `${(questionnaires.filter((q) =&gt; q.completed).length /
    questionnaires.length) * 100}%` }}
          /&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      {/* Questionnaires Grid */}
      &lt;div className="grid grid-cols-1 md:grid-cols-2 gap-4"&gt;
        {questionnaires.map((questionnaire) =&gt; (
          &lt;button
            key={questionnaire.id}
            onClick={() =&gt; setSelectedQuestionnaire(questionnaire.id)}
            className="bg-white rounded-xl shadow-md p-6 border border-border
    hover:shadow-lg hover:border-primary transition-all text-left group"
          &gt;
            &lt;div className="flex items-start justify-between mb-3"&gt;
              &lt;div className="flex items-center gap-3"&gt;
                &lt;div className="w-12 h-12 bg-medical-blue rounded-lg flex items-center
    justify-center"&gt;
                  &lt;FileText className="w-6 h-6 text-primary" /&gt;
                &lt;/div&gt;
                &lt;div&gt;
                  &lt;h4 className="text-text-primary group-hover:text-primary
    transition-colors"&gt;{questionnaire.name}&lt;/h4&gt;
                  &lt;p className="text-text-secondary"&gt;{questionnaire.fullName}&lt;/p&gt;
                &lt;/div&gt;
              &lt;/div&gt;
              &lt;ChevronRight className="w-5 h-5 text-text-secondary
    group-hover:text-primary transition-colors" /&gt;
            &lt;/div&gt;

            &lt;p className="text-text-secondary mb-3"&gt;{questionnaire.description}&lt;/p&gt;

            &lt;div className="flex items-center justify-between"&gt;
              {questionnaire.completed ? (
                &lt;div className="flex items-center gap-2 text-success"&gt;
                  &lt;CheckCircle className="w-5 h-5" /&gt;
                  &lt;span&gt;[redacted]&lt;/span&gt;
                &lt;/div&gt;
              ) : (
                &lt;div className="flex items-center gap-2 text-text-secondary"&gt;
                  &lt;Circle className="w-5 h-5" /&gt;
                  &lt;span&gt;[redacted]&lt;/span&gt;
                &lt;/div&gt;
              )}
              {questionnaire.lastCompleted &amp;&amp; (
                &lt;span className="text-text-secondary"&gt;
                  {questionnaire.lastCompleted}
                &lt;/span&gt;
              )}
            &lt;/div&gt;
          &lt;/button&gt;
        ))}
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

■ File: src\components\figma\ImageWithFallback.tsx

```
import React, { useState } from 'react'

const ERROR_IMG_SRC =
  '
    nczLm9yZy8yMDAwL3N2ZyIgc3Ryb2tlPSIjMDAwIiBzdHJva2UtbGluZWpvaW49InJvdW5kIiBvcGFjaXR5PSI
    uMyIgZmlsbD0ibm9uZSIgc3Ryb2tlLXdpZHRoPSIzLjciPjxyZWN0IHg9IjE2IiB5PSIxNiIgd2lkdGg9IjU2I
    iBoZWlnaHQ9IjU2IiByeD0iNiIvPjxwYXRoIGQ9Im0xNiA1OCAxNi0xOCAzMiAzMiIvPjxjaXJjbGUgY3g9IjU
    zIiBjeT0iMzUiIHI9IjciLz48L3N2Zz4KCg=='

export function ImageWithFallback(props: React.ImgHTMLAttributes&lt;HTMLImageElement&gt;) {
  const [didError, setDidError] = useState(false)

  const handleError = () =&gt; {
    setDidError(true)
  }

  const { src, alt, style, className, ...rest } = props

  return didError ? (
    &lt;div
      className={`inline-block bg-gray-100 text-center align-middle ${className ?? ''}`}
      style={style}
    &gt;
      &lt;div className="flex items-center justify-center w-full h-full"&gt;
        &lt;img src={ERROR_IMG_SRC} alt="Error loading image" {...rest}
     data-original-url={src} /&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  ) : (
    &lt;img src={src} alt={alt} className={className} style={style} {...rest}
     onError={handleError} /&gt;
  )
}
```

■ File: src\components\questionnaires\AllodyniaQuestionnaire.tsx

```
import { useState } from 'react';
import { ArrowLeft, Save } from 'lucide-react';

interface AllodyniaQuestionnaireProps {
  onBack: () =&gt; void;
  onComplete: () =&gt; void;
}

export function AllodyniaQuestionnaire({ onBack, onComplete }:
     AllodyniaQuestionnaireProps) {
  const [answers, setAnswers] = useState&lt;number[]&gt;(Array(12).fill(-1));

  const questions = [
    '■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■',
  ];

  const options = ['■', '■'];

  const handleSubmit = (e: React.FormEvent) =&gt; {
    e.preventDefault();
    if (answers.some((a) =&gt; a === -1)) {
      alert('■■■■■■■■');
      return;
    }

    const score = answers.reduce((sum, val) =&gt; sum + val, 0);
    let severity = '';
    if (score === 0) severity = '■■■■■■■';
    else if (score &lt;= 2) severity = '■■■■■■■■';
    else if (score &lt;= 6) severity = '■■■■■■■■';
    else severity = '■■■■■■■■';

    alert(`Allodynia ■■■■■■\n\n■■■■${score}/12\n■■■■${severity}\n\n■■■■■■■■■■■■`);
    onComplete();
  };

  const getScore = () =&gt; {
    return answers.reduce((sum, val) =&gt; sum + (val &gt;= 0 ? val : 0), 0);
  };

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="flex items-center gap-4"&gt;
        &lt;button
          onClick={onBack}
          className="flex items-center gap-2 px-4 py-2 border border-border rounded-lg
     hover:bg-gray-50 transition-colors"
        &gt;
          &lt;ArrowLeft className="w-5 h-5" /&gt;
          ■■
        &lt;/button&gt;
        &lt;div&gt;
          &lt;h2 className="text-text-primary"&gt;Allodynia ■■■■■■■■&lt;/h2&gt;
          &lt;p className="text-text-secondary"&gt;Allodynia Symptom Checklist&lt;/p&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      &lt;div className="bg-white rounded-xl shadow-lg p-6 border border-border"&gt;
        &lt;form onSubmit={handleSubmit} className="space-y-6"&gt;
          &lt;div className="bg-medical-blue p-4 rounded-lg"&gt;
            &lt;p className="text-text-primary"&gt;
              ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            &lt;/p&gt;
          &lt;/div&gt;

          {questions.map((question, index) =&gt; (
            &lt;div key={index} className="space-y-3 pb-4 border-b border-border
     last:border-b-0"&gt;
```

```
            &lt;label className="block text-text-primary"&gt;
              {index + 1}. {question}
            &lt;/label&gt;
            &lt;div className="grid grid-cols-2 gap-3"&gt;
              {options.map((option, optionIndex) =&gt; (
                &lt;button
                  key={optionIndex}
                  type="button"
                  onClick={() =&gt; {
                    const newAnswers = [...answers];
                    newAnswers[index] = optionIndex;
                    setAnswers(newAnswers);
                  }}
                  className={`p-4 rounded-lg border-2 transition-all ${
                    answers[index] === optionIndex
                      ? 'bg-primary text-white border-primary'
                      : 'bg-white text-text-secondary border-border
    hover:border-primary'
                  }`}
                &gt;
                  {option}
                &lt;/button&gt;
              ))}
            &lt;/div&gt;
          &lt;/div&gt;
        ))}

        {/* Score Display */}
        &lt;div className="bg-gray-50 p-6 rounded-lg border-2 border-border"&gt;
          &lt;div className="flex items-center justify-between"&gt;
            &lt;span className="text-text-secondary"&gt;■■&lt;/span&gt;
            &lt;span className="text-primary"&gt;{getScore()}/12&lt;/span&gt;
          &lt;/div&gt;
          &lt;p className="text-text-secondary mt-2"&gt;0■■ | 1-2■■■ | 3-6■■■ | 7-12■■■&lt;/p&gt;
        &lt;/div&gt;

        &lt;div className="flex gap-3 justify-end"&gt;
          &lt;button
            type="button"
            onClick={onBack}
            className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
    transition-colors"
          &gt;
            ■■
          &lt;/button&gt;
          &lt;button
            type="submit"
            className="flex items-center gap-2 px-6 py-2 bg-primary
    hover:bg-primary-dark text-white rounded-lg transition-all shadow-md"
          &gt;
            &lt;Save className="w-5 h-5" /&gt;
            ■■■■
          &lt;/button&gt;
        &lt;/div&gt;
      &lt;/form&gt;
    &lt;/div&gt;
  &lt;/div&gt;
  );
}
```

■ File: src\components\questionnaires\BDIQuestionnaire.tsx

```
import { useState } from 'react';
import { ArrowLeft, Save } from 'lucide-react';

interface BDIQuestionnaireProps {
  onBack: () =&gt; void;
  onComplete: () =&gt; void;
}

export function BDIQuestionnaire({ onBack, onComplete }: BDIQuestionnaireProps) {
  const [answers, setAnswers] = useState&lt;number[]&gt;(Array(10).fill(-1));

  const questions = [
    {
      text: '■■',
      options: ['■■■■■■■', '■■■■■■', '■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■',
      options: ['■■■■■■■■■■■■', '■■■■■■■■■', '■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■■■',
      options: ['■■■■■■■■■■■■', '■■■■■■■■■■■■', '■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■■■■',
      options: ['■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■■■',
      options: ['■■■■■■■■■', '■■■■■■■■■■■', '■■■■■■■■■■■■', '■■■■■■■■'],
    },
    {
      text: '■■■■',
      options: ['■■■■■■■■■■■■■', '■■■■■■■■■■■■■', '■■■■■■■■', '■■■■■■■■■■■■'],
    },
    {
      text: '■■■■■',
      options: ['■■■■■■■■■■■■■■■', '■■■■■■■■■', '■■■■■■■■■', '■■■■■■'],
    },
    {
      text: '■■■■■',
      options: ['■■■■■■■■■■■■■■■■', '■■■■■■■■■■■', '■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■',
      options: ['■■■■■■■■■■■■', '■■■■■■■■■■■', '■■■■■■■■■■■■', '■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■■■■',
      options: ['■■■■■■■■■■■■', '■■■■■■■■■■■■', '■■■■■■■■■■■■■■', '■■■■■■■■■■■■'],
    },
  ];

  const handleSubmit = (e: React.FormEvent) =&gt; {
    e.preventDefault();
    if (answers.some((a) =&gt; a === -1)) {
      alert('■■■■■■■■');
      return;
    }

    const total = answers.reduce((sum, val) =&gt; sum + val, 0);
    let level = '';
    if (total &lt;= 13) level = '■■■■■■■';
    else if (total &lt;= 19) level = '■■■■■';
    else if (total &lt;= 28) level = '■■■■■';
    else level = '■■■■■';

    alert(`BDI ■■■■■■\n\n■■■■${total}\n■■■■${level}\n\n■■■■■■■■■■■■■`);
    onComplete();
  };

  const getScore = () =&gt; {
    return answers.reduce((sum, val) =&gt; sum + (val &gt;= 0 ? val : 0), 0);
  };

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="flex items-center gap-4"&gt;
```

```
      &lt;button
        onClick={onBack}
        className="flex items-center gap-2 px-4 py-2 border border-border rounded-lg
    hover:bg-gray-50 transition-colors"
      &gt;
        &lt;ArrowLeft className="w-5 h-5" /&gt;
        ■■
      &lt;/button&gt;
      &lt;div&gt;
        &lt;h2 className="text-text-primary"&gt;BDI ■■■■■■■&lt;/h2&gt;
        &lt;p className="text-text-secondary"&gt;Beck Depression Inventory (■■■)&lt;/p&gt;
      &lt;/div&gt;
     &lt;/div&gt;

     &lt;div className="bg-white rounded-xl shadow-lg p-6 border border-border"&gt;
      &lt;form onSubmit={handleSubmit} className="space-y-6"&gt;
        &lt;div className="bg-medical-blue p-4 rounded-lg"&gt;
          &lt;p className="text-text-primary"&gt;■■■■■■■■■■■■■■■■■■&lt;/p&gt;
        &lt;/div&gt;

        {questions.map((question, index) =&gt; (
          &lt;div key={index} className="space-y-3 pb-4 border-b border-border
    last:border-b-0"&gt;
            &lt;label className="block text-text-primary"&gt;
              {index + 1}. {question.text}
            &lt;/label&gt;
            &lt;div className="space-y-2"&gt;
              {question.options.map((option, optionIndex) =&gt; (
                &lt;button
                  key={optionIndex}
                  type="button"
                  onClick={() =&gt; {
                    const newAnswers = [...answers];
                    newAnswers[index] = optionIndex;
                    setAnswers(newAnswers);
                  }}
                  className={`w-full p-3 rounded-lg border-2 transition-all text-left ${
                    answers[index] === optionIndex
                      ? 'bg-primary text-white border-primary'
                      : 'bg-white text-text-secondary border-border
    hover:border-primary'
                  }`}
                &gt;
                  {optionIndex}. {option}
                &lt;/button&gt;
              ))}
            &lt;/div&gt;
          &lt;/div&gt;
        ))}

        {/* Score Display */}
        &lt;div className="bg-gray-50 p-6 rounded-lg border-2 border-border"&gt;
          &lt;div className="flex items-center justify-between"&gt;
            &lt;span className="text-text-secondary"&gt;■■&lt;/span&gt;
            &lt;span className="text-primary"&gt;{getScore()}/39&lt;/span&gt;
          &lt;/div&gt;
          &lt;p className="text-text-secondary mt-2"&gt;0-13■■■■■ | 14-19■■■ | 20-28■■■ |
    29-63■■■&lt;/p&gt;
        &lt;/div&gt;

        &lt;div className="flex gap-3 justify-end"&gt;
          &lt;button
            type="button"
            onClick={onBack}
            className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
    transition-colors"
          &gt;
            ■■
          &lt;/button&gt;
          &lt;button
            type="submit"
            className="flex items-center gap-2 px-6 py-2 bg-primary
    hover:bg-primary-dark text-white rounded-lg transition-all shadow-md"
          &gt;
            &lt;Save className="w-5 h-5" /&gt;
            ■■■■
          &lt;/button&gt;
        &lt;/div&gt;
      &lt;/form&gt;
     &lt;/div&gt;
    &lt;/div&gt;
  );
```

```
}
```

■ File: src\components\questionnaires\FSSQuestionnaire.tsx

```
import { useState } from 'react';
import { ArrowLeft, Save } from 'lucide-react';

interface FSSQuestionnaireProps {
  onBack: () =&gt; void;
  onComplete: () =&gt; void;
}

export function FSSQuestionnaire({ onBack, onComplete }: FSSQuestionnaireProps) {
  const [answers, setAnswers] = useState&lt;number[]&gt;(Array(9).fill(-1));

  const questions = [
    '■■■■■■■',
    '■■■■■■■■',
    '■■■■■■',
    '■■■■■■■■■■■■',
    '■■■■■■■■■■',
    '■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■',
  ];

  const options = ['■■■■■■', '■■■■', '■■■■■■', '■■■', '■■■■■', '■■■', '■■■■■'];

  const handleSubmit = (e: React.FormEvent) =&gt; {
    e.preventDefault();
    if (answers.some((a) =&gt; a === -1)) {
      alert('■■■■■■■■');
      return;
    }

    const total = answers.reduce((sum, val) =&gt; sum + (val + 1), 0);
    const average = (total / 9).toFixed(1);
    const level = parseFloat(average) &gt;= 4 ? '■■■■■' : '■■■■■';

    alert(`FSS ■■■■■■\n\n■■■■■■${average}\n■■■■${level}\n\n■■■■■■■■■■■■`);
    onComplete();
  };

  const getAverage = () =&gt; {
    const validAnswers = answers.filter((a) =&gt; a &gt;= 0);
    if (validAnswers.length === 0) return 0;
    const total = validAnswers.reduce((sum, val) =&gt; sum + (val + 1), 0);
    return (total / validAnswers.length).toFixed(1);
  };

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="flex items-center gap-4"&gt;
        &lt;button
          onClick={onBack}
          className="flex items-center gap-2 px-4 py-2 border border-border rounded-lg
     hover:bg-gray-50 transition-colors"
        &gt;
          &lt;ArrowLeft className="w-5 h-5" /&gt;
          ■■
        &lt;/button&gt;
        &lt;div&gt;
          &lt;h2 className="text-text-primary"&gt;FSS ■■■■■■■■&lt;/h2&gt;
          &lt;p className="text-text-secondary"&gt;Fatigue Severity Scale&lt;/p&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      &lt;div className="bg-white rounded-xl shadow-lg p-6 border border-border"&gt;
        &lt;form onSubmit={handleSubmit} className="space-y-6"&gt;
          &lt;div className="bg-medical-blue p-4 rounded-lg"&gt;
            &lt;p className="text-text-primary"&gt;■■■■■■■■■■■■■■■■■■■■■■■1-7■■&lt;/p&gt;
          &lt;/div&gt;

          {questions.map((question, index) =&gt; (
            &lt;div key={index} className="space-y-3 pb-4 border-b border-border
     last:border-b-0"&gt;
              &lt;label className="block text-text-primary"&gt;
                {index + 1}. {question}
              &lt;/label&gt;
              &lt;div className="grid grid-cols-7 gap-2"&gt;
                {options.map((option, optionIndex) =&gt; (
                  &lt;button
```

```
                key={optionIndex}
                type="button"
                onClick={() =&gt; {
                  const newAnswers = [...answers];
                  newAnswers[index] = optionIndex;
                  setAnswers(newAnswers);
                }}
                className={`p-3 rounded-lg border-2 transition-all text-center ${
                  answers[index] === optionIndex
                    ? 'bg-primary text-white border-primary'
                    : 'bg-white text-text-secondary border-border
    hover:border-primary'
                }`}
                title={option}
              &gt;
                {optionIndex + 1}
              &lt;/button&gt;
            ))}
          &lt;/div&gt;
          &lt;div className="flex justify-between text-text-secondary"&gt;
            &lt;span&gt;■■■■■■&lt;/span&gt;
            &lt;span&gt;■■■■■&lt;/span&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      ))}

      {/* Score Display */}
      &lt;div className="bg-gray-50 p-6 rounded-lg border-2 border-border"&gt;
        &lt;div className="flex items-center justify-between"&gt;
          &lt;span className="text-text-secondary"&gt;■■■■&lt;/span&gt;
          &lt;span className="text-primary"&gt;{getAverage()}/7&lt;/span&gt;
        &lt;/div&gt;
        &lt;p className="text-text-secondary mt-2"&gt;■■■■ ≥ 4 ■■■■■■&lt;/p&gt;
      &lt;/div&gt;

      &lt;div className="flex gap-3 justify-end"&gt;
        &lt;button
          type="button"
          onClick={onBack}
          className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
    transition-colors"
        &gt;
          ■■
        &lt;/button&gt;
        &lt;button
          type="submit"
          className="flex items-center gap-2 px-6 py-2 bg-primary
    hover:bg-primary-dark text-white rounded-lg transition-all shadow-md"
        &gt;
          &lt;Save className="w-5 h-5" /&gt;
          ■■■■
        &lt;/button&gt;
      &lt;/div&gt;
    &lt;/form&gt;
  &lt;/div&gt;
&lt;/div&gt;
  );
}
```

■ File: src\components\questionnaires\HADSQuestionnaire.tsx

```
import { useState } from 'react';
import { ArrowLeft, Save } from 'lucide-react';

interface HADSQuestionnaireProps {
  onBack: () =&gt; void;
  onComplete: () =&gt; void;
}

export function HADSQuestionnaire({ onBack, onComplete }: HADSQuestionnaireProps) {
  const [answers, setAnswers] = useState&lt;number[]&gt;(Array(14).fill(-1));

  const questions = [
    { text: '■■■■■■■■■', type: 'A', options: ['■■■■■', '■■', '■■', '■■■■■'] },
    { text: '■■■■■■■■■■■■■■■', type: 'D', options: ['■■■■■', '■■■■■', '■■■■■■', '■■■■■'] },
    { text: '■■■■■■■■■■■■■■■■■■■■■■■■■', type: 'A', options: ['■■■■■', '■■■■', '■■■■■■■■■■',
     '■■■■■'] },
    { text: '■■■■■■■■■■■■■■■■■■■', type: 'D', options: ['■■■■■■', '■■■■■■■■', '■■■■■■',
     '■■■■■'] },
    { text: '■■■■■■■■■■■■■', type: 'A', options: ['■■■■■', '■■■■', '■■■■■■', '■■■■■'] },
    { text: '■■■■■■', type: 'D', options: ['■■■■■', '■■■■■', '■■■■■', '■■■■■■'] },
    { text: '■■■■■■■■■■■■■■■', type: 'A', options: ['■■■■■', '■■■■■', '■■■■■', '■■■■■'] },
    { text: '■■■■■■■■', type: 'D', options: ['■■■■■', '■■■■■', '■■■■■', '■■■■■'] },
    { text: '■■■■■■■■■■■■■■■■■■■■■■', type: 'A', options: ['■■■■■', '■■■■', '■■■■■', '■■■■■'] },
    { text: '■■■■■■■■■■■■', type: 'D', options: ['■■■■■', '■■■■■■■', '■■■■■■■', '■■■■■■'] },
    { text: '■■■■■■■■■■■■■■■■■■', type: 'A', options: ['■■■■■', '■■■■■', '■■■■■', '■■■■■'] },
    { text: '■■■■■■■■', type: 'D', options: ['■■■■■■', '■■■■■■■', '■■■■■■■', '■■■■■'] },
    { text: '■■■■■■■■', type: 'A', options: ['■■■■■', '■■■■■', '■■■■■', '■■■■■'] },
    { text: '■■■■■■■■■■■■■■■■■■■■', type: 'D', options: ['■■■■■', '■■■■■', '■■■■■', '■■■■■'] },
  ];

  const handleSubmit = (e: React.FormEvent) =&gt; {
    e.preventDefault();
    if (answers.some((a) =&gt; a === -1)) {
      alert('■■■■■■■■');
      return;
    }

    const anxietyScore = answers
      .filter((_, idx) =&gt; questions[idx].type === 'A')
      .reduce((sum, val) =&gt; sum + val, 0);

    const depressionScore = answers
      .filter((_, idx) =&gt; questions[idx].type === 'D')
      .reduce((sum, val) =&gt; sum + val, 0);

    alert(
      `HADS ■■■■■■\n\n■■■■■ (A)■${anxietyScore}\n■■■■■ (D)■${depressionScore}\n\n■■■■■■■■■■■■`
    );
    onComplete();
  };

  const getAnxietyScore = () =&gt; {
    return answers
      .filter((_, idx) =&gt; questions[idx].type === 'A')
      .reduce((sum, val) =&gt; sum + (val &gt;= 0 ? val : 0), 0);
  };

  const getDepressionScore = () =&gt; {
    return answers
      .filter((_, idx) =&gt; questions[idx].type === 'D')
      .reduce((sum, val) =&gt; sum + (val &gt;= 0 ? val : 0), 0);
  };

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="flex items-center gap-4"&gt;
        &lt;button
          onClick={onBack}
          className="flex items-center gap-2 px-4 py-2 border border-border rounded-lg
     hover:bg-gray-50 transition-colors"
        &gt;
          &lt;ArrowLeft className="w-5 h-5" /&gt;
          ■■
        &lt;/button&gt;
        &lt;div&gt;
          &lt;h2 className="text-text-primary"&gt;HADS ■■■■■■■■■&lt;/h2&gt;
          &lt;p className="text-text-secondary"&gt;Hospital Anxiety and Depression Scale&lt;/p&gt;
        &lt;/div&gt;
      &lt;/div&gt;
```

```
      &lt;div className="bg-white rounded-xl shadow-lg p-6 border border-border"&gt;
        &lt;form onSubmit={handleSubmit} className="space-y-6"&gt;
          &lt;div className="bg-medical-blue p-4 rounded-lg"&gt;
            &lt;p className="text-text-primary"&gt;
              ■■■■■■■■■■■■■■■■■■
            &lt;/p&gt;
          &lt;/div&gt;

          {questions.map((question, index) =&gt; (
            &lt;div key={index} className="space-y-3 pb-4 border-b border-border
    last:border-b-0"&gt;
              &lt;label className="block text-text-primary"&gt;
                {index + 1}. {question.text}
              &lt;/label&gt;
              &lt;div className="grid grid-cols-1 md:grid-cols-2 gap-2"&gt;
                {question.options.map((option, optionIndex) =&gt; (
                  &lt;button
                    key={optionIndex}
                    type="button"
                    onClick={() =&gt; {
                      const newAnswers = [...answers];
                      newAnswers[index] = optionIndex;
                      setAnswers(newAnswers);
                    }}
                    className={`p-3 rounded-lg border-2 transition-all text-left ${
                      answers[index] === optionIndex
                        ? 'bg-primary text-white border-primary'
                        : 'bg-white text-text-secondary border-border
    hover:border-primary'
                    }`}
                  &gt;
                    {option}
                  &lt;/button&gt;
                ))}
              &lt;/div&gt;
            &lt;/div&gt;
          ))}

          {/* Score Display */}
          &lt;div className="bg-gray-50 p-6 rounded-lg border-2 border-border space-y-3"&gt;
            &lt;h4 className="text-text-primary"&gt;■■&lt;/h4&gt;
            &lt;div className="flex items-center justify-between"&gt;
              &lt;span className="text-text-secondary"&gt;■■■■ (A)&lt;/span&gt;
              &lt;span className="text-primary"&gt;{getAnxietyScore()}/21&lt;/span&gt;
            &lt;/div&gt;
            &lt;div className="flex items-center justify-between"&gt;
              &lt;span className="text-text-secondary"&gt;■■■■ (D)&lt;/span&gt;
              &lt;span className="text-primary"&gt;{getDepressionScore()}/21&lt;/span&gt;
            &lt;/div&gt;
            &lt;p className="text-text-secondary"&gt;0-7■■■ | 8-10■■■ | 11-21■■■&lt;/p&gt;
          &lt;/div&gt;

          &lt;div className="flex gap-3 justify-end"&gt;
            &lt;button
              type="button"
              onClick={onBack}
              className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
    transition-colors"
            &gt;
              ■■
            &lt;/button&gt;
            &lt;button
              type="submit"
              className="flex items-center gap-2 px-6 py-2 bg-primary
    hover:bg-primary-dark text-white rounded-lg transition-all shadow-md"
            &gt;
              &lt;Save className="w-5 h-5" /&gt;
              ■■■■
            &lt;/button&gt;
          &lt;/div&gt;
        &lt;/form&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

■ File: src\components\questionnaires\MIDASQuestionnaire.tsx

```
import { useState } from 'react';
import { ArrowLeft, Save } from 'lucide-react';

interface MIDASQuestionnaireProps {
  onBack: () =&gt; void;
  onComplete: () =&gt; void;
}

export function MIDASQuestionnaire({ onBack, onComplete }: MIDASQuestionnaireProps) {
  const [answers, setAnswers] = useState&lt;(number | '')[]&gt;(Array(5).fill(''));

  const questions = [
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■1■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■3■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
  ];

  const handleSubmit = (e: React.FormEvent) =&gt; {
    e.preventDefault();
    if (answers.some((a) =&gt; a === '')) {
      alert('■■■■■■■■■■■');
      return;
    }
    const total = answers.reduce((sum, val) =&gt; sum + (val === '' ? 0 : val), 0);
    alert(`■■ MIDAS ■■■■■${total}\n\n■■■■■■■■■■■■■`);
    onComplete();
  };

  const getGrade = () =&gt; {
    const total = answers.reduce((sum, val) =&gt; sum + (val === '' ? 0 : val), 0);
    if (total &lt;= 5) return { grade: 'I', level: '■■■■■■■', color: 'text-success' };
    if (total &lt;= 10) return { grade: 'II', level: '■■■■■', color: 'text-warning' };
    if (total &lt;= 20) return { grade: 'III', level: '■■■■■', color: 'text-warning' };
    return { grade: 'IV', level: '■■■■■', color: 'text-danger' };
  };

  const grade = getGrade();

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="flex items-center gap-4"&gt;
        &lt;button
          onClick={onBack}
          className="flex items-center gap-2 px-4 py-2 border border-border rounded-lg
     hover:bg-gray-50 transition-colors"
        &gt;
          &lt;ArrowLeft className="w-5 h-5" /&gt;
          ■■
        &lt;/button&gt;
        &lt;div&gt;
          &lt;h2 className="text-text-primary"&gt;MIDAS ■■■■■■■■■■&lt;/h2&gt;
          &lt;p className="text-text-secondary"&gt;Migraine Disability Assessment&lt;/p&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      &lt;div className="bg-white rounded-xl shadow-lg p-6 border border-border"&gt;
        &lt;form onSubmit={handleSubmit} className="space-y-6"&gt;
          &lt;div className="bg-medical-blue p-4 rounded-lg"&gt;
            &lt;p className="text-text-primary"&gt;
              ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ 0■
            &lt;/p&gt;
          &lt;/div&gt;

          {questions.map((question, index) =&gt; (
            &lt;div key={index} className="space-y-2"&gt;
              &lt;label className="block text-text-primary"&gt;
                {index + 1}. {question}
              &lt;/label&gt;
              &lt;input
                type="number"
                min="0"
                max="90"
                value={answers[index]}
                onChange={(e) =&gt; {
                  const newAnswers = [...answers];
                  newAnswers[index] = e.target.value === '' ? '' :
     parseInt(e.target.value);
```

```
                setAnswers(newAnswers);
              }}
              className="w-full md:w-48 px-4 py-3 border border-border rounded-lg
    focus:outline-none focus:ring-2 focus:ring-primary"
              placeholder="■■"
              required
            /&gt;
          &lt;/div&gt;
        ))}

        {/* Score Display */}
        &lt;div className="bg-gray-50 p-6 rounded-lg border-2 border-border"&gt;
          &lt;div className="flex items-center justify-between mb-4"&gt;
            &lt;h4 className="text-text-primary"&gt;■■&lt;/h4&gt;
            &lt;span className="text-primary"&gt;{answers.reduce((sum, val) =&gt; sum + (val ===
    '' ? 0 : val), 0)} ■&lt;/span&gt;
          &lt;/div&gt;
          &lt;div className="flex items-center justify-between"&gt;
            &lt;span className="text-text-secondary"&gt;■■&lt;/span&gt;
            &lt;span className={`${grade.color}`}&gt;
              Grade {grade.grade} - {grade.level}
            &lt;/span&gt;
          &lt;/div&gt;
        &lt;/div&gt;

        &lt;div className="flex gap-3 justify-end"&gt;
          &lt;button
            type="button"
            onClick={onBack}
            className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
    transition-colors"
          &gt;
            ■■
          &lt;/button&gt;
          &lt;button
            type="submit"
            className="flex items-center gap-2 px-6 py-2 bg-primary
    hover:bg-primary-dark text-white rounded-lg transition-all shadow-md"
          &gt;
            &lt;Save className="w-5 h-5" /&gt;
            ■■■■
          &lt;/button&gt;
        &lt;/div&gt;
      &lt;/form&gt;
    &lt;/div&gt;
  &lt;/div&gt;
  );
}
```

■ File: src\components\questionnaires\PSQIQuestionnaire.tsx

```
import { useState } from 'react';
import { ArrowLeft, Save } from 'lucide-react';

interface PSQIQuestionnaireProps {
  onBack: () =&gt; void;
  onComplete: () =&gt; void;
}

export function PSQIQuestionnaire({ onBack, onComplete }: PSQIQuestionnaireProps) {
  const [bedTime, setBedTime] = useState('23:00');
  const [fallAsleepTime, setFallAsleepTime] = useState('30');
  const [wakeTime, setWakeTime] = useState('07:00');
  const [sleepHours, setSleepHours] = useState('7');
  const [answers, setAnswers] = useState&lt;number[]&gt;(Array(7).fill(-1));

  const questions = [
    {
      text: '■■■■■■■■■■ 30 ■■■■■■■■■■',
      options: ['■■', '■■■■■■■', '■■■■■■■', '■■■■■■■■'],
    },
    {
      text: '■■■■■■■■■■■■■■■■■■■■■',
      options: ['■■', '■■■■■■■', '■■■■■■■', '■■■■■■■■'],
    },
    {
      text: '■■■■■■■■■■■■■■■■■■■■■■■',
      options: ['■■', '■■■■■■■', '■■■■■■■', '■■■■■■■■'],
    },
    {
      text: '■■■■■■■■■■■■■■■■■■■■■■■',
      options: ['■■', '■■■■■■■', '■■■■■■■', '■■■■■■■■'],
    },
    {
      text: '■■■■■■■■■■■■■■■■■■■■',
      options: ['■■■■', '■■■■', '■■■■', '■■■■■'],
    },
    {
      text: '■■■■■■■■■■■■■■■■■■■■■■■',
      options: ['■■', '■■■■■■■', '■■■■■■■', '■■■■■■■■'],
    },
    {
      text: '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
      options: ['■■', '■■■■■■■', '■■■■■■■', '■■■■■■■■'],
    },
  ];

  const handleSubmit = (e: React.FormEvent) =&gt; {
    e.preventDefault();
    if (!bedTime || !fallAsleepTime || !wakeTime || !sleepHours) {
      alert('■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■');
      return;
    }
    if (answers.some((a) =&gt; a === -1)) {
      alert('■■■■■■■■');
      return;
    }

    const total = answers.reduce((sum, val) =&gt; sum + val, 0);
    const quality = total &gt; 5 ? '■■■■■■■' : '■■■■■■■';

    alert(`PSQI ■■■■■■\n\n■■■■${total}\n■■■■${quality}\n\n■■■■■■■■■■■■■`);
    onComplete();
  };

  const getScore = () =&gt; {
    return answers.reduce((sum, val) =&gt; sum + (val &gt;= 0 ? val : 0), 0);
  };

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="flex items-center gap-4"&gt;
        &lt;button
          onClick={onBack}
          className="flex items-center gap-2 px-4 py-2 border border-border rounded-lg
     hover:bg-gray-50 transition-colors"
        &gt;
          &lt;ArrowLeft className="w-5 h-5" /&gt;
          ■■
        &lt;/button&gt;
```

```
   &lt;div&gt;
     &lt;h2 className="text-text-primary"&gt;PSQI ■■■■■■■■■■&lt;/h2&gt;
     &lt;p className="text-text-secondary"&gt;Pittsburgh Sleep Quality Index&lt;/p&gt;
   &lt;/div&gt;
 &lt;/div&gt;

 &lt;div className="bg-white rounded-xl shadow-lg p-6 border border-border"&gt;
   &lt;form onSubmit={handleSubmit} className="space-y-6"&gt;
     &lt;div className="bg-medical-blue p-4 rounded-lg"&gt;
       &lt;p className="text-text-primary"&gt;■■■■■■■■■■■■■■■■■■■■&lt;/p&gt;
     &lt;/div&gt;

     {/* Basic sleep info */}
     &lt;div className="grid grid-cols-1 md:grid-cols-2 gap-4"&gt;
       &lt;div&gt;
         &lt;label className="block mb-2"&gt;■■■■■■■&lt;/label&gt;
         &lt;input
           type="time"
           value={bedTime}
           onChange={(e) =&gt; setBedTime(e.target.value)}
           className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
         /&gt;
       &lt;/div&gt;
       &lt;div&gt;
         &lt;label className="block mb-2"&gt;■■■■■■■■■■■&lt;/label&gt;
         &lt;input
           type="number"
           value={fallAsleepTime}
           onChange={(e) =&gt; setFallAsleepTime(e.target.value)}
           className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
         /&gt;
       &lt;/div&gt;
       &lt;div&gt;
         &lt;label className="block mb-2"&gt;■■■■■■■&lt;/label&gt;
         &lt;input
           type="time"
           value={wakeTime}
           onChange={(e) =&gt; setWakeTime(e.target.value)}
           className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
         /&gt;
       &lt;/div&gt;
       &lt;div&gt;
         &lt;label className="block mb-2"&gt;■■■■■■■&lt;/label&gt;
         &lt;input
           type="number"
           value={sleepHours}
           onChange={(e) =&gt; setSleepHours(e.target.value)}
           className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
           step="0.5"
         /&gt;
       &lt;/div&gt;
     &lt;/div&gt;

     {questions.map((question, index) =&gt; (
       &lt;div key={index} className="space-y-3 pb-4 border-b border-border
last:border-b-0"&gt;
         &lt;label className="block text-text-primary"&gt;{question.text}&lt;/label&gt;
         &lt;div className="grid grid-cols-1 md:grid-cols-2 gap-2"&gt;
           {question.options.map((option, optionIndex) =&gt; (
             &lt;button
               key={optionIndex}
               type="button"
               onClick={() =&gt; {
                 const newAnswers = [...answers];
                 newAnswers[index] = optionIndex;
                 setAnswers(newAnswers);
               }}
               className={`p-3 rounded-lg border-2 transition-all text-left ${
                 answers[index] === optionIndex
                   ? 'bg-primary text-white border-primary'
                   : 'bg-white text-text-secondary border-border
hover:border-primary'
               }`}
             &gt;
               {option}
             &lt;/button&gt;
           ))}
         &lt;/div&gt;
```

```
            &lt;/div&gt;
          ))}

          {/* Score Display */}
          &lt;div className="bg-gray-50 p-6 rounded-lg border-2 border-border"&gt;
            &lt;div className="flex items-center justify-between"&gt;
              &lt;span className="text-text-secondary"&gt;■■&lt;/span&gt;
              &lt;span className="text-primary"&gt;{getScore()}/21&lt;/span&gt;
            &lt;/div&gt;
            &lt;p className="text-text-secondary mt-2"&gt;■■ {'&gt;'} 5 ■■■■■■■■■&lt;/p&gt;
          &lt;/div&gt;

          &lt;div className="flex gap-3 justify-end"&gt;
            &lt;button
              type="button"
              onClick={onBack}
              className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
      transition-colors"
            &gt;
              ■■
            &lt;/button&gt;
            &lt;button
              type="submit"
              className="flex items-center gap-2 px-6 py-2 bg-primary
      hover:bg-primary-dark text-white rounded-lg transition-all shadow-md"
            &gt;
              &lt;Save className="w-5 h-5" /&gt;
              ■■■■
            &lt;/button&gt;
          &lt;/div&gt;
        &lt;/form&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

■ File: src\components\questionnaires\PSSQuestionnaire.tsx

```
import { useState } from 'react';
import { ArrowLeft, Save } from 'lucide-react';

interface PSSQuestionnaireProps {
  onBack: () =&gt; void;
  onComplete: () =&gt; void;
}

export function PSSQuestionnaire({ onBack, onComplete }: PSSQuestionnaireProps) {
  const [answers, setAnswers] = useState&lt;number[]&gt;(Array(10).fill(-1));

  const questions = [
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■',
  ];

  const options = ['■■', '■■■■', '■■■■', '■■■■■', '■■■■■'];

  const handleSubmit = (e: React.FormEvent) =&gt; {
    e.preventDefault();
    if (answers.some((a) =&gt; a === -1)) {
      alert('■■■■■■■■');
      return;
    }

    // Reverse scoring for items 3, 4, 6, 7
    const reverseItems = [3, 4, 6, 7];
    let total = 0;
    answers.forEach((answer, index) =&gt; {
      if (reverseItems.includes(index)) {
        total += 4 - answer;
      } else {
        total += answer;
      }
    });

    let level = '';
    if (total &lt;= 13) level = '■■■■';
    else if (total &lt;= 26) level = '■■■■■';
    else level = '■■■■';

    alert(`PSS ■■■■■■\n\n■■■■${total}\n■■■■${level}\n\n■■■■■■■■■■■■■`);
    onComplete();
  };

  const getScore = () =&gt; {
    const reverseItems = [3, 4, 6, 7];
    let total = 0;
    answers.forEach((answer, index) =&gt; {
      if (answer &gt;= 0) {
        if (reverseItems.includes(index)) {
          total += 4 - answer;
        } else {
          total += answer;
        }
      }
    });
    return total;
  };

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="flex items-center gap-4"&gt;
        &lt;button
          onClick={onBack}
          className="flex items-center gap-2 px-4 py-2 border border-border rounded-lg
     hover:bg-gray-50 transition-colors"
        &gt;
          &lt;ArrowLeft className="w-5 h-5" /&gt;
          ■■
        &lt;/button&gt;
```

```
        &lt;div&gt;
          &lt;h2 className="text-text-primary"&gt;PSS ■■■■■■■&lt;/h2&gt;
          &lt;p className="text-text-secondary"&gt;Perceived Stress Scale&lt;/p&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      &lt;div className="bg-white rounded-xl shadow-lg p-6 border border-border"&gt;
        &lt;form onSubmit={handleSubmit} className="space-y-6"&gt;
          &lt;div className="bg-medical-blue p-4 rounded-lg"&gt;
            &lt;p className="text-text-primary"&gt;
              ■■■■■■■■■■■■■■■■■■
            &lt;/p&gt;
          &lt;/div&gt;

          {questions.map((question, index) =&gt; (
            &lt;div key={index} className="space-y-3 pb-4 border-b border-border
      last:border-b-0"&gt;
              &lt;label className="block text-text-primary"&gt;
                {index + 1}. {question}
              &lt;/label&gt;
              &lt;div className="grid grid-cols-5 gap-2"&gt;
                {options.map((option, optionIndex) =&gt; (
                  &lt;button
                    key={optionIndex}
                    type="button"
                    onClick={() =&gt; {
                      const newAnswers = [...answers];
                      newAnswers[index] = optionIndex;
                      setAnswers(newAnswers);
                    }}
                    className={`p-3 rounded-lg border-2 transition-all text-center ${
                      answers[index] === optionIndex
                        ? 'bg-primary text-white border-primary'
                        : 'bg-white text-text-secondary border-border
      hover:border-primary'
                    }`}
                  &gt;
                    &lt;div className="hidden md:block"&gt;{option}&lt;/div&gt;
                    &lt;div className="md:hidden"&gt;{optionIndex}&lt;/div&gt;
                  &lt;/button&gt;
                ))}
              &lt;/div&gt;
              &lt;div className="flex justify-between text-text-secondary md:hidden"&gt;
                &lt;span&gt;■■&lt;/span&gt;
                &lt;span&gt;■■■■■&lt;/span&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          ))}

          {/* Score Display */}
          &lt;div className="bg-gray-50 p-6 rounded-lg border-2 border-border"&gt;
            &lt;div className="flex items-center justify-between"&gt;
              &lt;span className="text-text-secondary"&gt;■■&lt;/span&gt;
              &lt;span className="text-primary"&gt;{getScore()}/40&lt;/span&gt;
            &lt;/div&gt;
            &lt;p className="text-text-secondary mt-2"&gt;0-13■■■■■ | 14-26■■■■■ | 27-40■■■■■&lt;/p&gt;
          &lt;/div&gt;

          &lt;div className="flex gap-3 justify-end"&gt;
            &lt;button
              type="button"
              onClick={onBack}
              className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
      transition-colors"
            &gt;
              ■■
            &lt;/button&gt;
            &lt;button
              type="submit"
              className="flex items-center gap-2 px-6 py-2 bg-primary
      hover:bg-primary-dark text-white rounded-lg transition-all shadow-md"
            &gt;
              &lt;Save className="w-5 h-5" /&gt;
              ■■■■
            &lt;/button&gt;
          &lt;/div&gt;
        &lt;/form&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

■ File: src\components\questionnaires\WPIQuestionnaire.tsx

```
import { useState } from 'react';
import { ArrowLeft, Save } from 'lucide-react';

interface WPIQuestionnaireProps {
  onBack: () =&gt; void;
  onComplete: () =&gt; void;
}

export function WPIQuestionnaire({ onBack, onComplete }: WPIQuestionnaireProps) {
  const [selectedAreas, setSelectedAreas] = useState&lt;string[]&gt;([]);

  const bodyAreas = [
    { id: 'shoulder-left', label: '■■' },
    { id: 'shoulder-right', label: '■■' },
    { id: 'arm-left', label: '■■■' },
    { id: 'arm-right', label: '■■■' },
    { id: 'forearm-left', label: '■■■' },
    { id: 'forearm-right', label: '■■■' },
    { id: 'hip-left', label: '■■■' },
    { id: 'hip-right', label: '■■■' },
    { id: 'thigh-left', label: '■■■' },
    { id: 'thigh-right', label: '■■■' },
    { id: 'calf-left', label: '■■■' },
    { id: 'calf-right', label: '■■■' },
    { id: 'jaw-left', label: '■■■' },
    { id: 'jaw-right', label: '■■■' },
    { id: 'chest', label: '■■' },
    { id: 'abdomen', label: '■■' },
    { id: 'upper-back', label: '■■' },
    { id: 'lower-back', label: '■■' },
    { id: 'neck', label: '■■' },
  ];

  const toggleArea = (areaId: string) =&gt; {
    setSelectedAreas((prev) =&gt;
      prev.includes(areaId) ? prev.filter((id) =&gt; id !== areaId) : [...prev, areaId]
    );
  };

  const handleSubmit = (e: React.FormEvent) =&gt; {
    e.preventDefault();
    const score = selectedAreas.length;
    alert(`WPI ■■■■■■\n\n■■■■■■■■${score}/19\n\n■■■■■■■■■■■■`);
    onComplete();
  };

  return (
    &lt;div className="space-y-6"&gt;
      &lt;div className="flex items-center gap-4"&gt;
        &lt;button
          onClick={onBack}
          className="flex items-center gap-2 px-4 py-2 border border-border rounded-lg
     hover:bg-gray-50 transition-colors"
        &gt;
          &lt;ArrowLeft className="w-5 h-5" /&gt;
          ■■
        &lt;/button&gt;
        &lt;div&gt;
          &lt;h2 className="text-text-primary"&gt;WPI ■■■■■■■&lt;/h2&gt;
          &lt;p className="text-text-secondary"&gt;Widespread Pain Index&lt;/p&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      &lt;div className="bg-white rounded-xl shadow-lg p-6 border border-border"&gt;
        &lt;form onSubmit={handleSubmit} className="space-y-6"&gt;
          &lt;div className="bg-medical-blue p-4 rounded-lg"&gt;
            &lt;p className="text-text-primary"&gt;
              ■■■■■■■■■■■■■■■■■■■■■■
            &lt;/p&gt;
          &lt;/div&gt;

          &lt;div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-3"&gt;
            {bodyAreas.map((area) =&gt; (
              &lt;button
                key={area.id}
                type="button"
                onClick={() =&gt; toggleArea(area.id)}
                className={`p-4 rounded-lg border-2 transition-all ${
                  selectedAreas.includes(area.id)
```

```
                  ? 'bg-primary text-white border-primary'
                  : 'bg-white text-text-secondary border-border hover:border-primary'
              }`}
            &gt;
              {area.label}
            &lt;/button&gt;
          ))}
        &lt;/div&gt;

        {/* Score Display */}
        &lt;div className="bg-gray-50 p-6 rounded-lg border-2 border-border"&gt;
          &lt;div className="flex items-center justify-between"&gt;
            &lt;span className="text-text-secondary"&gt;■■■■■■■&lt;/span&gt;
            &lt;span className="text-primary"&gt;{selectedAreas.length}/19&lt;/span&gt;
          &lt;/div&gt;
        &lt;/div&gt;

        &lt;div className="flex gap-3 justify-end"&gt;
          &lt;button
            type="button"
            onClick={onBack}
            className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
    transition-colors"
          &gt;
            ■■
          &lt;/button&gt;
          &lt;button
            type="submit"
            className="flex items-center gap-2 px-6 py-2 bg-primary
    hover:bg-primary-dark text-white rounded-lg transition-all shadow-md"
          &gt;
            &lt;Save className="w-5 h-5" /&gt;
            ■■■■
          &lt;/button&gt;
        &lt;/div&gt;
      &lt;/form&gt;
    &lt;/div&gt;
  &lt;/div&gt;
);
}
```

■ File: src\components\ui\accordion.tsx

```
"use client";

import * as React from "react";
import * as AccordionPrimitive from "@radix-ui/react-accordion@1.2.3";
import { ChevronDownIcon } from "lucide-react@0.487.0";

import { cn } from "./utils";

function Accordion({
  ...props
}: React.ComponentProps&lt;typeof AccordionPrimitive.Root&gt;) {
  return &lt;AccordionPrimitive.Root data-slot="accordion" {...props} /&gt;;
}

function AccordionItem({
  className,
  ...props
}: React.ComponentProps&lt;typeof AccordionPrimitive.Item&gt;) {
  return (
    &lt;AccordionPrimitive.Item
      data-slot="accordion-item"
      className={cn("border-b last:border-b-0", className)}
      {...props}
    /&gt;
  );
}

function AccordionTrigger({
  className,
  children,
  ...props
}: React.ComponentProps&lt;typeof AccordionPrimitive.Trigger&gt;) {
  return (
    &lt;AccordionPrimitive.Header className="flex"&gt;
      &lt;AccordionPrimitive.Trigger
        data-slot="accordion-trigger"
        className={cn(
          "focus-visible:border-ring focus-visible:ring-ring/50 flex flex-1 items-start
     justify-between gap-4 rounded-md py-4 text-left text-sm font-medium transition-all
     outline-none hover:underline focus-visible:ring-[3px] disabled:pointer-events-none
     disabled:opacity-50 [&amp;[data-state=open]&gt;svg]:rotate-180",
          className,
        )}
        {...props}
      &gt;
        {children}
        &lt;ChevronDownIcon className="text-muted-foreground pointer-events-none size-4
     shrink-0 translate-y-0.5 transition-transform duration-200" /&gt;
      &lt;/AccordionPrimitive.Trigger&gt;
    &lt;/AccordionPrimitive.Header&gt;
  );
}

function AccordionContent({
  className,
  children,
  ...props
}: React.ComponentProps&lt;typeof AccordionPrimitive.Content&gt;) {
  return (
    &lt;AccordionPrimitive.Content
      data-slot="accordion-content"
      className="data-[state=closed]:animate-accordion-up
     data-[state=open]:animate-accordion-down overflow-hidden text-sm"
      {...props}
    &gt;
      &lt;div className={cn("pt-0 pb-4", className)}&gt;{children}&lt;/div&gt;
    &lt;/AccordionPrimitive.Content&gt;
  );
}

export { Accordion, AccordionItem, AccordionTrigger, AccordionContent };
```

■ File: src\components\ui\alert-dialog.tsx

```
"use client";

import * as React from "react";
import * as AlertDialogPrimitive from "@radix-ui/react-alert-dialog@1.1.6";

import { cn } from "./utils";
import { buttonVariants } from "./button";

function AlertDialog({
  ...props
}: React.ComponentProps&lt;typeof AlertDialogPrimitive.Root&gt;) {
  return &lt;AlertDialogPrimitive.Root data-slot="alert-dialog" {...props} /&gt;;
}

function AlertDialogTrigger({
  ...props
}: React.ComponentProps&lt;typeof AlertDialogPrimitive.Trigger&gt;) {
  return (
    &lt;AlertDialogPrimitive.Trigger data-slot="alert-dialog-trigger" {...props} /&gt;
  );
}

function AlertDialogPortal({
  ...props
}: React.ComponentProps&lt;typeof AlertDialogPrimitive.Portal&gt;) {
  return (
    &lt;AlertDialogPrimitive.Portal data-slot="alert-dialog-portal" {...props} /&gt;
  );
}

function AlertDialogOverlay({
  className,
  ...props
}: React.ComponentProps&lt;typeof AlertDialogPrimitive.Overlay&gt;) {
  return (
    &lt;AlertDialogPrimitive.Overlay
      data-slot="alert-dialog-overlay"
      className={cn(
        "data-[state=open]:animate-in data-[state=closed]:animate-out
     data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 fixed inset-0 z-50
     bg-black/50",
        className,
      )}
      {...props}
    /&gt;
  );
}

function AlertDialogContent({
  className,
  ...props
}: React.ComponentProps&lt;typeof AlertDialogPrimitive.Content&gt;) {
  return (
    &lt;AlertDialogPortal&gt;
      &lt;AlertDialogOverlay /&gt;
      &lt;AlertDialogPrimitive.Content
        data-slot="alert-dialog-content"
        className={cn(
          "bg-background data-[state=open]:animate-in data-[state=closed]:animate-out
     data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
     data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 fixed top-[50%]
     left-[50%] z-50 grid w-full max-w-[calc(100%-2rem)] translate-x-[-50%]
     translate-y-[-50%] gap-4 rounded-lg border p-6 shadow-lg duration-200 sm:max-w-lg",
          className,
        )}
        {...props}
      /&gt;
    &lt;/AlertDialogPortal&gt;
  );
}

function AlertDialogHeader({
  className,
  ...props
}: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="alert-dialog-header"
      className={cn("flex flex-col gap-2 text-center sm:text-left", className)}
```

```
      {...props}
    /&gt;
  );
}

function AlertDialogFooter({
  className,
  ...props
}: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="alert-dialog-footer"
      className={cn(
        "flex flex-col-reverse gap-2 sm:flex-row sm:justify-end",
        className,
      )}
      {...props}
    /&gt;
  );
}

function AlertDialogTitle({
  className,
  ...props
}: React.ComponentProps&lt;typeof AlertDialogPrimitive.Title&gt;) {
  return (
    &lt;AlertDialogPrimitive.Title
      data-slot="alert-dialog-title"
      className={cn("text-lg font-semibold", className)}
      {...props}
    /&gt;
  );
}

function AlertDialogDescription({
  className,
  ...props
}: React.ComponentProps&lt;typeof AlertDialogPrimitive.Description&gt;) {
  return (
    &lt;AlertDialogPrimitive.Description
      data-slot="alert-dialog-description"
      className={cn("text-muted-foreground text-sm", className)}
      {...props}
    /&gt;
  );
}

function AlertDialogAction({
  className,
  ...props
}: React.ComponentProps&lt;typeof AlertDialogPrimitive.Action&gt;) {
  return (
    &lt;AlertDialogPrimitive.Action
      className={cn(buttonVariants(), className)}
      {...props}
    /&gt;
  );
}

function AlertDialogCancel({
  className,
  ...props
}: React.ComponentProps&lt;typeof AlertDialogPrimitive.Cancel&gt;) {
  return (
    &lt;AlertDialogPrimitive.Cancel
      className={cn(buttonVariants({ variant: "outline" }), className)}
      {...props}
    /&gt;
  );
}

export {
  AlertDialog,
  AlertDialogPortal,
  AlertDialogOverlay,
  AlertDialogTrigger,
  AlertDialogContent,
  AlertDialogHeader,
  AlertDialogFooter,
  AlertDialogTitle,
  AlertDialogDescription,
  AlertDialogAction,
```

```
  AlertDialogCancel,
};
```

■ File: src\components\ui\alert.tsx

```
import * as React from "react";
import { cva, type VariantProps } from "class-variance-authority@0.7.1";

import { cn } from "./utils";

const alertVariants = cva(
  "relative w-full rounded-lg border px-4 py-3 text-sm grid
     has-[&gt;svg]:grid-cols-[calc(var(--spacing)*4)_1fr] grid-cols-[0_1fr]
     has-[&gt;svg]:gap-x-3 gap-y-0.5 items-start [&amp;&gt;svg]:size-4 [&amp;&gt;svg]:translate-y-0.5
     [&amp;&gt;svg]:text-current",
  {
    variants: {
      variant: {
        default: "bg-card text-card-foreground",
        destructive:
          "text-destructive bg-card [&amp;&gt;svg]:text-current
     *:data-[slot=alert-description]:text-destructive/90",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  },
);

function Alert({
  className,
  variant,
  ...props
}: React.ComponentProps&lt;"div"&gt; &amp; VariantProps&lt;typeof alertVariants&gt;) {
  return (
    &lt;div
      data-slot="alert"
      role="alert"
      className={cn(alertVariants({ variant }), className)}
      {...props}
    /&gt;
  );
}

function AlertTitle({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="alert-title"
      className={cn(
        "col-start-2 line-clamp-1 min-h-4 font-medium tracking-tight",
        className,
      )}
      {...props}
    /&gt;
  );
}

function AlertDescription({
  className,
  ...props
}: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="alert-description"
      className={cn(
        "text-muted-foreground col-start-2 grid justify-items-start gap-1 text-sm
     [&amp;_p]:leading-relaxed",
        className,
      )}
      {...props}
    /&gt;
  );
}

export { Alert, AlertTitle, AlertDescription };
```

■ File: src\components\ui\aspect-ratio.tsx

```
"use client";

import * as AspectRatioPrimitive from "@radix-ui/react-aspect-ratio@1.1.2";

function AspectRatio({
  ...props
}: React.ComponentProps&lt;typeof AspectRatioPrimitive.Root&gt;) {
  return &lt;AspectRatioPrimitive.Root data-slot="aspect-ratio" {...props} /&gt;;
}

export { AspectRatio };
```

■ File: src\components\ui\avatar.tsx

```
"use client";

import * as React from "react";
import * as AvatarPrimitive from "@radix-ui/react-avatar@1.1.3";

import { cn } from "./utils";

function Avatar({
  className,
  ...props
}: React.ComponentProps&lt;typeof AvatarPrimitive.Root&gt;) {
  return (
    &lt;AvatarPrimitive.Root
      data-slot="avatar"
      className={cn(
        "relative flex size-10 shrink-0 overflow-hidden rounded-full",
        className,
      )}
      {...props}
    /&gt;
  );
}

function AvatarImage({
  className,
  ...props
}: React.ComponentProps&lt;typeof AvatarPrimitive.Image&gt;) {
  return (
    &lt;AvatarPrimitive.Image
      data-slot="avatar-image"
      className={cn("aspect-square size-full", className)}
      {...props}
    /&gt;
  );
}

function AvatarFallback({
  className,
  ...props
}: React.ComponentProps&lt;typeof AvatarPrimitive.Fallback&gt;) {
  return (
    &lt;AvatarPrimitive.Fallback
      data-slot="avatar-fallback"
      className={cn(
        "bg-muted flex size-full items-center justify-center rounded-full",
        className,
      )}
      {...props}
    /&gt;
  );
}

export { Avatar, AvatarImage, AvatarFallback };
```

■ File: src\components\ui\badge.tsx

```
import * as React from "react";
import { Slot } from "@radix-ui/react-slot@1.1.2";
import { cva, type VariantProps } from "class-variance-authority@0.7.1";

import { cn } from "./utils";

const badgeVariants = cva(
  "inline-flex items-center justify-center rounded-md border px-2 py-0.5 text-xs
     font-medium w-fit whitespace-nowrap shrink-0 [&amp;&gt;svg]:size-3 gap-1
     [&amp;&gt;svg]:pointer-events-none focus-visible:border-ring focus-visible:ring-ring/50
     focus-visible:ring-[3px] aria-invalid:ring-destructive/20
     dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructive
     transition-[color,box-shadow] overflow-hidden",
  {
    variants: {
      variant: {
        default:
          "border-transparent bg-primary text-primary-foreground
     [a&amp;]:hover:bg-primary/90",
        secondary:
          "border-transparent bg-secondary text-secondary-foreground
     [a&amp;]:hover:bg-secondary/90",
        destructive:
          "border-transparent bg-destructive text-white [a&amp;]:hover:bg-destructive/90
     focus-visible:ring-destructive/20 dark:focus-visible:ring-destructive/40
     dark:bg-destructive/60",
        outline:
          "text-foreground [a&amp;]:hover:bg-accent [a&amp;]:hover:text-accent-foreground",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  },
);

function Badge({
  className,
  variant,
  asChild = false,
  ...props
}: React.ComponentProps&lt;"span"&gt; &amp;
  VariantProps&lt;typeof badgeVariants&gt; &amp; { asChild?: boolean }) {
  const Comp = asChild ? Slot : "span";

  return (
    &lt;Comp
      data-slot="badge"
      className={cn(badgeVariants({ variant }), className)}
      {...props}
    /&gt;
  );
}

export { Badge, badgeVariants };
```

■ File: src\components\ui\breadcrumb.tsx

```
import * as React from "react";
import { Slot } from "@radix-ui/react-slot@1.1.2";
import { ChevronRight, MoreHorizontal } from "lucide-react@0.487.0";

import { cn } from "./utils";

function Breadcrumb({ ...props }: React.ComponentProps&lt;"nav"&gt;) {
  return &lt;nav aria-label="breadcrumb" data-slot="breadcrumb" {...props} /&gt;;
}

function BreadcrumbList({ className, ...props }: React.ComponentProps&lt;"ol"&gt;) {
  return (
    &lt;ol
      data-slot="breadcrumb-list"
      className={cn(
        "text-muted-foreground flex flex-wrap items-center gap-1.5 text-sm break-words
     sm:gap-2.5",
        className,
      )}
      {...props}
    /&gt;
  );
}

function BreadcrumbItem({ className, ...props }: React.ComponentProps&lt;"li"&gt;) {
  return (
    &lt;li
      data-slot="breadcrumb-item"
      className={cn("inline-flex items-center gap-1.5", className)}
      {...props}
    /&gt;
  );
}

function BreadcrumbLink({
  asChild,
  className,
  ...props
}: React.ComponentProps&lt;"a"&gt; &amp; {
  asChild?: boolean;
}) {
  const Comp = asChild ? Slot : "a";

  return (
    &lt;Comp
      data-slot="breadcrumb-link"
      className={cn("hover:text-foreground transition-colors", className)}
      {...props}
    /&gt;
  );
}

function BreadcrumbPage({ className, ...props }: React.ComponentProps&lt;"span"&gt;) {
  return (
    &lt;span
      data-slot="breadcrumb-page"
      role="link"
      aria-disabled="true"
      aria-current="page"
      className={cn("text-foreground font-normal", className)}
      {...props}
    /&gt;
  );
}

function BreadcrumbSeparator({
  children,
  className,
  ...props
}: React.ComponentProps&lt;"li"&gt;) {
  return (
    &lt;li
      data-slot="breadcrumb-separator"
      role="presentation"
      aria-hidden="true"
      className={cn("[&amp;&gt;svg]:size-3.5", className)}
      {...props}
    &gt;
      {children ?? &lt;ChevronRight /&gt;}
```

```
    &lt;/li&gt;
  );
}

function BreadcrumbEllipsis({
  className,
  ...props
}: React.ComponentProps&lt;"span"&gt;) {
  return (
    &lt;span
      data-slot="breadcrumb-ellipsis"
      role="presentation"
      aria-hidden="true"
      className={cn("flex size-9 items-center justify-center", className)}
      {...props}
    &gt;
      &lt;MoreHorizontal className="size-4" /&gt;
      &lt;span className="sr-only"&gt;More&lt;/span&gt;
    &lt;/span&gt;
  );
}

export {
  Breadcrumb,
  BreadcrumbList,
  BreadcrumbItem,
  BreadcrumbLink,
  BreadcrumbPage,
  BreadcrumbSeparator,
  BreadcrumbEllipsis,
};
```

■ File: src\components\ui\button.tsx

```
import * as React from "react";
import { Slot } from "@radix-ui/react-slot@1.1.2";
import { cva, type VariantProps } from "class-variance-authority@0.7.1";

import { cn } from "./utils";

const buttonVariants = cva(
  "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm
     font-medium transition-all disabled:pointer-events-none disabled:opacity-50
     [&amp;_svg]:pointer-events-none [&amp;_svg:not([class*='size-'])]:size-4 shrink-0
     [&amp;_svg]:shrink-0 outline-none focus-visible:border-ring focus-visible:ring-ring/50
     focus-visible:ring-[3px] aria-invalid:ring-destructive/20
     dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructive",
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground hover:bg-primary/90",
        destructive:
          "bg-destructive text-white hover:bg-destructive/90
    focus-visible:ring-destructive/20 dark:focus-visible:ring-destructive/40
    dark:bg-destructive/60",
        outline:
          "border bg-background text-foreground hover:bg-accent
    hover:text-accent-foreground dark:bg-input/30 dark:border-input
    dark:hover:bg-input/50",
        secondary:
          "bg-secondary text-secondary-foreground hover:bg-secondary/80",
        ghost:
          "hover:bg-accent hover:text-accent-foreground dark:hover:bg-accent/50",
        link: "text-primary underline-offset-4 hover:underline",
      },
      size: {
        default: "h-9 px-4 py-2 has-[&gt;svg]:px-3",
        sm: "h-8 rounded-md gap-1.5 px-3 has-[&gt;svg]:px-2.5",
        lg: "h-10 rounded-md px-6 has-[&gt;svg]:px-4",
        icon: "size-9 rounded-md",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
);

function Button({
  className,
  variant,
  size,
  asChild = false,
  ...props
}: React.ComponentProps&lt;"button"&gt; &amp;
  VariantProps&lt;typeof buttonVariants&gt; &amp; {
    asChild?: boolean;
  }) {
  const Comp = asChild ? Slot : "button";

  return (
    &lt;Comp
      data-slot="button"
      className={cn(buttonVariants({ variant, size, className }))}
      {...props}
    /&gt;
  );
}

export { Button, buttonVariants };
```

■ File: src\components\ui\calendar.tsx

```
"use client";

import * as React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react@0.487.0";
import { DayPicker } from "react-day-picker@8.10.1";

import { cn } from "./utils";
import { buttonVariants } from "./button";

function Calendar({
  className,
  classNames,
  showOutsideDays = true,
  ...props
}: React.ComponentProps&lt;typeof DayPicker&gt;) {
  return (
    &lt;DayPicker
      showOutsideDays={showOutsideDays}
      className={cn("p-3", className)}
      classNames={{
        months: "flex flex-col sm:flex-row gap-2",
        month: "flex flex-col gap-4",
        caption: "flex justify-center pt-1 relative items-center w-full",
        caption_label: "text-sm font-medium",
        nav: "flex items-center gap-1",
        nav_button: cn(
          buttonVariants({ variant: "outline" }),
          "size-7 bg-transparent p-0 opacity-50 hover:opacity-100",
        ),
        nav_button_previous: "absolute left-1",
        nav_button_next: "absolute right-1",
        table: "w-full border-collapse space-x-1",
        head_row: "flex",
        head_cell:
          "text-muted-foreground rounded-md w-8 font-normal text-[0.8rem]",
        row: "flex w-full mt-2",
        cell: cn(
          "relative p-0 text-center text-sm focus-within:relative focus-within:z-20
     [&amp;:has([aria-selected])]:bg-accent [&amp;:has([aria-selected].day-range-end)]:rounded-r-m
    d",
          props.mode === "range"
            ? "[&amp;:has(&gt;.day-range-end)]:rounded-r-md
     [&amp;:has(&gt;.day-range-start)]:rounded-l-md first:[&amp;:has([aria-selected])]:rounded-l-md
     last:[&amp;:has([aria-selected])]:rounded-r-md"
            : "[&amp;:has([aria-selected])]:rounded-md",
        ),
        day: cn(
          buttonVariants({ variant: "ghost" }),
          "size-8 p-0 font-normal aria-selected:opacity-100",
        ),
        day_range_start:
          "day-range-start aria-selected:bg-primary
     aria-selected:text-primary-foreground",
        day_range_end:
          "day-range-end aria-selected:bg-primary aria-selected:text-primary-foreground",
        day_selected:
          "bg-primary text-primary-foreground hover:bg-primary
     hover:text-primary-foreground focus:bg-primary focus:text-primary-foreground",
        day_today: "bg-accent text-accent-foreground",
        day_outside:
          "day-outside text-muted-foreground aria-selected:text-muted-foreground",
        day_disabled: "text-muted-foreground opacity-50",
        day_range_middle:
          "aria-selected:bg-accent aria-selected:text-accent-foreground",
        day_hidden: "invisible",
        ...classNames,
      }}
      components={{
        IconLeft: ({ className, ...props }) =&gt; (
          &lt;ChevronLeft className={cn("size-4", className)} {...props} /&gt;
        ),
        IconRight: ({ className, ...props }) =&gt; (
          &lt;ChevronRight className={cn("size-4", className)} {...props} /&gt;
        ),
      }}
      {...props}
    /&gt;
  );
}
```

```
export { Calendar };
```

■ File: src\components\ui\card.tsx

```
import * as React from "react";

import { cn } from "./utils";

function Card({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="card"
      className={cn(
        "bg-card text-card-foreground flex flex-col gap-6 rounded-xl border",
        className,
      )}
      {...props}
    /&gt;
  );
}

function CardHeader({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="card-header"
      className={cn(
        "@container/card-header grid auto-rows-min grid-rows-[auto_auto] items-start
     gap-1.5 px-6 pt-6 has-data-[slot=card-action]:grid-cols-[1fr_auto] [.border-b]:pb-6",
        className,
      )}
      {...props}
    /&gt;
  );
}

function CardTitle({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;h4
      data-slot="card-title"
      className={cn("leading-none", className)}
      {...props}
    /&gt;
  );
}

function CardDescription({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;p
      data-slot="card-description"
      className={cn("text-muted-foreground", className)}
      {...props}
    /&gt;
  );
}

function CardAction({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="card-action"
      className={cn(
        "col-start-2 row-span-2 row-start-1 self-start justify-self-end",
        className,
      )}
      {...props}
    /&gt;
  );
}

function CardContent({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="card-content"
      className={cn("px-6 [&amp;:last-child]:pb-6", className)}
      {...props}
    /&gt;
  );
}

function CardFooter({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="card-footer"
      className={cn("flex items-center px-6 pb-6 [.border-t]:pt-6", className)}
```

```
      {...props}
    /&gt;
  );
}

export {
  Card,
  CardHeader,
  CardFooter,
  CardTitle,
  CardAction,
  CardDescription,
  CardContent,
};
```

■ File: src\components\ui\carousel.tsx

```
"use client";

import * as React from "react";
import useEmblaCarousel, {
  type UseEmblaCarouselType,
} from "embla-carousel-react@8.6.0";
import { ArrowLeft, ArrowRight } from "lucide-react@0.487.0";

import { cn } from "./utils";
import { Button } from "./button";

type CarouselApi = UseEmblaCarouselType[1];
type UseCarouselParameters = Parameters&amp;lt;typeof useEmblaCarousel&amp;gt;;
type CarouselOptions = UseCarouselParameters[0];
type CarouselPlugin = UseCarouselParameters[1];

type CarouselProps = {
  opts?: CarouselOptions;
  plugins?: CarouselPlugin;
  orientation?: "horizontal" | "vertical";
  setApi?: (api: CarouselApi) =&amp;gt; void;
};

type CarouselContextProps = {
  carouselRef: ReturnType&amp;lt;typeof useEmblaCarousel&amp;gt;[0];
  api: ReturnType&amp;lt;typeof useEmblaCarousel&amp;gt;[1];
  scrollPrev: () =&amp;gt; void;
  scrollNext: () =&amp;gt; void;
  canScrollPrev: boolean;
  canScrollNext: boolean;
} &amp;amp; CarouselProps;

const CarouselContext = React.createContext&amp;lt;CarouselContextProps | null&amp;gt;(null);

function useCarousel() {
  const context = React.useContext(CarouselContext);

  if (!context) {
    throw new Error("useCarousel must be used within a &amp;lt;Carousel /&amp;gt;");
  }

  return context;
}

function Carousel({
  orientation = "horizontal",
  opts,
  setApi,
  plugins,
  className,
  children,
  ...props
}: React.ComponentProps&amp;lt;"div"&amp;gt; &amp;amp; CarouselProps) {
  const [carouselRef, api] = useEmblaCarousel(
    {
      ...opts,
      axis: orientation === "horizontal" ? "x" : "y",
    },
    plugins,
  );
  const [canScrollPrev, setCanScrollPrev] = React.useState(false);
  const [canScrollNext, setCanScrollNext] = React.useState(false);

  const onSelect = React.useCallback((api: CarouselApi) =&amp;gt; {
    if (!api) return;
    setCanScrollPrev(api.canScrollPrev());
    setCanScrollNext(api.canScrollNext());
  }, []);

  const scrollPrev = React.useCallback(() =&amp;gt; {
    api?.scrollPrev();
  }, [api]);

  const scrollNext = React.useCallback(() =&amp;gt; {
    api?.scrollNext();
  }, [api]);

  const handleKeyDown = React.useCallback(
    (event: React.KeyboardEvent&amp;lt;HTMLDivElement&amp;gt;) =&amp;gt; {
```

```
      if (event.key === "ArrowLeft") {
        event.preventDefault();
        scrollPrev();
      } else if (event.key === "ArrowRight") {
        event.preventDefault();
        scrollNext();
      }
    },
    [scrollPrev, scrollNext],
  );

  React.useEffect(() =&gt; {
    if (!api || !setApi) return;
    setApi(api);
  }, [api, setApi]);

  React.useEffect(() =&gt; {
    if (!api) return;
    onSelect(api);
    api.on("reInit", onSelect);
    api.on("select", onSelect);

    return () =&gt; {
      api?.off("select", onSelect);
    };
  }, [api, onSelect]);

  return (
    &lt;CarouselContext.Provider
      value={{
        carouselRef,
        api: api,
        opts,
        orientation:
          orientation || (opts?.axis === "y" ? "vertical" : "horizontal"),
        scrollPrev,
        scrollNext,
        canScrollPrev,
        canScrollNext,
      }}
    &gt;
      &lt;div
        onKeyDownCapture={handleKeyDown}
        className={cn("relative", className)}
        role="region"
        aria-roledescription="carousel"
        data-slot="carousel"
        {...props}
      &gt;
        {children}
      &lt;/div&gt;
    &lt;/CarouselContext.Provider&gt;
  );
}

function CarouselContent({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  const { carouselRef, orientation } = useCarousel();

  return (
    &lt;div
      ref={carouselRef}
      className="overflow-hidden"
      data-slot="carousel-content"
    &gt;
      &lt;div
        className={cn(
          "flex",
          orientation === "horizontal" ? "-ml-4" : "-mt-4 flex-col",
          className,
        )}
        {...props}
      /&gt;
    &lt;/div&gt;
  );
}

function CarouselItem({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  const { orientation } = useCarousel();

  return (
    &lt;div
      role="group"
```

```
      aria-roledescription="slide"
      data-slot="carousel-item"
      className={cn(
        "min-w-0 shrink-0 grow-0 basis-full",
        orientation === "horizontal" ? "pl-4" : "pt-4",
        className,
      )}
      {...props}
    /&gt;
  );
}

function CarouselPrevious({
  className,
  variant = "outline",
  size = "icon",
  ...props
}: React.ComponentProps&lt;typeof Button&gt;) {
  const { orientation, scrollPrev, canScrollPrev } = useCarousel();

  return (
    &lt;Button
      data-slot="carousel-previous"
      variant={variant}
      size={size}
      className={cn(
        "absolute size-8 rounded-full",
        orientation === "horizontal"
          ? "top-1/2 -left-12 -translate-y-1/2"
          : "-top-12 left-1/2 -translate-x-1/2 rotate-90",
        className,
      )}
      disabled={!canScrollPrev}
      onClick={scrollPrev}
      {...props}
    &gt;
      &lt;ArrowLeft /&gt;
      &lt;span className="sr-only"&gt;Previous slide&lt;/span&gt;
    &lt;/Button&gt;
  );
}

function CarouselNext({
  className,
  variant = "outline",
  size = "icon",
  ...props
}: React.ComponentProps&lt;typeof Button&gt;) {
  const { orientation, scrollNext, canScrollNext } = useCarousel();

  return (
    &lt;Button
      data-slot="carousel-next"
      variant={variant}
      size={size}
      className={cn(
        "absolute size-8 rounded-full",
        orientation === "horizontal"
          ? "top-1/2 -right-12 -translate-y-1/2"
          : "-bottom-12 left-1/2 -translate-x-1/2 rotate-90",
        className,
      )}
      disabled={!canScrollNext}
      onClick={scrollNext}
      {...props}
    &gt;
      &lt;ArrowRight /&gt;
      &lt;span className="sr-only"&gt;Next slide&lt;/span&gt;
    &lt;/Button&gt;
  );
}

export {
  type CarouselApi,
  Carousel,
  CarouselContent,
  CarouselItem,
  CarouselPrevious,
  CarouselNext,
};
```

■ File: src\components\ui\chart.tsx

```
"use client";

import * as React from "react";
import * as RechartsPrimitive from "recharts@2.15.2";

import { cn } from "./utils";

// Format: { THEME_NAME: CSS_SELECTOR }
const THEMES = { light: "", dark: ".dark" } as const;

export type ChartConfig = {
  [k in string]: {
    label?: React.ReactNode;
    icon?: React.ComponentType;
  } &amp; (
    | { color?: string; theme?: never }
    | { color?: never; theme: Record&lt;keyof typeof THEMES, string&gt; }
  );
};

type ChartContextProps = {
  config: ChartConfig;
};

const ChartContext = React.createContext&lt;ChartContextProps | null&gt;(null);

function useChart() {
  const context = React.useContext(ChartContext);

  if (!context) {
    throw new Error("useChart must be used within a &lt;ChartContainer /&gt;");
  }

  return context;
}

function ChartContainer({
  id,
  className,
  children,
  config,
  ...props
}: React.ComponentProps&lt;"div"&gt; &amp; {
  config: ChartConfig;
  children: React.ComponentProps&lt;
    typeof RechartsPrimitive.ResponsiveContainer
  &gt;["children"];
}) {
  const uniqueId = React.useId();
  const chartId = `chart-${id || uniqueId.replace(/:/g, "")}`;

  return (
    &lt;ChartContext.Provider value={{ config }}&gt;
      &lt;div
        data-slot="chart"
        data-chart={chartId}
        className={cn(
          "[&amp;_.recharts-cartesian-axis-tick_text]:fill-muted-foreground
     [&amp;_.recharts-cartesian-grid_line[stroke='#ccc']]:stroke-border/50
     [&amp;_.recharts-curve.recharts-tooltip-cursor]:stroke-border
     [&amp;_.recharts-polar-grid_[stroke='#ccc']]:stroke-border
     [&amp;_.recharts-radial-bar-background-sector]:fill-muted
     [&amp;_.recharts-rectangle.recharts-tooltip-cursor]:fill-muted
     [&amp;_.recharts-reference-line_[stroke='#ccc']]:stroke-border flex aspect-video
     justify-center text-xs [&amp;_.recharts-dot[stroke='#fff']]:stroke-transparent
     [&amp;_.recharts-layer]:outline-hidden [&amp;_.recharts-sector]:outline-hidden
     [&amp;_.recharts-sector[stroke='#fff']]:stroke-transparent
     [&amp;_.recharts-surface]:outline-hidden",
          className,
        )}
        {...props}
      &gt;
        &lt;ChartStyle id={chartId} config={config} /&gt;
        &lt;RechartsPrimitive.ResponsiveContainer&gt;
          {children}
        &lt;/RechartsPrimitive.ResponsiveContainer&gt;
      &lt;/div&gt;
    &lt;/ChartContext.Provider&gt;
  );
```

```
}

const ChartStyle = ({ id, config }: { id: string; config: ChartConfig }) =&gt; {
  const colorConfig = Object.entries(config).filter(
    ([, config]) =&gt; config.theme || config.color,
  );

  if (!colorConfig.length) {
    return null;
  }

  return (
    &lt;style
      dangerouslySetInnerHTML={{
        __html: Object.entries(THEMES)
          .map(
            ([theme, prefix]) =&gt; `
${prefix} [data-chart=${id}] {
${colorConfig
  .map(([key, itemConfig]) =&gt; {
    const color =
      itemConfig.theme?.[theme as keyof typeof itemConfig.theme] ||
      itemConfig.color;
    return color ? `  --color-${key}: ${color};` : null;
  })
  .join("\n")}
}
`,
          )
          .join("\n"),
      }}
    /&gt;
  );
};

const ChartTooltip = RechartsPrimitive.Tooltip;

function ChartTooltipContent({
  active,
  payload,
  className,
  indicator = "dot",
  hideLabel = false,
  hideIndicator = false,
  label,
  labelFormatter,
  labelClassName,
  formatter,
  color,
  nameKey,
  labelKey,
}: React.ComponentProps&lt;typeof RechartsPrimitive.Tooltip&gt; &amp;
  React.ComponentProps&lt;"div"&gt; &amp; {
    hideLabel?: boolean;
    hideIndicator?: boolean;
    indicator?: "line" | "dot" | "dashed";
    nameKey?: string;
    labelKey?: string;
  }) {
  const { config } = useChart();

  const tooltipLabel = React.useMemo(() =&gt; {
    if (hideLabel || !payload?.length) {
      return null;
    }

    const [item] = payload;
    const key = `${labelKey || item?.dataKey || item?.name || "value"}`;
    const itemConfig = getPayloadConfigFromPayload(config, item, key);
    const value =
      !labelKey &amp;&amp; typeof label === "string"
        ? config[label as keyof typeof config]?.label || label
        : itemConfig?.label;

    if (labelFormatter) {
      return (
        &lt;div className={cn("font-medium", labelClassName)}&gt;
          {labelFormatter(value, payload)}
        &lt;/div&gt;
      );
    }
```

```
  if (!value) {
    return null;
  }

  return &lt;div className={cn("font-medium", labelClassName)}&gt;{value}&lt;/div&gt;;
}, [
  label,
  labelFormatter,
  payload,
  hideLabel,
  labelClassName,
  config,
  labelKey,
]);

if (!active || !payload?.length) {
  return null;
}

const nestLabel = payload.length === 1 &amp;&amp; indicator !== "dot";

return (
  &lt;div
    className={cn(
      "border-border/50 bg-background grid min-w-[8rem] items-start gap-1.5 rounded-lg
   border px-2.5 py-1.5 text-xs shadow-xl",
      className,
    )}
  &gt;
    {!nestLabel ? tooltipLabel : null}
    &lt;div className="grid gap-1.5"&gt;
      {payload.map((item, index) =&gt; {
        const key = `${nameKey || item.name || item.dataKey || "value"}`;
        const itemConfig = getPayloadConfigFromPayload(config, item, key);
        const indicatorColor = color || item.payload.fill || item.color;

        return (
          &lt;div
            key={item.dataKey}
            className={cn(
              "[&amp;&gt;svg]:text-muted-foreground flex w-full flex-wrap items-stretch gap-2
   [&amp;&gt;svg]:h-2.5 [&amp;&gt;svg]:w-2.5",
              indicator === "dot" &amp;&amp; "items-center",
            )}
          &gt;
            {formatter &amp;&amp; item?.value !== undefined &amp;&amp; item.name ? (
              formatter(item.value, item.name, item, index, item.payload)
            ) : (
              &lt;&gt;
                {itemConfig?.icon ? (
                  &lt;itemConfig.icon /&gt;
                ) : (
                  !hideIndicator &amp;&amp; (
                    &lt;div
                      className={cn(
                        "shrink-0 rounded-[2px] border-(--color-border)
   bg-(--color-bg)",
                        {
                          "h-2.5 w-2.5": indicator === "dot",
                          "w-1": indicator === "line",
                          "w-0 border-[1.5px] border-dashed bg-transparent":
                            indicator === "dashed",
                          "my-0.5": nestLabel &amp;&amp; indicator === "dashed",
                        },
                      )}
                      style={
                        {
                          "--color-bg": indicatorColor,
                          "--color-border": indicatorColor,
                        } as React.CSSProperties
                      }
                    /&gt;
                  )
                )}
                &lt;div
                  className={cn(
                    "flex flex-1 justify-between leading-none",
                    nestLabel ? "items-end" : "items-center",
                  )}
                &gt;
                  &lt;div className="grid gap-1.5"&gt;
                    {nestLabel ? tooltipLabel : null}
```

```
                    &lt;span className="text-muted-foreground"&gt;
                      {itemConfig?.label || item.name}
                    &lt;/span&gt;
                  &lt;/div&gt;
                  {item.value &amp;&amp; (
                    &lt;span className="text-foreground font-mono font-medium
    tabular-nums"&gt;
                      {item.value.toLocaleString()}
                    &lt;/span&gt;
                  )}
                &lt;/div&gt;
              &lt;/&gt;
            )}
          &lt;/div&gt;
        );
      })}
    &lt;/div&gt;
  &lt;/div&gt;
 );
}

const ChartLegend = RechartsPrimitive.Legend;

function ChartLegendContent({
 className,
 hideIcon = false,
 payload,
 verticalAlign = "bottom",
 nameKey,
}: React.ComponentProps&lt;"div"&gt; &amp;
 Pick&lt;RechartsPrimitive.LegendProps, "payload" | "verticalAlign"&gt; &amp; {
   hideIcon?: boolean;
   nameKey?: string;
 }) {
 const { config } = useChart();

 if (!payload?.length) {
   return null;
 }

 return (
   &lt;div
     className={cn(
       "flex items-center justify-center gap-4",
       verticalAlign === "top" ? "pb-3" : "pt-3",
       className,
     )}
   &gt;
     {payload.map((item) =&gt; {
       const key = `${nameKey || item.dataKey || "value"}`;
       const itemConfig = getPayloadConfigFromPayload(config, item, key);

       return (
         &lt;div
           key={item.value}
           className={cn(
             "[&amp;&gt;svg]:text-muted-foreground flex items-center gap-1.5 [&amp;&gt;svg]:h-3
   [&amp;&gt;svg]:w-3",
           )}
         &gt;
           {itemConfig?.icon &amp;&amp; !hideIcon ? (
             &lt;itemConfig.icon /&gt;
           ) : (
             &lt;div
               className="h-2 w-2 shrink-0 rounded-[2px]"
               style={{
                 backgroundColor: item.color,
               }}
             /&gt;
           )}
           {itemConfig?.label}
         &lt;/div&gt;
       );
     })}
   &lt;/div&gt;
 );
}

// Helper to extract item config from a payload.
function getPayloadConfigFromPayload(
 config: ChartConfig,
 payload: unknown,
```

```
  key: string,
) {
  if (typeof payload !== "object" || payload === null) {
    return undefined;
  }

  const payloadPayload =
    "payload" in payload &amp;&amp;
    typeof payload.payload === "object" &amp;&amp;
    payload.payload !== null
      ? payload.payload
      : undefined;

  let configLabelKey: string = key;

  if (
    key in payload &amp;&amp;
    typeof payload[key as keyof typeof payload] === "string"
  ) {
    configLabelKey = payload[key as keyof typeof payload] as string;
  } else if (
    payloadPayload &amp;&amp;
    key in payloadPayload &amp;&amp;
    typeof payloadPayload[key as keyof typeof payloadPayload] === "string"
  ) {
    configLabelKey = payloadPayload[
      key as keyof typeof payloadPayload
    ] as string;
  }

  return configLabelKey in config
    ? config[configLabelKey]
    : config[key as keyof typeof config];
}

export {
  ChartContainer,
  ChartTooltip,
  ChartTooltipContent,
  ChartLegend,
  ChartLegendContent,
  ChartStyle,
};
```

■ File: src\components\ui\checkbox.tsx

```
"use client";

import * as React from "react";
import * as CheckboxPrimitive from "@radix-ui/react-checkbox@1.1.4";
import { CheckIcon } from "lucide-react@0.487.0";

import { cn } from "./utils";

function Checkbox({
  className,
  ...props
}: React.ComponentProps&lt;typeof CheckboxPrimitive.Root&gt;) {
  return (
    &lt;CheckboxPrimitive.Root
      data-slot="checkbox"
      className={cn(
        "peer border bg-input-background dark:bg-input/30 data-[state=checked]:bg-primary
     data-[state=checked]:text-primary-foreground dark:data-[state=checked]:bg-primary
     data-[state=checked]:border-primary focus-visible:border-ring
     focus-visible:ring-ring/50 aria-invalid:ring-destructive/20
     dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructive size-4
     shrink-0 rounded-[4px] border shadow-xs transition-shadow outline-none
     focus-visible:ring-[3px] disabled:cursor-not-allowed disabled:opacity-50",
        className,
      )}
      {...props}
    &gt;
      &lt;CheckboxPrimitive.Indicator
        data-slot="checkbox-indicator"
        className="flex items-center justify-center text-current transition-none"
      &gt;
        &lt;CheckIcon className="size-3.5" /&gt;
      &lt;/CheckboxPrimitive.Indicator&gt;
    &lt;/CheckboxPrimitive.Root&gt;
  );
}

export { Checkbox };
```

■ File: src\components\ui\collapsible.tsx

```
"use client";

import * as CollapsiblePrimitive from "@radix-ui/react-collapsible@1.1.3";

function Collapsible({
  ...props
}: React.ComponentProps&lt;typeof CollapsiblePrimitive.Root&gt;) {
  return &lt;CollapsiblePrimitive.Root data-slot="collapsible" {...props} /&gt;;
}

function CollapsibleTrigger({
  ...props
}: React.ComponentProps&lt;typeof CollapsiblePrimitive.CollapsibleTrigger&gt;) {
  return (
    &lt;CollapsiblePrimitive.CollapsibleTrigger
      data-slot="collapsible-trigger"
      {...props}
    /&gt;
  );
}

function CollapsibleContent({
  ...props
}: React.ComponentProps&lt;typeof CollapsiblePrimitive.CollapsibleContent&gt;) {
  return (
    &lt;CollapsiblePrimitive.CollapsibleContent
      data-slot="collapsible-content"
      {...props}
    /&gt;
  );
}

export { Collapsible, CollapsibleTrigger, CollapsibleContent };
```

■ File: src\components\ui\command.tsx

```
"use client";

import * as React from "react";
import { Command as CommandPrimitive } from "cmdk@1.1.1";
import { SearchIcon } from "lucide-react@0.487.0";

import { cn } from "./utils";
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogHeader,
  DialogTitle,
} from "./dialog";

function Command({
  className,
  ...props
}: React.ComponentProps&lt;typeof CommandPrimitive&gt;) {
  return (
    &lt;CommandPrimitive
      data-slot="command"
      className={cn(
        "bg-popover text-popover-foreground flex h-full w-full flex-col overflow-hidden
     rounded-md",
        className,
      )}
      {...props}
    /&gt;
  );
}

function CommandDialog({
  title = "Command Palette",
  description = "Search for a command to run...",
  children,
  ...props
}: React.ComponentProps&lt;typeof Dialog&gt; &amp; {
  title?: string;
  description?: string;
}) {
  return (
    &lt;Dialog {...props}&gt;
      &lt;DialogHeader className="sr-only"&gt;
        &lt;DialogTitle&gt;{title}&lt;/DialogTitle&gt;
        &lt;DialogDescription&gt;{description}&lt;/DialogDescription&gt;
      &lt;/DialogHeader&gt;
      &lt;DialogContent className="overflow-hidden p-0"&gt;
        &lt;Command className="[&amp;_[cmdk-group-heading]]:text-muted-foreground
     **:data-[slot=command-input-wrapper]:h-12 [&amp;_[cmdk-group-heading]]:px-2
     [&amp;_[cmdk-group-heading]]:font-medium [&amp;_[cmdk-group]]:px-2
     [&amp;_[cmdk-group]:not([hidden])_~[cmdk-group]]:pt-0 [&amp;_[cmdk-input-wrapper]_svg]:h-5
     [&amp;_[cmdk-input-wrapper]_svg]:w-5 [&amp;_[cmdk-input]]:h-12 [&amp;_[cmdk-item]]:px-2
     [&amp;_[cmdk-item]]:py-3 [&amp;_[cmdk-item]_svg]:h-5 [&amp;_[cmdk-item]_svg]:w-5"&gt;
          {children}
        &lt;/Command&gt;
      &lt;/DialogContent&gt;
    &lt;/Dialog&gt;
  );
}

function CommandInput({
  className,
  ...props
}: React.ComponentProps&lt;typeof CommandPrimitive.Input&gt;) {
  return (
    &lt;div
      data-slot="command-input-wrapper"
      className="flex h-9 items-center gap-2 border-b px-3"
    &gt;
      &lt;SearchIcon className="size-4 shrink-0 opacity-50" /&gt;
      &lt;CommandPrimitive.Input
        data-slot="command-input"
        className={cn(
          "placeholder:text-muted-foreground flex h-10 w-full rounded-md bg-transparent
     py-3 text-sm outline-hidden disabled:cursor-not-allowed disabled:opacity-50",
          className,
        )}
        {...props}
```

```
      /&gt;
    &lt;/div&gt;
  );
}

function CommandList({
  className,
  ...props
}: React.ComponentProps&lt;typeof CommandPrimitive.List&gt;) {
  return (
    &lt;CommandPrimitive.List
      data-slot="command-list"
      className={cn(
        "max-h-[300px] scroll-py-1 overflow-x-hidden overflow-y-auto",
        className,
      )}
      {...props}
    /&gt;
  );
}

function CommandEmpty({
  ...props
}: React.ComponentProps&lt;typeof CommandPrimitive.Empty&gt;) {
  return (
    &lt;CommandPrimitive.Empty
      data-slot="command-empty"
      className="py-6 text-center text-sm"
      {...props}
    /&gt;
  );
}

function CommandGroup({
  className,
  ...props
}: React.ComponentProps&lt;typeof CommandPrimitive.Group&gt;) {
  return (
    &lt;CommandPrimitive.Group
      data-slot="command-group"
      className={cn(
        "text-foreground [&amp;_[cmdk-group-heading]]:text-muted-foreground overflow-hidden
    p-1 [&amp;_[cmdk-group-heading]]:px-2 [&amp;_[cmdk-group-heading]]:py-1.5
    [&amp;_[cmdk-group-heading]]:text-xs [&amp;_[cmdk-group-heading]]:font-medium",
        className,
      )}
      {...props}
    /&gt;
  );
}

function CommandSeparator({
  className,
  ...props
}: React.ComponentProps&lt;typeof CommandPrimitive.Separator&gt;) {
  return (
    &lt;CommandPrimitive.Separator
      data-slot="command-separator"
      className={cn("bg-border -mx-1 h-px", className)}
      {...props}
    /&gt;
  );
}

function CommandItem({
  className,
  ...props
}: React.ComponentProps&lt;typeof CommandPrimitive.Item&gt;) {
  return (
    &lt;CommandPrimitive.Item
      data-slot="command-item"
      className={cn(
        "data-[selected=true]:bg-accent data-[selected=true]:text-accent-foreground
    [&amp;_svg:not([class*='text-'])]:text-muted-foreground relative flex cursor-default
    items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-hidden select-none
    data-[disabled=true]:pointer-events-none data-[disabled=true]:opacity-50
    [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0 [&amp;_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    /&gt;
  );
```

```
}

function CommandShortcut({
  className,
  ...props
}: React.ComponentProps&lt;"span"&gt;) {
  return (
    &lt;span
      data-slot="command-shortcut"
      className={cn(
        "text-muted-foreground ml-auto text-xs tracking-widest",
        className,
      )}
      {...props}
    /&gt;
  );
}

export {
  Command,
  CommandDialog,
  CommandInput,
  CommandList,
  CommandEmpty,
  CommandGroup,
  CommandItem,
  CommandShortcut,
  CommandSeparator,
};
```

■ File: src\components\ui\context-menu.tsx

```
"use client";

import * as React from "react";
import * as ContextMenuPrimitive from "@radix-ui/react-context-menu@2.2.6";
import { CheckIcon, ChevronRightIcon, CircleIcon } from "lucide-react@0.487.0";

import { cn } from "./utils";

function ContextMenu({
  ...props
}: React.ComponentProps&lt;typeof ContextMenuPrimitive.Root&gt;) {
  return &lt;ContextMenuPrimitive.Root data-slot="context-menu" {...props} /&gt;;
}

function ContextMenuTrigger({
  ...props
}: React.ComponentProps&lt;typeof ContextMenuPrimitive.Trigger&gt;) {
  return (
    &lt;ContextMenuPrimitive.Trigger data-slot="context-menu-trigger" {...props} /&gt;
  );
}

function ContextMenuGroup({
  ...props
}: React.ComponentProps&lt;typeof ContextMenuPrimitive.Group&gt;) {
  return (
    &lt;ContextMenuPrimitive.Group data-slot="context-menu-group" {...props} /&gt;
  );
}

function ContextMenuPortal({
  ...props
}: React.ComponentProps&lt;typeof ContextMenuPrimitive.Portal&gt;) {
  return (
    &lt;ContextMenuPrimitive.Portal data-slot="context-menu-portal" {...props} /&gt;
  );
}

function ContextMenuSub({
  ...props
}: React.ComponentProps&lt;typeof ContextMenuPrimitive.Sub&gt;) {
  return &lt;ContextMenuPrimitive.Sub data-slot="context-menu-sub" {...props} /&gt;;
}

function ContextMenuRadioGroup({
  ...props
}: React.ComponentProps&lt;typeof ContextMenuPrimitive.RadioGroup&gt;) {
  return (
    &lt;ContextMenuPrimitive.RadioGroup
      data-slot="context-menu-radio-group"
      {...props}
    /&gt;
  );
}

function ContextMenuSubTrigger({
  className,
  inset,
  children,
  ...props
}: React.ComponentProps&lt;typeof ContextMenuPrimitive.SubTrigger&gt; &amp; {
  inset?: boolean;
}) {
  return (
    &lt;ContextMenuPrimitive.SubTrigger
      data-slot="context-menu-sub-trigger"
      data-inset={inset}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent
     data-[state=open]:text-accent-foreground flex cursor-default items-center rounded-sm
     px-2 py-1.5 text-sm outline-hidden select-none data-[inset]:pl-8
     [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0 [&amp;_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    &gt;
      {children}
      &lt;ChevronRightIcon className="ml-auto" /&gt;
    &lt;/ContextMenuPrimitive.SubTrigger&gt;
```

```
  );
}

function ContextMenuSubContent({
  className,
  ...props
}: React.ComponentProps&lt;typeof ContextMenuPrimitive.SubContent&gt;) {
  return (
    &lt;ContextMenuPrimitive.SubContent
      data-slot="context-menu-sub-content"
      className={cn(
        "bg-popover text-popover-foreground data-[state=open]:animate-in
     data-[state=closed]:animate-out data-[state=closed]:fade-out-0
     data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
     data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
     data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
     data-[side=top]:slide-in-from-bottom-2 z-50 min-w-[8rem]
     origin-(--radix-context-menu-content-transform-origin) overflow-hidden rounded-md
     border p-1 shadow-lg",
        className,
      )}
      {...props}
    /&gt;
  );
}

function ContextMenuContent({
  className,
  ...props
}: React.ComponentProps&lt;typeof ContextMenuPrimitive.Content&gt;) {
  return (
    &lt;ContextMenuPrimitive.Portal&gt;
      &lt;ContextMenuPrimitive.Content
        data-slot="context-menu-content"
        className={cn(
          "bg-popover text-popover-foreground data-[state=open]:animate-in
     data-[state=closed]:animate-out data-[state=closed]:fade-out-0
     data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
     data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
     data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
     data-[side=top]:slide-in-from-bottom-2 z-50
     max-h-(--radix-context-menu-content-available-height) min-w-[8rem]
     origin-(--radix-context-menu-content-transform-origin) overflow-x-hidden
     overflow-y-auto rounded-md border p-1 shadow-md",
          className,
        )}
        {...props}
      /&gt;
    &lt;/ContextMenuPrimitive.Portal&gt;
  );
}

function ContextMenuItem({
  className,
  inset,
  variant = "default",
  ...props
}: React.ComponentProps&lt;typeof ContextMenuPrimitive.Item&gt; &amp;amp; {
  inset?: boolean;
  variant?: "default" | "destructive";
}) {
  return (
    &lt;ContextMenuPrimitive.Item
      data-slot="context-menu-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground
     data-[variant=destructive]:text-destructive
     data-[variant=destructive]:focus:bg-destructive/10
     dark:data-[variant=destructive]:focus:bg-destructive/20
     data-[variant=destructive]:focus:text-destructive
     data-[variant=destructive]:*:[svg]:!text-destructive
     [&amp;_svg:not([class*='text-'])]:text-muted-foreground relative flex cursor-default
     items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-hidden select-none
     data-[disabled]:pointer-events-none data-[disabled]:opacity-50 data-[inset]:pl-8
     [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0 [&amp;_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    /&gt;
  );
```

```
}

function ContextMenuCheckboxItem({
  className,
  children,
  checked,
  ...props
}: React.ComponentProps&lt;typeof ContextMenuPrimitive.CheckboxItem&gt;) {
  return (
    &lt;ContextMenuPrimitive.CheckboxItem
      data-slot="context-menu-checkbox-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default
     items-center gap-2 rounded-sm py-1.5 pr-2 pl-8 text-sm outline-hidden select-none
     data-[disabled]:pointer-events-none data-[disabled]:opacity-50
     [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0 [&amp;_svg:not([class*='size-'])]:size-4",
        className,
      )}
      checked={checked}
      {...props}
    &gt;
      &lt;span className="pointer-events-none absolute left-2 flex size-3.5 items-center
     justify-center"&gt;
        &lt;ContextMenuPrimitive.ItemIndicator&gt;
          &lt;CheckIcon className="size-4" /&gt;
        &lt;/ContextMenuPrimitive.ItemIndicator&gt;
      &lt;/span&gt;
      {children}
    &lt;/ContextMenuPrimitive.CheckboxItem&gt;
  );
}

function ContextMenuRadioItem({
  className,
  children,
  ...props
}: React.ComponentProps&lt;typeof ContextMenuPrimitive.RadioItem&gt;) {
  return (
    &lt;ContextMenuPrimitive.RadioItem
      data-slot="context-menu-radio-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default
     items-center gap-2 rounded-sm py-1.5 pr-2 pl-8 text-sm outline-hidden select-none
     data-[disabled]:pointer-events-none data-[disabled]:opacity-50
     [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0 [&amp;_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    &gt;
      &lt;span className="pointer-events-none absolute left-2 flex size-3.5 items-center
     justify-center"&gt;
        &lt;ContextMenuPrimitive.ItemIndicator&gt;
          &lt;CircleIcon className="size-2 fill-current" /&gt;
        &lt;/ContextMenuPrimitive.ItemIndicator&gt;
      &lt;/span&gt;
      {children}
    &lt;/ContextMenuPrimitive.RadioItem&gt;
  );
}

function ContextMenuLabel({
  className,
  inset,
  ...props
}: React.ComponentProps&lt;typeof ContextMenuPrimitive.Label&gt; &amp; {
  inset?: boolean;
}) {
  return (
    &lt;ContextMenuPrimitive.Label
      data-slot="context-menu-label"
      data-inset={inset}
      className={cn(
        "text-foreground px-2 py-1.5 text-sm font-medium data-[inset]:pl-8",
        className,
      )}
      {...props}
    /&gt;
  );
}

function ContextMenuSeparator({
  className,
```

```
  ...props
}: React.ComponentProps&lt;typeof ContextMenuPrimitive.Separator&gt;) {
  return (
    &lt;ContextMenuPrimitive.Separator
      data-slot="context-menu-separator"
      className={cn("bg-border -mx-1 my-1 h-px", className)}
      {...props}
    /&gt;
  );
}

function ContextMenuShortcut({
  className,
  ...props
}: React.ComponentProps&lt;"span"&gt;) {
  return (
    &lt;span
      data-slot="context-menu-shortcut"
      className={cn(
        "text-muted-foreground ml-auto text-xs tracking-widest",
        className,
      )}
      {...props}
    /&gt;
  );
}

export {
  ContextMenu,
  ContextMenuTrigger,
  ContextMenuContent,
  ContextMenuItem,
  ContextMenuCheckboxItem,
  ContextMenuRadioItem,
  ContextMenuLabel,
  ContextMenuSeparator,
  ContextMenuShortcut,
  ContextMenuGroup,
  ContextMenuPortal,
  ContextMenuSub,
  ContextMenuSubContent,
  ContextMenuSubTrigger,
  ContextMenuRadioGroup,
};
```

■ File: src\components\ui\dialog.tsx

```
"use client";

import * as React from "react";
import * as DialogPrimitive from "@radix-ui/react-dialog@1.1.6";
import { XIcon } from "lucide-react@0.487.0";

import { cn } from "./utils";

function Dialog({
  ...props
}: React.ComponentProps&lt;typeof DialogPrimitive.Root&gt;) {
  return &lt;DialogPrimitive.Root data-slot="dialog" {...props} /&gt;;
}

function DialogTrigger({
  ...props
}: React.ComponentProps&lt;typeof DialogPrimitive.Trigger&gt;) {
  return &lt;DialogPrimitive.Trigger data-slot="dialog-trigger" {...props} /&gt;;
}

function DialogPortal({
  ...props
}: React.ComponentProps&lt;typeof DialogPrimitive.Portal&gt;) {
  return &lt;DialogPrimitive.Portal data-slot="dialog-portal" {...props} /&gt;;
}

function DialogClose({
  ...props
}: React.ComponentProps&lt;typeof DialogPrimitive.Close&gt;) {
  return &lt;DialogPrimitive.Close data-slot="dialog-close" {...props} /&gt;;
}

function DialogOverlay({
  className,
  ...props
}: React.ComponentProps&lt;typeof DialogPrimitive.Overlay&gt;) {
  return (
    &lt;DialogPrimitive.Overlay
      data-slot="dialog-overlay"
      className={cn(
        "data-[state=open]:animate-in data-[state=closed]:animate-out
     data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 fixed inset-0 z-50
     bg-black/50",
        className,
      )}
      {...props}
    /&gt;
  );
}

function DialogContent({
  className,
  children,
  ...props
}: React.ComponentProps&lt;typeof DialogPrimitive.Content&gt;) {
  return (
    &lt;DialogPortal data-slot="dialog-portal"&gt;
      &lt;DialogOverlay /&gt;
      &lt;DialogPrimitive.Content
        data-slot="dialog-content"
        className={cn(
          "bg-background data-[state=open]:animate-in data-[state=closed]:animate-out
     data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
     data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 fixed top-[50%]
     left-[50%] z-50 grid w-full max-w-[calc(100%-2rem)] translate-x-[-50%]
     translate-y-[-50%] gap-4 rounded-lg border p-6 shadow-lg duration-200 sm:max-w-lg",
          className,
        )}
        {...props}
      &gt;
        {children}
        &lt;DialogPrimitive.Close className="ring-offset-background focus:ring-ring
     data-[state=open]:bg-accent data-[state=open]:text-muted-foreground absolute top-4
     right-4 rounded-xs opacity-70 transition-opacity hover:opacity-100 focus:ring-2
     focus:ring-offset-2 focus:outline-hidden disabled:pointer-events-none
     [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0 [&amp;_svg:not([class*='size-'])]:size-4"&gt;
          &lt;XIcon /&gt;
          &lt;span className="sr-only"&gt;Close&lt;/span&gt;
        &lt;/DialogPrimitive.Close&gt;
```

```
      &lt;/DialogPrimitive.Content&gt;
    &lt;/DialogPortal&gt;
  );
}

function DialogHeader({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="dialog-header"
      className={cn("flex flex-col gap-2 text-center sm:text-left", className)}
      {...props}
    /&gt;
  );
}

function DialogFooter({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="dialog-footer"
      className={cn(
        "flex flex-col-reverse gap-2 sm:flex-row sm:justify-end",
        className,
      )}
      {...props}
    /&gt;
  );
}

function DialogTitle({
  className,
  ...props
}: React.ComponentProps&lt;typeof DialogPrimitive.Title&gt;) {
  return (
    &lt;DialogPrimitive.Title
      data-slot="dialog-title"
      className={cn("text-lg leading-none font-semibold", className)}
      {...props}
    /&gt;
  );
}

function DialogDescription({
  className,
  ...props
}: React.ComponentProps&lt;typeof DialogPrimitive.Description&gt;) {
  return (
    &lt;DialogPrimitive.Description
      data-slot="dialog-description"
      className={cn("text-muted-foreground text-sm", className)}
      {...props}
    /&gt;
  );
}

export {
  Dialog,
  DialogClose,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogOverlay,
  DialogPortal,
  DialogTitle,
  DialogTrigger,
};
```

■ File: src\components\ui\drawer.tsx

```
"use client";

import * as React from "react";
import { Drawer as DrawerPrimitive } from "vaul@1.1.2";

import { cn } from "./utils";

function Drawer({
  ...props
}: React.ComponentProps&lt;typeof DrawerPrimitive.Root&gt;) {
  return &lt;DrawerPrimitive.Root data-slot="drawer" {...props} /&gt;;
}

function DrawerTrigger({
  ...props
}: React.ComponentProps&lt;typeof DrawerPrimitive.Trigger&gt;) {
  return &lt;DrawerPrimitive.Trigger data-slot="drawer-trigger" {...props} /&gt;;
}

function DrawerPortal({
  ...props
}: React.ComponentProps&lt;typeof DrawerPrimitive.Portal&gt;) {
  return &lt;DrawerPrimitive.Portal data-slot="drawer-portal" {...props} /&gt;;
}

function DrawerClose({
  ...props
}: React.ComponentProps&lt;typeof DrawerPrimitive.Close&gt;) {
  return &lt;DrawerPrimitive.Close data-slot="drawer-close" {...props} /&gt;;
}

function DrawerOverlay({
  className,
  ...props
}: React.ComponentProps&lt;typeof DrawerPrimitive.Overlay&gt;) {
  return (
    &lt;DrawerPrimitive.Overlay
      data-slot="drawer-overlay"
      className={cn(
        "data-[state=open]:animate-in data-[state=closed]:animate-out
     data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 fixed inset-0 z-50
     bg-black/50",
        className,
      )}
      {...props}
    /&gt;
  );
}

function DrawerContent({
  className,
  children,
  ...props
}: React.ComponentProps&lt;typeof DrawerPrimitive.Content&gt;) {
  return (
    &lt;DrawerPortal data-slot="drawer-portal"&gt;
      &lt;DrawerOverlay /&gt;
      &lt;DrawerPrimitive.Content
        data-slot="drawer-content"
        className={cn(
          "group/drawer-content bg-background fixed z-50 flex h-auto flex-col",
          "data-[vaul-drawer-direction=top]:inset-x-0
     data-[vaul-drawer-direction=top]:top-0 data-[vaul-drawer-direction=top]:mb-24
     data-[vaul-drawer-direction=top]:max-h-[80vh]
     data-[vaul-drawer-direction=top]:rounded-b-lg
     data-[vaul-drawer-direction=top]:border-b",
          "data-[vaul-drawer-direction=bottom]:inset-x-0
     data-[vaul-drawer-direction=bottom]:bottom-0
     data-[vaul-drawer-direction=bottom]:mt-24
     data-[vaul-drawer-direction=bottom]:max-h-[80vh]
     data-[vaul-drawer-direction=bottom]:rounded-t-lg
     data-[vaul-drawer-direction=bottom]:border-t",
          "data-[vaul-drawer-direction=right]:inset-y-0
     data-[vaul-drawer-direction=right]:right-0 data-[vaul-drawer-direction=right]:w-3/4
     data-[vaul-drawer-direction=right]:border-l
     data-[vaul-drawer-direction=right]:sm:max-w-sm",
          "data-[vaul-drawer-direction=left]:inset-y-0
     data-[vaul-drawer-direction=left]:left-0 data-[vaul-drawer-direction=left]:w-3/4
     data-[vaul-drawer-direction=left]:border-r
```

```
     data-[vaul-drawer-direction=left]:sm:max-w-sm",
          className,
        )}
        {...props}
      &gt;
        &lt;div className="bg-muted mx-auto mt-4 hidden h-2 w-[100px] shrink-0 rounded-full
     group-data-[vaul-drawer-direction=bottom]/drawer-content:block" /&gt;
        {children}
      &lt;/DrawerPrimitive.Content&gt;
    &lt;/DrawerPortal&gt;
  );
}

function DrawerHeader({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="drawer-header"
      className={cn("flex flex-col gap-1.5 p-4", className)}
      {...props}
    /&gt;
  );
}

function DrawerFooter({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="drawer-footer"
      className={cn("mt-auto flex flex-col gap-2 p-4", className)}
      {...props}
    /&gt;
  );
}

function DrawerTitle({
  className,
  ...props
}: React.ComponentProps&lt;typeof DrawerPrimitive.Title&gt;) {
  return (
    &lt;DrawerPrimitive.Title
      data-slot="drawer-title"
      className={cn("text-foreground font-semibold", className)}
      {...props}
    /&gt;
  );
}

function DrawerDescription({
  className,
  ...props
}: React.ComponentProps&lt;typeof DrawerPrimitive.Description&gt;) {
  return (
    &lt;DrawerPrimitive.Description
      data-slot="drawer-description"
      className={cn("text-muted-foreground text-sm", className)}
      {...props}
    /&gt;
  );
}

export {
  Drawer,
  DrawerPortal,
  DrawerOverlay,
  DrawerTrigger,
  DrawerClose,
  DrawerContent,
  DrawerHeader,
  DrawerFooter,
  DrawerTitle,
  DrawerDescription,
};
```

■ File: src\components\ui\dropdown-menu.tsx

```
"use client";

import * as React from "react";
import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu@2.1.6";
import { CheckIcon, ChevronRightIcon, CircleIcon } from "lucide-react@0.487.0";

import { cn } from "./utils";

function DropdownMenu({
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.Root&gt;) {
  return &lt;DropdownMenuPrimitive.Root data-slot="dropdown-menu" {...props} /&gt;;
}

function DropdownMenuPortal({
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.Portal&gt;) {
  return (
    &lt;DropdownMenuPrimitive.Portal data-slot="dropdown-menu-portal" {...props} /&gt;
  );
}

function DropdownMenuTrigger({
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.Trigger&gt;) {
  return (
    &lt;DropdownMenuPrimitive.Trigger
      data-slot="dropdown-menu-trigger"
      {...props}
    /&gt;
  );
}

function DropdownMenuContent({
  className,
  sideOffset = 4,
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.Content&gt;) {
  return (
    &lt;DropdownMenuPrimitive.Portal&gt;
      &lt;DropdownMenuPrimitive.Content
        data-slot="dropdown-menu-content"
        sideOffset={sideOffset}
        className={cn(
          "bg-popover text-popover-foreground data-[state=open]:animate-in
     data-[state=closed]:animate-out data-[state=closed]:fade-out-0
     data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
     data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
     data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
     data-[side=top]:slide-in-from-bottom-2 z-50
     max-h-(--radix-dropdown-menu-content-available-height) min-w-[8rem]
     origin-(--radix-dropdown-menu-content-transform-origin) overflow-x-hidden
     overflow-y-auto rounded-md border p-1 shadow-md",
          className,
        )}
        {...props}
      /&gt;
    &lt;/DropdownMenuPrimitive.Portal&gt;
  );
}

function DropdownMenuGroup({
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.Group&gt;) {
  return (
    &lt;DropdownMenuPrimitive.Group data-slot="dropdown-menu-group" {...props} /&gt;
  );
}

function DropdownMenuItem({
  className,
  inset,
  variant = "default",
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.Item&gt; &amp;amp; {
  inset?: boolean;
  variant?: "default" | "destructive";
}) {
  return (
```

```
    &lt;DropdownMenuPrimitive.Item
      data-slot="dropdown-menu-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground
     data-[variant=destructive]:text-destructive
     data-[variant=destructive]:focus:bg-destructive/10
     dark:data-[variant=destructive]:focus:bg-destructive/20
     data-[variant=destructive]:focus:text-destructive
     data-[variant=destructive]:*:[svg]:!text-destructive
     [&amp;_svg:not([class*='text-'])]:text-muted-foreground relative flex cursor-default
     items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-hidden select-none
     data-[disabled]:pointer-events-none data-[disabled]:opacity-50 data-[inset]:pl-8
     [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0 [&amp;_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    /&gt;
  );
}

function DropdownMenuCheckboxItem({
  className,
  children,
  checked,
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.CheckboxItem&gt;) {
  return (
    &lt;DropdownMenuPrimitive.CheckboxItem
      data-slot="dropdown-menu-checkbox-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default
    items-center gap-2 rounded-sm py-1.5 pr-2 pl-8 text-sm outline-hidden select-none
    data-[disabled]:pointer-events-none data-[disabled]:opacity-50
    [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0 [&amp;_svg:not([class*='size-'])]:size-4",
        className,
      )}
      checked={checked}
      {...props}
    &gt;
      &lt;span className="pointer-events-none absolute left-2 flex size-3.5 items-center
     justify-center"&gt;
        &lt;DropdownMenuPrimitive.ItemIndicator&gt;
          &lt;CheckIcon className="size-4" /&gt;
        &lt;/DropdownMenuPrimitive.ItemIndicator&gt;
      &lt;/span&gt;
      {children}
    &lt;/DropdownMenuPrimitive.CheckboxItem&gt;
  );
}

function DropdownMenuRadioGroup({
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.RadioGroup&gt;) {
  return (
    &lt;DropdownMenuPrimitive.RadioGroup
      data-slot="dropdown-menu-radio-group"
      {...props}
    /&gt;
  );
}

function DropdownMenuRadioItem({
  className,
  children,
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.RadioItem&gt;) {
  return (
    &lt;DropdownMenuPrimitive.RadioItem
      data-slot="dropdown-menu-radio-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default
    items-center gap-2 rounded-sm py-1.5 pr-2 pl-8 text-sm outline-hidden select-none
    data-[disabled]:pointer-events-none data-[disabled]:opacity-50
    [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0 [&amp;_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    &gt;
      &lt;span className="pointer-events-none absolute left-2 flex size-3.5 items-center
     justify-center"&gt;
```

```
        &lt;DropdownMenuPrimitive.ItemIndicator&gt;
          &lt;CircleIcon className="size-2 fill-current" /&gt;
        &lt;/DropdownMenuPrimitive.ItemIndicator&gt;
      &lt;/span&gt;
      {children}
    &lt;/DropdownMenuPrimitive.RadioItem&gt;
  );
}

function DropdownMenuLabel({
  className,
  inset,
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.Label&gt; &amp; {
  inset?: boolean;
}) {
  return (
    &lt;DropdownMenuPrimitive.Label
      data-slot="dropdown-menu-label"
      data-inset={inset}
      className={cn(
        "px-2 py-1.5 text-sm font-medium data-[inset]:pl-8",
        className,
      )}
      {...props}
    /&gt;
  );
}

function DropdownMenuSeparator({
  className,
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.Separator&gt;) {
  return (
    &lt;DropdownMenuPrimitive.Separator
      data-slot="dropdown-menu-separator"
      className={cn("bg-border -mx-1 my-1 h-px", className)}
      {...props}
    /&gt;
  );
}

function DropdownMenuShortcut({
  className,
  ...props
}: React.ComponentProps&lt;"span"&gt;) {
  return (
    &lt;span
      data-slot="dropdown-menu-shortcut"
      className={cn(
        "text-muted-foreground ml-auto text-xs tracking-widest",
        className,
      )}
      {...props}
    /&gt;
  );
}

function DropdownMenuSub({
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.Sub&gt;) {
  return &lt;DropdownMenuPrimitive.Sub data-slot="dropdown-menu-sub" {...props} /&gt;;
}

function DropdownMenuSubTrigger({
  className,
  inset,
  children,
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.SubTrigger&gt; &amp; {
  inset?: boolean;
}) {
  return (
    &lt;DropdownMenuPrimitive.SubTrigger
      data-slot="dropdown-menu-sub-trigger"
      data-inset={inset}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent
     data-[state=open]:text-accent-foreground flex cursor-default items-center rounded-sm
     px-2 py-1.5 text-sm outline-hidden select-none data-[inset]:pl-8",
        className,
      )}
```

```
      {...props}
    &gt;
      {children}
      &lt;ChevronRightIcon className="ml-auto size-4" /&gt;
    &lt;/DropdownMenuPrimitive.SubTrigger&gt;
  );
}

function DropdownMenuSubContent({
  className,
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.SubContent&gt;) {
  return (
    &lt;DropdownMenuPrimitive.SubContent
      data-slot="dropdown-menu-sub-content"
      className={cn(
        "bg-popover text-popover-foreground data-[state=open]:animate-in
     data-[state=closed]:animate-out data-[state=closed]:fade-out-0
     data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
     data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
     data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
     data-[side=top]:slide-in-from-bottom-2 z-50 min-w-[8rem]
     origin-(--radix-dropdown-menu-content-transform-origin) overflow-hidden rounded-md
     border p-1 shadow-lg",
        className,
      )}
      {...props}
    /&gt;
  );
}

export {
  DropdownMenu,
  DropdownMenuPortal,
  DropdownMenuTrigger,
  DropdownMenuContent,
  DropdownMenuGroup,
  DropdownMenuLabel,
  DropdownMenuItem,
  DropdownMenuCheckboxItem,
  DropdownMenuRadioGroup,
  DropdownMenuRadioItem,
  DropdownMenuSeparator,
  DropdownMenuShortcut,
  DropdownMenuSub,
  DropdownMenuSubTrigger,
  DropdownMenuSubContent,
};
```

■ File: src\components\ui\form.tsx

```
"use client";

import * as React from "react";
import * as LabelPrimitive from "@radix-ui/react-label@2.1.2";
import { Slot } from "@radix-ui/react-slot@1.1.2";
import {
  Controller,
  FormProvider,
  useFormContext,
  useFormState,
  type ControllerProps,
  type FieldPath,
  type FieldValues,
} from "react-hook-form@7.55.0";

import { cn } from "./utils";
import { Label } from "./label";

const Form = FormProvider;

type FormFieldContextValue&lt;
  TFieldValues extends FieldValues = FieldValues,
  TName extends FieldPath&lt;TFieldValues&gt; = FieldPath&lt;TFieldValues&gt;,
&gt; = {
  name: TName;
};

const FormFieldContext = React.createContext&lt;FormFieldContextValue&gt;(
  {} as FormFieldContextValue,
);

const FormField = &lt;
  TFieldValues extends FieldValues = FieldValues,
  TName extends FieldPath&lt;TFieldValues&gt; = FieldPath&lt;TFieldValues&gt;,
&gt;({
  ...props
}: ControllerProps&lt;TFieldValues, TName&gt;) =&gt; {
  return (
    &lt;FormFieldContext.Provider value={{ name: props.name }}&gt;
      &lt;Controller {...props} /&gt;
    &lt;/FormFieldContext.Provider&gt;
  );
};

const useFormField = () =&gt; {
  const fieldContext = React.useContext(FormFieldContext);
  const itemContext = React.useContext(FormItemContext);
  const { getFieldState } = useFormContext();
  const formState = useFormState({ name: fieldContext.name });
  const fieldState = getFieldState(fieldContext.name, formState);

  if (!fieldContext) {
    throw new Error("useFormField should be used within &lt;FormField&gt;");
  }

  const { id } = itemContext;

  return {
    id,
    name: fieldContext.name,
    formItemId: `${id}-form-item`,
    formDescriptionId: `${id}-form-item-description`,
    formMessageId: `${id}-form-item-message`,
    ...fieldState,
  };
};

type FormItemContextValue = {
  id: string;
};

const FormItemContext = React.createContext&lt;FormItemContextValue&gt;(
  {} as FormItemContextValue,
);

function FormItem({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  const id = React.useId();

  return (
```

```
    &lt;FormItemContext.Provider value={{ id }}&gt;
      &lt;div
        data-slot="form-item"
        className={cn("grid gap-2", className)}
        {...props}
      /&gt;
    &lt;/FormItemContext.Provider&gt;
  );
}

function FormLabel({
  className,
  ...props
}: React.ComponentProps&lt;typeof LabelPrimitive.Root&gt;) {
  const { error, formItemId } = useFormField();

  return (
    &lt;Label
      data-slot="form-label"
      data-error={!!error}
      className={cn("data-[error=true]:text-destructive", className)}
      htmlFor={formItemId}
      {...props}
    /&gt;
  );
}

function FormControl({ ...props }: React.ComponentProps&lt;typeof Slot&gt;) {
  const { error, formItemId, formDescriptionId, formMessageId } =
    useFormField();

  return (
    &lt;Slot
      data-slot="form-control"
      id={formItemId}
      aria-describedby={
        !error
          ? `${formDescriptionId}`
          : `${formDescriptionId} ${formMessageId}`
      }
      aria-invalid={!!error}
      {...props}
    /&gt;
  );
}

function FormDescription({ className, ...props }: React.ComponentProps&lt;"p"&gt;) {
  const { formDescriptionId } = useFormField();

  return (
    &lt;p
      data-slot="form-description"
      id={formDescriptionId}
      className={cn("text-muted-foreground text-sm", className)}
      {...props}
    /&gt;
  );
}

function FormMessage({ className, ...props }: React.ComponentProps&lt;"p"&gt;) {
  const { error, formMessageId } = useFormField();
  const body = error ? String(error?.message ?? "") : props.children;

  if (!body) {
    return null;
  }

  return (
    &lt;p
      data-slot="form-message"
      id={formMessageId}
      className={cn("text-destructive text-sm", className)}
      {...props}
    &gt;
      {body}
    &lt;/p&gt;
  );
}

export {
  useFormField,
  Form,
```

```
  FormItem,
  FormLabel,
  FormControl,
  FormDescription,
  FormMessage,
  FormField,
};
```

■ File: src\components\ui\hover-card.tsx

```
"use client";

import * as React from "react";
import * as HoverCardPrimitive from "@radix-ui/react-hover-card@1.1.6";

import { cn } from "./utils";

function HoverCard({
  ...props
}: React.ComponentProps&lt;typeof HoverCardPrimitive.Root&gt;) {
  return &lt;HoverCardPrimitive.Root data-slot="hover-card" {...props} /&gt;;
}

function HoverCardTrigger({
  ...props
}: React.ComponentProps&lt;typeof HoverCardPrimitive.Trigger&gt;) {
  return (
    &lt;HoverCardPrimitive.Trigger data-slot="hover-card-trigger" {...props} /&gt;
  );
}

function HoverCardContent({
  className,
  align = "center",
  sideOffset = 4,
  ...props
}: React.ComponentProps&lt;typeof HoverCardPrimitive.Content&gt;) {
  return (
    &lt;HoverCardPrimitive.Portal data-slot="hover-card-portal"&gt;
      &lt;HoverCardPrimitive.Content
        data-slot="hover-card-content"
        align={align}
        sideOffset={sideOffset}
        className={cn(
          "bg-popover text-popover-foreground data-[state=open]:animate-in
     data-[state=closed]:animate-out data-[state=closed]:fade-out-0
     data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
     data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
     data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
     data-[side=top]:slide-in-from-bottom-2 z-50 w-64
     origin-(--radix-hover-card-content-transform-origin) rounded-md border p-4 shadow-md
     outline-hidden",
          className,
        )}
        {...props}
      /&gt;
    &lt;/HoverCardPrimitive.Portal&gt;
  );
}

export { HoverCard, HoverCardTrigger, HoverCardContent };
```

■ File: src\components\ui\input-otp.tsx

```
"use client";

import * as React from "react";
import { OTPInput, OTPInputContext } from "input-otp@1.4.2";
import { MinusIcon } from "lucide-react@0.487.0";

import { cn } from "./utils";

function InputOTP({
  className,
  containerClassName,
  ...props
}: React.ComponentProps&lt;typeof OTPInput&gt; &amp; {
  containerClassName?: string;
}) {
  return (
    &lt;OTPInput
      data-slot="input-otp"
      containerClassName={cn(
        "flex items-center gap-2 has-disabled:opacity-50",
        containerClassName,
      )}
      className={cn("disabled:cursor-not-allowed", className)}
      {...props}
    /&gt;
  );
}

function InputOTPGroup({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="input-otp-group"
      className={cn("flex items-center gap-1", className)}
      {...props}
    /&gt;
  );
}

function InputOTPSlot({
  index,
  className,
  ...props
}: React.ComponentProps&lt;"div"&gt; &amp; {
  index: number;
}) {
  const inputOTPContext = React.useContext(OTPInputContext);
  const { char, hasFakeCaret, isActive } = inputOTPContext?.slots[index] ?? {};

  return (
    &lt;div
      data-slot="input-otp-slot"
      data-active={isActive}
      className={cn(
        "data-[active=true]:border-ring data-[active=true]:ring-ring/50
    data-[active=true]:aria-invalid:ring-destructive/20
    dark:data-[active=true]:aria-invalid:ring-destructive/40
    aria-invalid:border-destructive data-[active=true]:aria-invalid:border-destructive
    dark:bg-input/30 border-input relative flex h-9 w-9 items-center justify-center
    border-y border-r text-sm bg-input-background transition-all outline-none
    first:rounded-l-md first:border-l last:rounded-r-md data-[active=true]:z-10
    data-[active=true]:ring-[3px]",
        className,
      )}
      {...props}
    &gt;
      {char}
      {hasFakeCaret &amp;&amp; (
        &lt;div className="pointer-events-none absolute inset-0 flex items-center
    justify-center"&gt;
          &lt;div className="animate-caret-blink bg-foreground h-4 w-px duration-1000" /&gt;
        &lt;/div&gt;
      )}
    &lt;/div&gt;
  );
}

function InputOTPSeparator({ ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div data-slot="input-otp-separator" role="separator" {...props}&gt;
```

```
      &lt;MinusIcon /&gt;
    &lt;/div&gt;
  );
}

export { InputOTP, InputOTPGroup, InputOTPSlot, InputOTPSeparator };
```

■ File: src\components\ui\input.tsx

```
import * as React from "react";

import { cn } from "./utils";

function Input({ className, type, ...props }: React.ComponentProps&lt;"input"&gt;) {
  return (
    &lt;input
      type={type}
      data-slot="input"
      className={cn(
        "file:text-foreground placeholder:text-muted-foreground selection:bg-primary
     selection:text-primary-foreground dark:bg-input/30 border-input flex h-9 w-full
     min-w-0 rounded-md border px-3 py-1 text-base bg-input-background
     transition-[color,box-shadow] outline-none file:inline-flex file:h-7 file:border-0
     file:bg-transparent file:text-sm file:font-medium disabled:pointer-events-none
     disabled:cursor-not-allowed disabled:opacity-50 md:text-sm",
        "focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-[3px]",
        "aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40
     aria-invalid:border-destructive",
        className,
      )}
      {...props}
    /&gt;
  );
}

export { Input };
```

■ File: src\components\ui\label.tsx

```
"use client";

import * as React from "react";
import * as LabelPrimitive from "@radix-ui/react-label@2.1.2";

import { cn } from "./utils";

function Label({
  className,
  ...props
}: React.ComponentProps&lt;typeof LabelPrimitive.Root&gt;) {
  return (
    &lt;LabelPrimitive.Root
      data-slot="label"
      className={cn(
        "flex items-center gap-2 text-sm leading-none font-medium select-none
     group-data-[disabled=true]:pointer-events-none group-data-[disabled=true]:opacity-50
     peer-disabled:cursor-not-allowed peer-disabled:opacity-50",
        className,
      )}
      {...props}
    /&gt;
  );
}

export { Label };
```

■ File: src\components\ui\menubar.tsx

```
"use client";

import * as React from "react";
import * as MenubarPrimitive from "@radix-ui/react-menubar@1.1.6";
import { CheckIcon, ChevronRightIcon, CircleIcon } from "lucide-react@0.487.0";

import { cn } from "./utils";

function Menubar({
  className,
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.Root&gt;) {
  return (
    &lt;MenubarPrimitive.Root
      data-slot="menubar"
      className={cn(
        "bg-background flex h-9 items-center gap-1 rounded-md border p-1 shadow-xs",
        className,
      )}
      {...props}
    /&gt;
  );
}

function MenubarMenu({
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.Menu&gt;) {
  return &lt;MenubarPrimitive.Menu data-slot="menubar-menu" {...props} /&gt;;
}

function MenubarGroup({
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.Group&gt;) {
  return &lt;MenubarPrimitive.Group data-slot="menubar-group" {...props} /&gt;;
}

function MenubarPortal({
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.Portal&gt;) {
  return &lt;MenubarPrimitive.Portal data-slot="menubar-portal" {...props} /&gt;;
}

function MenubarRadioGroup({
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.RadioGroup&gt;) {
  return (
    &lt;MenubarPrimitive.RadioGroup data-slot="menubar-radio-group" {...props} /&gt;
  );
}

function MenubarTrigger({
  className,
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.Trigger&gt;) {
  return (
    &lt;MenubarPrimitive.Trigger
      data-slot="menubar-trigger"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent
     data-[state=open]:text-accent-foreground flex items-center rounded-sm px-2 py-1
     text-sm font-medium outline-hidden select-none",
        className,
      )}
      {...props}
    /&gt;
  );
}

function MenubarContent({
  className,
  align = "start",
  alignOffset = -4,
  sideOffset = 8,
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.Content&gt;) {
  return (
    &lt;MenubarPortal&gt;
      &lt;MenubarPrimitive.Content
        data-slot="menubar-content"
```

```
        align={align}
        alignOffset={alignOffset}
        sideOffset={sideOffset}
        className={cn(
          "bg-popover text-popover-foreground data-[state=open]:animate-in
     data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
     data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95
     data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2
     data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 z-50
     min-w-[12rem] origin-(--radix-menubar-content-transform-origin) overflow-hidden
     rounded-md border p-1 shadow-md",
          className,
        )}
        {...props}
      /&gt;
    &lt;/MenubarPortal&gt;
  );
}

function MenubarItem({
  className,
  inset,
  variant = "default",
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.Item&gt; &amp; {
  inset?: boolean;
  variant?: "default" | "destructive";
}) {
  return (
    &lt;MenubarPrimitive.Item
      data-slot="menubar-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground
     data-[variant=destructive]:text-destructive
     data-[variant=destructive]:focus:bg-destructive/10
     dark:data-[variant=destructive]:focus:bg-destructive/20
     data-[variant=destructive]:focus:text-destructive
     data-[variant=destructive]:*:[svg]:!text-destructive
     [&amp;_svg:not([class*='text-'])]:text-muted-foreground relative flex cursor-default
     items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-hidden select-none
     data-[disabled]:pointer-events-none data-[disabled]:opacity-50 data-[inset]:pl-8
     [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0 [&amp;_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    /&gt;
  );
}

function MenubarCheckboxItem({
  className,
  children,
  checked,
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.CheckboxItem&gt;) {
  return (
    &lt;MenubarPrimitive.CheckboxItem
      data-slot="menubar-checkbox-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default
     items-center gap-2 rounded-xs py-1.5 pr-2 pl-8 text-sm outline-hidden select-none
     data-[disabled]:pointer-events-none data-[disabled]:opacity-50
     [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0 [&amp;_svg:not([class*='size-'])]:size-4",
        className,
      )}
      checked={checked}
      {...props}
    &gt;
      &lt;span className="pointer-events-none absolute left-2 flex size-3.5 items-center
     justify-center"&gt;
        &lt;MenubarPrimitive.ItemIndicator&gt;
          &lt;CheckIcon className="size-4" /&gt;
        &lt;/MenubarPrimitive.ItemIndicator&gt;
      &lt;/span&gt;
      {children}
    &lt;/MenubarPrimitive.CheckboxItem&gt;
  );
}

function MenubarRadioItem({
```

```
  className,
  children,
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.RadioItem&gt;) {
  return (
    &lt;MenubarPrimitive.RadioItem
      data-slot="menubar-radio-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default
     items-center gap-2 rounded-xs py-1.5 pr-2 pl-8 text-sm outline-hidden select-none
     data-[disabled]:pointer-events-none data-[disabled]:opacity-50
     [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0 [&amp;_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    &gt;
      &lt;span className="pointer-events-none absolute left-2 flex size-3.5 items-center
     justify-center"&gt;
        &lt;MenubarPrimitive.ItemIndicator&gt;
          &lt;CircleIcon className="size-2 fill-current" /&gt;
        &lt;/MenubarPrimitive.ItemIndicator&gt;
      &lt;/span&gt;
      {children}
    &lt;/MenubarPrimitive.RadioItem&gt;
  );
}

function MenubarLabel({
  className,
  inset,
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.Label&gt; &amp; {
  inset?: boolean;
}) {
  return (
    &lt;MenubarPrimitive.Label
      data-slot="menubar-label"
      data-inset={inset}
      className={cn(
        "px-2 py-1.5 text-sm font-medium data-[inset]:pl-8",
        className,
      )}
      {...props}
    /&gt;
  );
}

function MenubarSeparator({
  className,
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.Separator&gt;) {
  return (
    &lt;MenubarPrimitive.Separator
      data-slot="menubar-separator"
      className={cn("bg-border -mx-1 my-1 h-px", className)}
      {...props}
    /&gt;
  );
}

function MenubarShortcut({
  className,
  ...props
}: React.ComponentProps&lt;"span"&gt;) {
  return (
    &lt;span
      data-slot="menubar-shortcut"
      className={cn(
        "text-muted-foreground ml-auto text-xs tracking-widest",
        className,
      )}
      {...props}
    /&gt;
  );
}

function MenubarSub({
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.Sub&gt;) {
  return &lt;MenubarPrimitive.Sub data-slot="menubar-sub" {...props} /&gt;;
}
```

```
function MenubarSubTrigger({
  className,
  inset,
  children,
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.SubTrigger&gt; &amp; {
  inset?: boolean;
}) {
  return (
    &lt;MenubarPrimitive.SubTrigger
      data-slot="menubar-sub-trigger"
      data-inset={inset}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent
     data-[state=open]:text-accent-foreground flex cursor-default items-center rounded-sm
     px-2 py-1.5 text-sm outline-none select-none data-[inset]:pl-8",
        className,
      )}
      {...props}
    &gt;
      {children}
      &lt;ChevronRightIcon className="ml-auto h-4 w-4" /&gt;
    &lt;/MenubarPrimitive.SubTrigger&gt;
  );
}

function MenubarSubContent({
  className,
  ...props
}: React.ComponentProps&lt;typeof MenubarPrimitive.SubContent&gt;) {
  return (
    &lt;MenubarPrimitive.SubContent
      data-slot="menubar-sub-content"
      className={cn(
        "bg-popover text-popover-foreground data-[state=open]:animate-in
     data-[state=closed]:animate-out data-[state=closed]:fade-out-0
     data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
     data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
     data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
     data-[side=top]:slide-in-from-bottom-2 z-50 min-w-[8rem]
     origin-(--radix-menubar-content-transform-origin) overflow-hidden rounded-md border
     p-1 shadow-lg",
        className,
      )}
      {...props}
    /&gt;
  );
}

export {
  Menubar,
  MenubarPortal,
  MenubarMenu,
  MenubarTrigger,
  MenubarContent,
  MenubarGroup,
  MenubarSeparator,
  MenubarLabel,
  MenubarItem,
  MenubarShortcut,
  MenubarCheckboxItem,
  MenubarRadioGroup,
  MenubarRadioItem,
  MenubarSub,
  MenubarSubTrigger,
  MenubarSubContent,
};
```

■ File: src\components\ui\navigation-menu.tsx

```
import * as React from "react";
import * as NavigationMenuPrimitive from "@radix-ui/react-navigation-menu@1.2.5";
import { cva } from "class-variance-authority@0.7.1";
import { ChevronDownIcon } from "lucide-react@0.487.0";

import { cn } from "./utils";

function NavigationMenu({
  className,
  children,
  viewport = true,
  ...props
}: React.ComponentProps&lt;typeof NavigationMenuPrimitive.Root&gt; &amp; {
  viewport?: boolean;
}) {
  return (
    &lt;NavigationMenuPrimitive.Root
      data-slot="navigation-menu"
      data-viewport={viewport}
      className={cn(
        "group/navigation-menu relative flex max-w-max flex-1 items-center
     justify-center",
        className,
      )}
      {...props}
    &gt;
      {children}
      {viewport &amp;&amp; &lt;NavigationMenuViewport /&gt;}
    &lt;/NavigationMenuPrimitive.Root&gt;
  );
}

function NavigationMenuList({
  className,
  ...props
}: React.ComponentProps&lt;typeof NavigationMenuPrimitive.List&gt;) {
  return (
    &lt;NavigationMenuPrimitive.List
      data-slot="navigation-menu-list"
      className={cn(
        "group flex flex-1 list-none items-center justify-center gap-1",
        className,
      )}
      {...props}
    /&gt;
  );
}

function NavigationMenuItem({
  className,
  ...props
}: React.ComponentProps&lt;typeof NavigationMenuPrimitive.Item&gt;) {
  return (
    &lt;NavigationMenuPrimitive.Item
      data-slot="navigation-menu-item"
      className={cn("relative", className)}
      {...props}
    /&gt;
  );
}

const navigationMenuTriggerStyle = cva(
  "group inline-flex h-9 w-max items-center justify-center rounded-md bg-background px-4
     py-2 text-sm font-medium hover:bg-accent hover:text-accent-foreground
     focus:bg-accent focus:text-accent-foreground disabled:pointer-events-none
     disabled:opacity-50 data-[state=open]:hover:bg-accent
     data-[state=open]:text-accent-foreground data-[state=open]:focus:bg-accent
     data-[state=open]:bg-accent/50 focus-visible:ring-ring/50 outline-none
     transition-[color,box-shadow] focus-visible:ring-[3px] focus-visible:outline-1",
);

function NavigationMenuTrigger({
  className,
  children,
  ...props
}: React.ComponentProps&lt;typeof NavigationMenuPrimitive.Trigger&gt;) {
  return (
    &lt;NavigationMenuPrimitive.Trigger
      data-slot="navigation-menu-trigger"
```

```
      className={cn(navigationMenuTriggerStyle(), "group", className)}
      {...props}
    &gt;
      {children}{" "}
      &lt;ChevronDownIcon
        className="relative top-[1px] ml-1 size-3 transition duration-300
     group-data-[state=open]:rotate-180"
        aria-hidden="true"
      /&gt;
    &lt;/NavigationMenuPrimitive.Trigger&gt;
  );
}

function NavigationMenuContent({
  className,
  ...props
}: React.ComponentProps&lt;typeof NavigationMenuPrimitive.Content&gt;) {
  return (
    &lt;NavigationMenuPrimitive.Content
      data-slot="navigation-menu-content"
      className={cn(
        "data-[motion^=from-]:animate-in data-[motion^=to-]:animate-out
     data-[motion^=from-]:fade-in data-[motion^=to-]:fade-out
     data-[motion=from-end]:slide-in-from-right-52
     data-[motion=from-start]:slide-in-from-left-52
     data-[motion=to-end]:slide-out-to-right-52
     data-[motion=to-start]:slide-out-to-left-52 top-0 left-0 w-full p-2 pr-2.5
     md:absolute md:w-auto",
        "group-data-[viewport=false]/navigation-menu:bg-popover
     group-data-[viewport=false]/navigation-menu:text-popover-foreground
     group-data-[viewport=false]/navigation-menu:data-[state=open]:animate-in
     group-data-[viewport=false]/navigation-menu:data-[state=closed]:animate-out
     group-data-[viewport=false]/navigation-menu:data-[state=closed]:zoom-out-95
     group-data-[viewport=false]/navigation-menu:data-[state=open]:zoom-in-95
     group-data-[viewport=false]/navigation-menu:data-[state=open]:fade-in-0
     group-data-[viewport=false]/navigation-menu:data-[state=closed]:fade-out-0
     group-data-[viewport=false]/navigation-menu:top-full
     group-data-[viewport=false]/navigation-menu:mt-1.5
     group-data-[viewport=false]/navigation-menu:overflow-hidden
     group-data-[viewport=false]/navigation-menu:rounded-md
     group-data-[viewport=false]/navigation-menu:border
     group-data-[viewport=false]/navigation-menu:shadow
     group-data-[viewport=false]/navigation-menu:duration-200
     **:data-[slot=navigation-menu-link]:focus:ring-0
     **:data-[slot=navigation-menu-link]:focus:outline-none",
        className,
      )}
      {...props}
    /&gt;
  );
}

function NavigationMenuViewport({
  className,
  ...props
}: React.ComponentProps&lt;typeof NavigationMenuPrimitive.Viewport&gt;) {
  return (
    &lt;div
      className={cn(
        "absolute top-full left-0 isolate z-50 flex justify-center",
      )}
    &gt;
      &lt;NavigationMenuPrimitive.Viewport
        data-slot="navigation-menu-viewport"
        className={cn(
          "origin-top-center bg-popover text-popover-foreground
     data-[state=open]:animate-in data-[state=closed]:animate-out
     data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-90 relative mt-1.5
     h-[var(--radix-navigation-menu-viewport-height)] w-full overflow-hidden rounded-md
     border shadow md:w-[var(--radix-navigation-menu-viewport-width)]",
          className,
        )}
        {...props}
      /&gt;
    &lt;/div&gt;
  );
}

function NavigationMenuLink({
  className,
  ...props
}: React.ComponentProps&lt;typeof NavigationMenuPrimitive.Link&gt;) {
```

```
  return (
    &lt;NavigationMenuPrimitive.Link
      data-slot="navigation-menu-link"
      className={cn(
        "data-[active=true]:focus:bg-accent data-[active=true]:hover:bg-accent
     data-[active=true]:bg-accent/50 data-[active=true]:text-accent-foreground
     hover:bg-accent hover:text-accent-foreground focus:bg-accent
     focus:text-accent-foreground focus-visible:ring-ring/50
     [&amp;_svg:not([class*='text-'])]:text-muted-foreground flex flex-col gap-1 rounded-sm
     p-2 text-sm transition-all outline-none focus-visible:ring-[3px]
     focus-visible:outline-1 [&amp;_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    /&gt;
  );
}

function NavigationMenuIndicator({
  className,
  ...props
}: React.ComponentProps&lt;typeof NavigationMenuPrimitive.Indicator&gt;) {
  return (
    &lt;NavigationMenuPrimitive.Indicator
      data-slot="navigation-menu-indicator"
      className={cn(
        "data-[state=visible]:animate-in data-[state=hidden]:animate-out
     data-[state=hidden]:fade-out data-[state=visible]:fade-in top-full z-[1] flex h-1.5
     items-end justify-center overflow-hidden",
        className,
      )}
      {...props}
    &gt;
      &lt;div className="bg-border relative top-[60%] h-2 w-2 rotate-45 rounded-tl-sm
     shadow-md" /&gt;
    &lt;/NavigationMenuPrimitive.Indicator&gt;
  );
}

export {
  NavigationMenu,
  NavigationMenuList,
  NavigationMenuItem,
  NavigationMenuContent,
  NavigationMenuTrigger,
  NavigationMenuLink,
  NavigationMenuIndicator,
  NavigationMenuViewport,
  navigationMenuTriggerStyle,
};
```

■ File: src\components\ui\pagination.tsx

```
import * as React from "react";
import {
  ChevronLeftIcon,
  ChevronRightIcon,
  MoreHorizontalIcon,
} from "lucide-react@0.487.0";

import { cn } from "./utils";
import { Button, buttonVariants } from "./button";

function Pagination({ className, ...props }: React.ComponentProps&lt;"nav"&gt;) {
  return (
    &lt;nav
      role="navigation"
      aria-label="pagination"
      data-slot="pagination"
      className={cn("mx-auto flex w-full justify-center", className)}
      {...props}
    /&gt;
  );
}

function PaginationContent({
  className,
  ...props
}: React.ComponentProps&lt;"ul"&gt;) {
  return (
    &lt;ul
      data-slot="pagination-content"
      className={cn("flex flex-row items-center gap-1", className)}
      {...props}
    /&gt;
  );
}

function PaginationItem({ ...props }: React.ComponentProps&lt;"li"&gt;) {
  return &lt;li data-slot="pagination-item" {...props} /&gt;;
}

type PaginationLinkProps = {
  isActive?: boolean;
} &amp; Pick&lt;React.ComponentProps&lt;typeof Button&gt;, "size"&gt; &amp;
  React.ComponentProps&lt;"a"&gt;;

function PaginationLink({
  className,
  isActive,
  size = "icon",
  ...props
}: PaginationLinkProps) {
  return (
    &lt;a
      aria-current={isActive ? "page" : undefined}
      data-slot="pagination-link"
      data-active={isActive}
      className={cn(
        buttonVariants({
          variant: isActive ? "outline" : "ghost",
          size,
        }),
        className,
      )}
      {...props}
    /&gt;
  );
}

function PaginationPrevious({
  className,
  ...props
}: React.ComponentProps&lt;typeof PaginationLink&gt;) {
  return (
    &lt;PaginationLink
      aria-label="Go to previous page"
      size="default"
      className={cn("gap-1 px-2.5 sm:pl-2.5", className)}
      {...props}
    &gt;
      &lt;ChevronLeftIcon /&gt;
```

```
      &lt;span className="hidden sm:block"&gt;Previous&lt;/span&gt;
    &lt;/PaginationLink&gt;
  );
}

function PaginationNext({
  className,
  ...props
}: React.ComponentProps&lt;typeof PaginationLink&gt;) {
  return (
    &lt;PaginationLink
      aria-label="Go to next page"
      size="default"
      className={cn("gap-1 px-2.5 sm:pr-2.5", className)}
      {...props}
    &gt;
      &lt;span className="hidden sm:block"&gt;Next&lt;/span&gt;
      &lt;ChevronRightIcon /&gt;
    &lt;/PaginationLink&gt;
  );
}

function PaginationEllipsis({
  className,
  ...props
}: React.ComponentProps&lt;"span"&gt;) {
  return (
    &lt;span
      aria-hidden
      data-slot="pagination-ellipsis"
      className={cn("flex size-9 items-center justify-center", className)}
      {...props}
    &gt;
      &lt;MoreHorizontalIcon className="size-4" /&gt;
      &lt;span className="sr-only"&gt;More pages&lt;/span&gt;
    &lt;/span&gt;
  );
}

export {
  Pagination,
  PaginationContent,
  PaginationLink,
  PaginationItem,
  PaginationPrevious,
  PaginationNext,
  PaginationEllipsis,
};
```

■ File: src\components\ui\popover.tsx

```
"use client";

import * as React from "react";
import * as PopoverPrimitive from "@radix-ui/react-popover@1.1.6";

import { cn } from "./utils";

function Popover({
  ...props
}: React.ComponentProps&lt;typeof PopoverPrimitive.Root&gt;) {
  return &lt;PopoverPrimitive.Root data-slot="popover" {...props} /&gt;;
}

function PopoverTrigger({
  ...props
}: React.ComponentProps&lt;typeof PopoverPrimitive.Trigger&gt;) {
  return &lt;PopoverPrimitive.Trigger data-slot="popover-trigger" {...props} /&gt;;
}

function PopoverContent({
  className,
  align = "center",
  sideOffset = 4,
  ...props
}: React.ComponentProps&lt;typeof PopoverPrimitive.Content&gt;) {
  return (
    &lt;PopoverPrimitive.Portal&gt;
      &lt;PopoverPrimitive.Content
        data-slot="popover-content"
        align={align}
        sideOffset={sideOffset}
        className={cn(
          "bg-popover text-popover-foreground data-[state=open]:animate-in
     data-[state=closed]:animate-out data-[state=closed]:fade-out-0
     data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
     data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
     data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
     data-[side=top]:slide-in-from-bottom-2 z-50 w-72
     origin-(--radix-popover-content-transform-origin) rounded-md border p-4 shadow-md
     outline-hidden",
          className,
        )}
        {...props}
      /&gt;
    &lt;/PopoverPrimitive.Portal&gt;
  );
}

function PopoverAnchor({
  ...props
}: React.ComponentProps&lt;typeof PopoverPrimitive.Anchor&gt;) {
  return &lt;PopoverPrimitive.Anchor data-slot="popover-anchor" {...props} /&gt;;
}

export { Popover, PopoverTrigger, PopoverContent, PopoverAnchor };
```

■ File: src\components\ui\progress.tsx

```
"use client";

import * as React from "react";
import * as ProgressPrimitive from "@radix-ui/react-progress@1.1.2";

import { cn } from "./utils";

function Progress({
  className,
  value,
  ...props
}: React.ComponentProps&lt;typeof ProgressPrimitive.Root&gt;) {
  return (
    &lt;ProgressPrimitive.Root
      data-slot="progress"
      className={cn(
        "bg-primary/20 relative h-2 w-full overflow-hidden rounded-full",
        className,
      )}
      {...props}
    &gt;
      &lt;ProgressPrimitive.Indicator
        data-slot="progress-indicator"
        className="bg-primary h-full w-full flex-1 transition-all"
        style={{ transform: `translateX(-${100 - (value || 0)}%)` }}
      /&gt;
    &lt;/ProgressPrimitive.Root&gt;
  );
}

export { Progress };
```

■ File: src\components\ui\radio-group.tsx

```
"use client";

import * as React from "react";
import * as RadioGroupPrimitive from "@radix-ui/react-radio-group@1.2.3";
import { CircleIcon } from "lucide-react@0.487.0";

import { cn } from "./utils";

function RadioGroup({
  className,
  ...props
}: React.ComponentProps&lt;typeof RadioGroupPrimitive.Root&gt;) {
  return (
    &lt;RadioGroupPrimitive.Root
      data-slot="radio-group"
      className={cn("grid gap-3", className)}
      {...props}
    /&gt;
  );
}

function RadioGroupItem({
  className,
  ...props
}: React.ComponentProps&lt;typeof RadioGroupPrimitive.Item&gt;) {
  return (
    &lt;RadioGroupPrimitive.Item
      data-slot="radio-group-item"
      className={cn(
        "border-input text-primary focus-visible:border-ring focus-visible:ring-ring/50
     aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40
     aria-invalid:border-destructive dark:bg-input/30 aspect-square size-4 shrink-0
     rounded-full border shadow-xs transition-[color,box-shadow] outline-none
     focus-visible:ring-[3px] disabled:cursor-not-allowed disabled:opacity-50",
        className,
      )}
      {...props}
    &gt;
      &lt;RadioGroupPrimitive.Indicator
        data-slot="radio-group-indicator"
        className="relative flex items-center justify-center"
      &gt;
        &lt;CircleIcon className="fill-primary absolute top-1/2 left-1/2 size-2
     -translate-x-1/2 -translate-y-1/2" /&gt;
      &lt;/RadioGroupPrimitive.Indicator&gt;
    &lt;/RadioGroupPrimitive.Item&gt;
  );
}

export { RadioGroup, RadioGroupItem };
```

■ File: src\components\ui\resizable.tsx

```
"use client";

import * as React from "react";
import { GripVerticalIcon } from "lucide-react@0.487.0";
import * as ResizablePrimitive from "react-resizable-panels@2.1.7";

import { cn } from "./utils";

function ResizablePanelGroup({
  className,
  ...props
}: React.ComponentProps&lt;typeof ResizablePrimitive.PanelGroup&gt;) {
  return (
    &lt;ResizablePrimitive.PanelGroup
      data-slot="resizable-panel-group"
      className={cn(
        "flex h-full w-full data-[panel-group-direction=vertical]:flex-col",
        className,
      )}
      {...props}
    /&gt;
  );
}

function ResizablePanel({
  ...props
}: React.ComponentProps&lt;typeof ResizablePrimitive.Panel&gt;) {
  return &lt;ResizablePrimitive.Panel data-slot="resizable-panel" {...props} /&gt;;
}

function ResizableHandle({
  withHandle,
  className,
  ...props
}: React.ComponentProps&lt;typeof ResizablePrimitive.PanelResizeHandle&gt; &amp; {
  withHandle?: boolean;
}) {
  return (
    &lt;ResizablePrimitive.PanelResizeHandle
      data-slot="resizable-handle"
      className={cn(
        "bg-border focus-visible:ring-ring relative flex w-px items-center justify-center
     after:absolute after:inset-y-0 after:left-1/2 after:w-1 after:-translate-x-1/2
     focus-visible:ring-1 focus-visible:ring-offset-1 focus-visible:outline-hidden
     data-[panel-group-direction=vertical]:h-px
     data-[panel-group-direction=vertical]:w-full
     data-[panel-group-direction=vertical]:after:left-0
     data-[panel-group-direction=vertical]:after:h-1
     data-[panel-group-direction=vertical]:after:w-full
     data-[panel-group-direction=vertical]:after:-translate-y-1/2
     data-[panel-group-direction=vertical]:after:translate-x-0
     [&amp;[data-panel-group-direction=vertical]&gt;div]:rotate-90",
        className,
      )}
      {...props}
    &gt;
      {withHandle &amp;&amp; (
        &lt;div className="bg-border z-10 flex h-4 w-3 items-center justify-center
     rounded-xs border"&gt;
          &lt;GripVerticalIcon className="size-2.5" /&gt;
        &lt;/div&gt;
      )}
    &lt;/ResizablePrimitive.PanelResizeHandle&gt;
  );
}

export { ResizablePanelGroup, ResizablePanel, ResizableHandle };
```

■ File: src\components\ui\scroll-area.tsx

```
"use client";

import * as React from "react";
import * as ScrollAreaPrimitive from "@radix-ui/react-scroll-area@1.2.3";

import { cn } from "./utils";

function ScrollArea({
  className,
  children,
  ...props
}: React.ComponentProps&lt;typeof ScrollAreaPrimitive.Root&gt;) {
  return (
    &lt;ScrollAreaPrimitive.Root
      data-slot="scroll-area"
      className={cn("relative", className)}
      {...props}
    &gt;
      &lt;ScrollAreaPrimitive.Viewport
        data-slot="scroll-area-viewport"
        className="focus-visible:ring-ring/50 size-full rounded-[inherit]
     transition-[color,box-shadow] outline-none focus-visible:ring-[3px]
     focus-visible:outline-1"
      &gt;
        {children}
      &lt;/ScrollAreaPrimitive.Viewport&gt;
      &lt;ScrollBar /&gt;
      &lt;ScrollAreaPrimitive.Corner /&gt;
    &lt;/ScrollAreaPrimitive.Root&gt;
  );
}

function ScrollBar({
  className,
  orientation = "vertical",
  ...props
}: React.ComponentProps&lt;typeof ScrollAreaPrimitive.ScrollAreaScrollbar&gt;) {
  return (
    &lt;ScrollAreaPrimitive.ScrollAreaScrollbar
      data-slot="scroll-area-scrollbar"
      orientation={orientation}
      className={cn(
        "flex touch-none p-px transition-colors select-none",
        orientation === "vertical" &amp;amp;&amp;amp;
          "h-full w-2.5 border-l border-l-transparent",
        orientation === "horizontal" &amp;amp;&amp;amp;
          "h-2.5 flex-col border-t border-t-transparent",
        className,
      )}
      {...props}
    &gt;
      &lt;ScrollAreaPrimitive.ScrollAreaThumb
        data-slot="scroll-area-thumb"
        className="bg-border relative flex-1 rounded-full"
      /&gt;
    &lt;/ScrollAreaPrimitive.ScrollAreaScrollbar&gt;
  );
}

export { ScrollArea, ScrollBar };
```

■ File: src\components\ui\select.tsx

```
"use client";

import * as React from "react";
import * as SelectPrimitive from "@radix-ui/react-select@2.1.6";
import {
  CheckIcon,
  ChevronDownIcon,
  ChevronUpIcon,
} from "lucide-react@0.487.0";

import { cn } from "./utils";

function Select({
  ...props
}: React.ComponentProps&lt;typeof SelectPrimitive.Root&gt;) {
  return &lt;SelectPrimitive.Root data-slot="select" {...props} /&gt;;
}

function SelectGroup({
  ...props
}: React.ComponentProps&lt;typeof SelectPrimitive.Group&gt;) {
  return &lt;SelectPrimitive.Group data-slot="select-group" {...props} /&gt;;
}

function SelectValue({
  ...props
}: React.ComponentProps&lt;typeof SelectPrimitive.Value&gt;) {
  return &lt;SelectPrimitive.Value data-slot="select-value" {...props} /&gt;;
}

function SelectTrigger({
  className,
  size = "default",
  children,
  ...props
}: React.ComponentProps&lt;typeof SelectPrimitive.Trigger&gt; &amp; {
  size?: "sm" | "default";
}) {
  return (
    &lt;SelectPrimitive.Trigger
      data-slot="select-trigger"
      data-size={size}
      className={cn(
        "border-input data-[placeholder]:text-muted-foreground
    [&amp;_svg:not([class*='text-'])]:text-muted-foreground focus-visible:border-ring
    focus-visible:ring-ring/50 aria-invalid:ring-destructive/20
    dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructive
    dark:bg-input/30 dark:hover:bg-input/50 flex w-full items-center justify-between
    gap-2 rounded-md border bg-input-background px-3 py-2 text-sm whitespace-nowrap
    transition-[color,box-shadow] outline-none focus-visible:ring-[3px]
    disabled:cursor-not-allowed disabled:opacity-50 data-[size=default]:h-9
    data-[size=sm]:h-8 *:data-[slot=select-value]:line-clamp-1
    *:data-[slot=select-value]:flex *:data-[slot=select-value]:items-center
    *:data-[slot=select-value]:gap-2 [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0
    [&amp;_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    &gt;
      {children}
      &lt;SelectPrimitive.Icon asChild&gt;
        &lt;ChevronDownIcon className="size-4 opacity-50" /&gt;
      &lt;/SelectPrimitive.Icon&gt;
    &lt;/SelectPrimitive.Trigger&gt;
  );
}

function SelectContent({
  className,
  children,
  position = "popper",
  ...props
}: React.ComponentProps&lt;typeof SelectPrimitive.Content&gt;) {
  return (
    &lt;SelectPrimitive.Portal&gt;
      &lt;SelectPrimitive.Content
        data-slot="select-content"
        className={cn(
          "bg-popover text-popover-foreground data-[state=open]:animate-in
```

```
      data-[state=closed]:animate-out data-[state=closed]:fade-out-0
      data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
      data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
      data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
      data-[side=top]:slide-in-from-bottom-2 relative z-50
      max-h-(--radix-select-content-available-height) min-w-[8rem]
      origin-(--radix-select-content-transform-origin) overflow-x-hidden overflow-y-auto
      rounded-md border shadow-md",
           position === "popper" &amp;&amp;
             "data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1
      data-[side=right]:translate-x-1 data-[side=top]:-translate-y-1",
           className,
         )}
         position={position}
         {...props}
       &gt;
         &lt;SelectScrollUpButton /&gt;
         &lt;SelectPrimitive.Viewport
           className={cn(
             "p-1",
             position === "popper" &amp;&amp;
               "h-[var(--radix-select-trigger-height)] w-full
      min-w-[var(--radix-select-trigger-width)] scroll-my-1",
           )}
         &gt;
           {children}
         &lt;/SelectPrimitive.Viewport&gt;
         &lt;SelectScrollDownButton /&gt;
       &lt;/SelectPrimitive.Content&gt;
    &lt;/SelectPrimitive.Portal&gt;
  );
}

function SelectLabel({
  className,
  ...props
}: React.ComponentProps&lt;typeof SelectPrimitive.Label&gt;) {
  return (
    &lt;SelectPrimitive.Label
      data-slot="select-label"
      className={cn("text-muted-foreground px-2 py-1.5 text-xs", className)}
      {...props}
    /&gt;
  );
}

function SelectItem({
  className,
  children,
  ...props
}: React.ComponentProps&lt;typeof SelectPrimitive.Item&gt;) {
  return (
    &lt;SelectPrimitive.Item
      data-slot="select-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground
      [&amp;_svg:not([class*='text-'])]:text-muted-foreground relative flex w-full
      cursor-default items-center gap-2 rounded-sm py-1.5 pr-8 pl-2 text-sm outline-hidden
      select-none data-[disabled]:pointer-events-none data-[disabled]:opacity-50
      [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0 [&amp;_svg:not([class*='size-'])]:size-4
      *:[span]:last:flex *:[span]:last:items-center *:[span]:last:gap-2",
        className,
      )}
      {...props}
    &gt;
      &lt;span className="absolute right-2 flex size-3.5 items-center justify-center"&gt;
        &lt;SelectPrimitive.ItemIndicator&gt;
          &lt;CheckIcon className="size-4" /&gt;
        &lt;/SelectPrimitive.ItemIndicator&gt;
      &lt;/span&gt;
      &lt;SelectPrimitive.ItemText&gt;{children}&lt;/SelectPrimitive.ItemText&gt;
    &lt;/SelectPrimitive.Item&gt;
  );
}

function SelectSeparator({
  className,
  ...props
}: React.ComponentProps&lt;typeof SelectPrimitive.Separator&gt;) {
  return (
    &lt;SelectPrimitive.Separator
      data-slot="select-separator"
```

```
      className={cn("bg-border pointer-events-none -mx-1 my-1 h-px", className)}
      {...props}
    /&gt;
  );
}

function SelectScrollUpButton({
  className,
  ...props
}: React.ComponentProps&lt;typeof SelectPrimitive.ScrollUpButton&gt;) {
  return (
    &lt;SelectPrimitive.ScrollUpButton
      data-slot="select-scroll-up-button"
      className={cn(
        "flex cursor-default items-center justify-center py-1",
        className,
      )}
      {...props}
    &gt;
      &lt;ChevronUpIcon className="size-4" /&gt;
    &lt;/SelectPrimitive.ScrollUpButton&gt;
  );
}

function SelectScrollDownButton({
  className,
  ...props
}: React.ComponentProps&lt;typeof SelectPrimitive.ScrollDownButton&gt;) {
  return (
    &lt;SelectPrimitive.ScrollDownButton
      data-slot="select-scroll-down-button"
      className={cn(
        "flex cursor-default items-center justify-center py-1",
        className,
      )}
      {...props}
    &gt;
      &lt;ChevronDownIcon className="size-4" /&gt;
    &lt;/SelectPrimitive.ScrollDownButton&gt;
  );
}

export {
  Select,
  SelectContent,
  SelectGroup,
  SelectItem,
  SelectLabel,
  SelectScrollDownButton,
  SelectScrollUpButton,
  SelectSeparator,
  SelectTrigger,
  SelectValue,
};
```

■ File: src\components\ui\separator.tsx

```
"use client";

import * as React from "react";
import * as SeparatorPrimitive from "@radix-ui/react-separator@1.1.2";

import { cn } from "./utils";

function Separator({
  className,
  orientation = "horizontal",
  decorative = true,
  ...props
}: React.ComponentProps&lt;typeof SeparatorPrimitive.Root&gt;) {
  return (
    &lt;SeparatorPrimitive.Root
      data-slot="separator-root"
      decorative={decorative}
      orientation={orientation}
      className={cn(
        "bg-border shrink-0 data-[orientation=horizontal]:h-px
     data-[orientation=horizontal]:w-full data-[orientation=vertical]:h-full
     data-[orientation=vertical]:w-px",
         className,
      )}
      {...props}
    /&gt;
  );
}

export { Separator };
```

■ File: src\components\ui\sheet.tsx

```
"use client";

import * as React from "react";
import * as SheetPrimitive from "@radix-ui/react-dialog@1.1.6";
import { XIcon } from "lucide-react@0.487.0";

import { cn } from "./utils";

function Sheet({ ...props }: React.ComponentProps&lt;typeof SheetPrimitive.Root&gt;) {
  return &lt;SheetPrimitive.Root data-slot="sheet" {...props} /&gt;;
}

function SheetTrigger({
  ...props
}: React.ComponentProps&lt;typeof SheetPrimitive.Trigger&gt;) {
  return &lt;SheetPrimitive.Trigger data-slot="sheet-trigger" {...props} /&gt;;
}

function SheetClose({
  ...props
}: React.ComponentProps&lt;typeof SheetPrimitive.Close&gt;) {
  return &lt;SheetPrimitive.Close data-slot="sheet-close" {...props} /&gt;;
}

function SheetPortal({
  ...props
}: React.ComponentProps&lt;typeof SheetPrimitive.Portal&gt;) {
  return &lt;SheetPrimitive.Portal data-slot="sheet-portal" {...props} /&gt;;
}

function SheetOverlay({
  className,
  ...props
}: React.ComponentProps&lt;typeof SheetPrimitive.Overlay&gt;) {
  return (
    &lt;SheetPrimitive.Overlay
      data-slot="sheet-overlay"
      className={cn(
        "data-[state=open]:animate-in data-[state=closed]:animate-out
     data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 fixed inset-0 z-50
     bg-black/50",
        className,
      )}
      {...props}
    /&gt;
  );
}

function SheetContent({
  className,
  children,
  side = "right",
  ...props
}: React.ComponentProps&lt;typeof SheetPrimitive.Content&gt; &amp; {
  side?: "top" | "right" | "bottom" | "left";
}) {
  return (
    &lt;SheetPortal&gt;
      &lt;SheetOverlay /&gt;
      &lt;SheetPrimitive.Content
        data-slot="sheet-content"
        className={cn(
          "bg-background data-[state=open]:animate-in data-[state=closed]:animate-out
     fixed z-50 flex flex-col gap-4 shadow-lg transition ease-in-out
     data-[state=closed]:duration-300 data-[state=open]:duration-500",
          side === "right" &amp;&amp;
            "data-[state=closed]:slide-out-to-right data-[state=open]:slide-in-from-right
     inset-y-0 right-0 h-full w-3/4 border-l sm:max-w-sm",
          side === "left" &amp;&amp;
            "data-[state=closed]:slide-out-to-left data-[state=open]:slide-in-from-left
     inset-y-0 left-0 h-full w-3/4 border-r sm:max-w-sm",
          side === "top" &amp;&amp;
            "data-[state=closed]:slide-out-to-top data-[state=open]:slide-in-from-top
     inset-x-0 top-0 h-auto border-b",
          side === "bottom" &amp;&amp;
            "data-[state=closed]:slide-out-to-bottom
     data-[state=open]:slide-in-from-bottom inset-x-0 bottom-0 h-auto border-t",
          className,
        )}
```

```
        {...props}
      &gt;
        {children}
        &lt;SheetPrimitive.Close className="ring-offset-background focus:ring-ring
     data-[state=open]:bg-secondary absolute top-4 right-4 rounded-xs opacity-70
     transition-opacity hover:opacity-100 focus:ring-2 focus:ring-offset-2
     focus:outline-hidden disabled:pointer-events-none"&gt;
          &lt;XIcon className="size-4" /&gt;
          &lt;span className="sr-only"&gt;Close&lt;/span&gt;
        &lt;/SheetPrimitive.Close&gt;
      &lt;/SheetPrimitive.Content&gt;
    &lt;/SheetPortal&gt;
  );
}

function SheetHeader({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="sheet-header"
      className={cn("flex flex-col gap-1.5 p-4", className)}
      {...props}
    /&gt;
  );
}

function SheetFooter({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="sheet-footer"
      className={cn("mt-auto flex flex-col gap-2 p-4", className)}
      {...props}
    /&gt;
  );
}

function SheetTitle({
  className,
  ...props
}: React.ComponentProps&lt;typeof SheetPrimitive.Title&gt;) {
  return (
    &lt;SheetPrimitive.Title
      data-slot="sheet-title"
      className={cn("text-foreground font-semibold", className)}
      {...props}
    /&gt;
  );
}

function SheetDescription({
  className,
  ...props
}: React.ComponentProps&lt;typeof SheetPrimitive.Description&gt;) {
  return (
    &lt;SheetPrimitive.Description
      data-slot="sheet-description"
      className={cn("text-muted-foreground text-sm", className)}
      {...props}
    /&gt;
  );
}

export {
  Sheet,
  SheetTrigger,
  SheetClose,
  SheetContent,
  SheetHeader,
  SheetFooter,
  SheetTitle,
  SheetDescription,
};
```

■ File: src\components\ui\sidebar.tsx

```
"use client";

import * as React from "react";
import { Slot } from "@radix-ui/react-slot@1.1.2";
import { VariantProps, cva } from "class-variance-authority@0.7.1";
import { PanelLeftIcon } from "lucide-react@0.487.0";

import { useIsMobile } from "./use-mobile";
import { cn } from "./utils";
import { Button } from "./button";
import { Input } from "./input";
import { Separator } from "./separator";
import {
  Sheet,
  SheetContent,
  SheetDescription,
  SheetHeader,
  SheetTitle,
} from "./sheet";
import { Skeleton } from "./skeleton";
import {
  Tooltip,
  TooltipContent,
  TooltipProvider,
  TooltipTrigger,
} from "./tooltip";

const SIDEBAR_COOKIE_NAME = "sidebar_state";
const SIDEBAR_COOKIE_MAX_AGE = 60 * 60 * 24 * 7;
const SIDEBAR_WIDTH = "16rem";
const SIDEBAR_WIDTH_MOBILE = "18rem";
const SIDEBAR_WIDTH_ICON = "3rem";
const SIDEBAR_KEYBOARD_SHORTCUT = "b";

type SidebarContextProps = {
  state: "expanded" | "collapsed";
  open: boolean;
  setOpen: (open: boolean) =&gt; void;
  openMobile: boolean;
  setOpenMobile: (open: boolean) =&gt; void;
  isMobile: boolean;
  toggleSidebar: () =&gt; void;
};

const SidebarContext = React.createContext&lt;SidebarContextProps | null&gt;(null);

function useSidebar() {
  const context = React.useContext(SidebarContext);
  if (!context) {
    throw new Error("useSidebar must be used within a SidebarProvider.");
  }

  return context;
}

function SidebarProvider({
  defaultOpen = true,
  open: openProp,
  onOpenChange: setOpenProp,
  className,
  style,
  children,
  ...props
}: React.ComponentProps&lt;"div"&gt; &amp;amp; {
  defaultOpen?: boolean;
  open?: boolean;
  onOpenChange?: (open: boolean) =&gt; void;
}) {
  const isMobile = useIsMobile();
  const [openMobile, setOpenMobile] = React.useState(false);

  // This is the internal state of the sidebar.
  // We use openProp and setOpenProp for control from outside the component.
  const [_open, _setOpen] = React.useState(defaultOpen);
  const open = openProp ?? _open;
  const setOpen = React.useCallback(
    (value: boolean | ((value: boolean) =&gt; boolean)) =&gt; {
      const openState = typeof value === "function" ? value(open) : value;
      if (setOpenProp) {
```

```
        setOpenProp(openState);
      } else {
        _setOpen(openState);
      }

      // This sets the cookie to keep the sidebar state.
      document.cookie = `${SIDEBAR_COOKIE_NAME}=${openState}; path=/;
     max-age=${SIDEBAR_COOKIE_MAX_AGE}`;
    },
    [setOpenProp, open],
  );

  // Helper to toggle the sidebar.
  const toggleSidebar = React.useCallback(() =&gt; {
    return isMobile ? setOpenMobile((open) =&gt; !open) : setOpen((open) =&gt; !open);
  }, [isMobile, setOpen, setOpenMobile]);

  // Adds a keyboard shortcut to toggle the sidebar.
  React.useEffect(() =&gt; {
    const handleKeyDown = (event: KeyboardEvent) =&gt; {
      if (
        event.key === SIDEBAR_KEYBOARD_SHORTCUT &amp;&amp;
        (event.metaKey || event.ctrlKey)
      ) {
        event.preventDefault();
        toggleSidebar();
      }
    };

    window.addEventListener("keydown", handleKeyDown);
    return () =&gt; window.removeEventListener("keydown", handleKeyDown);
  }, [toggleSidebar]);

  // We add a state so that we can do data-state="expanded" or "collapsed".
  // This makes it easier to style the sidebar with Tailwind classes.
  const state = open ? "expanded" : "collapsed";

  const contextValue = React.useMemo&lt;SidebarContextProps&gt;(
    () =&gt; ({
      state,
      open,
      setOpen,
      isMobile,
      openMobile,
      setOpenMobile,
      toggleSidebar,
    }),
    [state, open, setOpen, isMobile, openMobile, setOpenMobile, toggleSidebar],
  );

  return (
    &lt;SidebarContext.Provider value={contextValue}&gt;
      &lt;TooltipProvider delayDuration={0}&gt;
        &lt;div
          data-slot="sidebar-wrapper"
          style={
            {
              "--sidebar-width": SIDEBAR_WIDTH,
              "--sidebar-width-icon": SIDEBAR_WIDTH_ICON,
              ...style,
            } as React.CSSProperties
          }
          className={cn(
            "group/sidebar-wrapper has-data-[variant=inset]:bg-sidebar flex min-h-svh
     w-full",
            className,
          )}
          {...props}
        &gt;
          {children}
        &lt;/div&gt;
      &lt;/TooltipProvider&gt;
    &lt;/SidebarContext.Provider&gt;
  );
}

function Sidebar({
  side = "left",
  variant = "sidebar",
  collapsible = "offcanvas",
  className,
  children,
```

```
  ...props
}: React.ComponentProps&lt;"div"&gt; &amp; {
  side?: "left" | "right";
  variant?: "sidebar" | "floating" | "inset";
  collapsible?: "offcanvas" | "icon" | "none";
}) {
  const { isMobile, state, openMobile, setOpenMobile } = useSidebar();

  if (collapsible === "none") {
    return (
      &lt;div
        data-slot="sidebar"
        className={cn(
          "bg-sidebar text-sidebar-foreground flex h-full w-(--sidebar-width) flex-col",
          className,
        )}
        {...props}
      &gt;
        {children}
      &lt;/div&gt;
    );
  }

  if (isMobile) {
    return (
      &lt;Sheet open={openMobile} onOpenChange={setOpenMobile} {...props}&gt;
        &lt;SheetContent
          data-sidebar="sidebar"
          data-slot="sidebar"
          data-mobile="true"
          className="bg-sidebar text-sidebar-foreground w-(--sidebar-width) p-0
     [&amp;&gt;button]:hidden"
          style={
            {
              "--sidebar-width": SIDEBAR_WIDTH_MOBILE,
            } as React.CSSProperties
          }
          side={side}
        &gt;
          &lt;SheetHeader className="sr-only"&gt;
            &lt;SheetTitle&gt;Sidebar&lt;/SheetTitle&gt;
            &lt;SheetDescription&gt;Displays the mobile sidebar.&lt;/SheetDescription&gt;
          &lt;/SheetHeader&gt;
          &lt;div className="flex h-full w-full flex-col"&gt;{children}&lt;/div&gt;
        &lt;/SheetContent&gt;
      &lt;/Sheet&gt;
    );
  }

  return (
    &lt;div
      className="group peer text-sidebar-foreground hidden md:block"
      data-state={state}
      data-collapsible={state === "collapsed" ? collapsible : ""}
      data-variant={variant}
      data-side={side}
      data-slot="sidebar"
    &gt;
      {/* This is what handles the sidebar gap on desktop */}
      &lt;div
        data-slot="sidebar-gap"
        className={cn(
          "relative w-(--sidebar-width) bg-transparent transition-[width] duration-200
     ease-linear",
          "group-data-[collapsible=offcanvas]:w-0",
          "group-data-[side=right]:rotate-180",
          variant === "floating" || variant === "inset"
            ? "group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)+(--spacing(
    4)))]"
            : "group-data-[collapsible=icon]:w-(--sidebar-width-icon)",
        )}
      /&gt;
      &lt;div
        data-slot="sidebar-container"
        className={cn(
          "fixed inset-y-0 z-10 hidden h-svh w-(--sidebar-width)
     transition-[left,right,width] duration-200 ease-linear md:flex",
          side === "left"
            ? "left-0 group-data-[collapsible=offcanvas]:left-[calc(var(--sidebar-width)*-
    1)]"
            : "right-0 group-data-[collapsible=offcanvas]:right-[calc(var(--sidebar-width)
    *-1)]",
```

```
          // Adjust the padding for floating and inset variants.
          variant === "floating" || variant === "inset"
            ? "p-2 group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)+(--spac
    ing(4))+2px)]"
            : "group-data-[collapsible=icon]:w-(--sidebar-width-icon)
     group-data-[side=left]:border-r group-data-[side=right]:border-l",
          className,
        )}
        {...props}
      &gt;
        &lt;div
          data-sidebar="sidebar"
          data-slot="sidebar-inner"
          className="bg-sidebar group-data-[variant=floating]:border-sidebar-border flex
     h-full w-full flex-col group-data-[variant=floating]:rounded-lg
     group-data-[variant=floating]:border group-data-[variant=floating]:shadow-sm"
        &gt;
          {children}
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}

function SidebarTrigger({
  className,
  onClick,
  ...props
}: React.ComponentProps&lt;typeof Button&gt;) {
  const { toggleSidebar } = useSidebar();

  return (
    &lt;Button
      data-sidebar="trigger"
      data-slot="sidebar-trigger"
      variant="ghost"
      size="icon"
      className={cn("size-7", className)}
      onClick={(event) =&gt; {
        onClick?.(event);
        toggleSidebar();
      }}
      {...props}
    &gt;
      &lt;PanelLeftIcon /&gt;
      &lt;span className="sr-only"&gt;Toggle Sidebar&lt;/span&gt;
    &lt;/Button&gt;
  );
}

function SidebarRail({ className, ...props }: React.ComponentProps&lt;"button"&gt;) {
  const { toggleSidebar } = useSidebar();

  return (
    &lt;button
      data-sidebar="rail"
      data-slot="sidebar-rail"
      aria-label="Toggle Sidebar"
      tabIndex={-1}
      onClick={toggleSidebar}
      title="Toggle Sidebar"
      className={cn(
        "hover:after:bg-sidebar-border absolute inset-y-0 z-20 hidden w-4
     -translate-x-1/2 transition-all ease-linear group-data-[side=left]:-right-4
     group-data-[side=right]:left-0 after:absolute after:inset-y-0 after:left-1/2
     after:w-[2px] sm:flex",
        "in-data-[side=left]:cursor-w-resize in-data-[side=right]:cursor-e-resize",
        "[[data-side=left][data-state=collapsed]_&amp;]:cursor-e-resize
     [[data-side=right][data-state=collapsed]_&amp;]:cursor-w-resize",
        "hover:group-data-[collapsible=offcanvas]:bg-sidebar
     group-data-[collapsible=offcanvas]:translate-x-0
     group-data-[collapsible=offcanvas]:after:left-full",
        "[[data-side=left][data-collapsible=offcanvas]_&amp;]:-right-2",
        "[[data-side=right][data-collapsible=offcanvas]_&amp;]:-left-2",
        className,
      )}
      {...props}
    /&gt;
  );
}

function SidebarInset({ className, ...props }: React.ComponentProps&lt;"main"&gt;) {
```

```
  return (
    &lt;main
      data-slot="sidebar-inset"
      className={cn(
        "bg-background relative flex w-full flex-1 flex-col",
        "md:peer-data-[variant=inset]:m-2 md:peer-data-[variant=inset]:ml-0
     md:peer-data-[variant=inset]:rounded-xl md:peer-data-[variant=inset]:shadow-sm
     md:peer-data-[variant=inset]:peer-data-[state=collapsed]:ml-2",
        className,
      )}
      {...props}
    /&gt;
  );
}

function SidebarInput({
  className,
  ...props
}: React.ComponentProps&lt;typeof Input&gt;) {
  return (
    &lt;Input
      data-slot="sidebar-input"
      data-sidebar="input"
      className={cn("bg-background h-8 w-full shadow-none", className)}
      {...props}
    /&gt;
  );
}

function SidebarHeader({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="sidebar-header"
      data-sidebar="header"
      className={cn("flex flex-col gap-2 p-2", className)}
      {...props}
    /&gt;
  );
}

function SidebarFooter({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="sidebar-footer"
      data-sidebar="footer"
      className={cn("flex flex-col gap-2 p-2", className)}
      {...props}
    /&gt;
  );
}

function SidebarSeparator({
  className,
  ...props
}: React.ComponentProps&lt;typeof Separator&gt;) {
  return (
    &lt;Separator
      data-slot="sidebar-separator"
      data-sidebar="separator"
      className={cn("bg-sidebar-border mx-2 w-auto", className)}
      {...props}
    /&gt;
  );
}

function SidebarContent({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="sidebar-content"
      data-sidebar="content"
      className={cn(
        "flex min-h-0 flex-1 flex-col gap-2 overflow-auto
    group-data-[collapsible=icon]:overflow-hidden",
        className,
      )}
      {...props}
    /&gt;
  );
}

function SidebarGroup({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
```

```
    &lt;div
      data-slot="sidebar-group"
      data-sidebar="group"
      className={cn("relative flex w-full min-w-0 flex-col p-2", className)}
      {...props}
    /&gt;
  );
}

function SidebarGroupLabel({
  className,
  asChild = false,
  ...props
}: React.ComponentProps&lt;"div"&gt; &amp; { asChild?: boolean }) {
  const Comp = asChild ? Slot : "div";

  return (
    &lt;Comp
      data-slot="sidebar-group-label"
      data-sidebar="group-label"
      className={cn(
        "text-sidebar-foreground/70 ring-sidebar-ring flex h-8 shrink-0 items-center
    rounded-md px-2 text-xs font-medium outline-hidden transition-[margin,opacity]
    duration-200 ease-linear focus-visible:ring-2 [&amp;&gt;svg]:size-4 [&amp;&gt;svg]:shrink-0",
        "group-data-[collapsible=icon]:-mt-8 group-data-[collapsible=icon]:opacity-0",
        className,
      )}
      {...props}
    /&gt;
  );
}

function SidebarGroupAction({
  className,
  asChild = false,
  ...props
}: React.ComponentProps&lt;"button"&gt; &amp; { asChild?: boolean }) {
  const Comp = asChild ? Slot : "button";

  return (
    &lt;Comp
      data-slot="sidebar-group-action"
      data-sidebar="group-action"
      className={cn(
        "text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent
    hover:text-sidebar-accent-foreground absolute top-3.5 right-3 flex aspect-square w-5
    items-center justify-center rounded-md p-0 outline-hidden transition-transform
    focus-visible:ring-2 [&amp;&gt;svg]:size-4 [&amp;&gt;svg]:shrink-0",
        // Increases the hit area of the button on mobile.
        "after:absolute after:-inset-2 md:after:hidden",
        "group-data-[collapsible=icon]:hidden",
        className,
      )}
      {...props}
    /&gt;
  );
}

function SidebarGroupContent({
  className,
  ...props
}: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="sidebar-group-content"
      data-sidebar="group-content"
      className={cn("w-full text-sm", className)}
      {...props}
    /&gt;
  );
}

function SidebarMenu({ className, ...props }: React.ComponentProps&lt;"ul"&gt;) {
  return (
    &lt;ul
      data-slot="sidebar-menu"
      data-sidebar="menu"
      className={cn("flex w-full min-w-0 flex-col gap-1", className)}
      {...props}
    /&gt;
  );
}
```

```
function SidebarMenuItem({ className, ...props }: React.ComponentProps&lt;"li"&gt;) {
  return (
    &lt;li
      data-slot="sidebar-menu-item"
      data-sidebar="menu-item"
      className={cn("group/menu-item relative", className)}
      {...props}
    /&gt;
  );
}

const sidebarMenuButtonVariants = cva(
  "peer/menu-button flex w-full items-center gap-2 overflow-hidden rounded-md p-2
     text-left text-sm outline-hidden ring-sidebar-ring transition-[width,height,padding]
     hover:bg-sidebar-accent hover:text-sidebar-accent-foreground focus-visible:ring-2
     active:bg-sidebar-accent active:text-sidebar-accent-foreground
     disabled:pointer-events-none disabled:opacity-50
     group-has-data-[sidebar=menu-action]/menu-item:pr-8
     aria-disabled:pointer-events-none aria-disabled:opacity-50
     data-[active=true]:bg-sidebar-accent data-[active=true]:font-medium
     data-[active=true]:text-sidebar-accent-foreground
     data-[state=open]:hover:bg-sidebar-accent
     data-[state=open]:hover:text-sidebar-accent-foreground
     group-data-[collapsible=icon]:size-8! group-data-[collapsible=icon]:p-2!
     [&amp;&gt;span:last-child]:truncate [&amp;&gt;svg]:size-4 [&amp;&gt;svg]:shrink-0",
  {
    variants: {
      variant: {
        default: "hover:bg-sidebar-accent hover:text-sidebar-accent-foreground",
        outline:
          "bg-background shadow-[0_0_0_1px_hsl(var(--sidebar-border))]
     hover:bg-sidebar-accent hover:text-sidebar-accent-foreground
     hover:shadow-[0_0_0_1px_hsl(var(--sidebar-accent))]",
      },
      size: {
        default: "h-8 text-sm",
        sm: "h-7 text-xs",
        lg: "h-12 text-sm group-data-[collapsible=icon]:p-0!",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
);

function SidebarMenuButton({
  asChild = false,
  isActive = false,
  variant = "default",
  size = "default",
  tooltip,
  className,
  ...props
}: React.ComponentProps&lt;"button"&gt; &amp; {
  asChild?: boolean;
  isActive?: boolean;
  tooltip?: string | React.ComponentProps&lt;typeof TooltipContent&gt;;
} &amp; VariantProps&lt;typeof sidebarMenuButtonVariants&gt;) {
  const Comp = asChild ? Slot : "button";
  const { isMobile, state } = useSidebar();

  const button = (
    &lt;Comp
      data-slot="sidebar-menu-button"
      data-sidebar="menu-button"
      data-size={size}
      data-active={isActive}
      className={cn(sidebarMenuButtonVariants({ variant, size }), className)}
      {...props}
    /&gt;
  );

  if (!tooltip) {
    return button;
  }

  if (typeof tooltip === "string") {
    tooltip = {
      children: tooltip,
    };
```

```
  }

  return (
    &lt;Tooltip&gt;
      &lt;TooltipTrigger asChild&gt;{button}&lt;/TooltipTrigger&gt;
      &lt;TooltipContent
        side="right"
        align="center"
        hidden={state !== "collapsed" || isMobile}
        {...tooltip}
      /&gt;
    &lt;/Tooltip&gt;
  );
}

function SidebarMenuAction({
  className,
  asChild = false,
  showOnHover = false,
  ...props
}: React.ComponentProps&lt;"button"&gt; &amp; {
  asChild?: boolean;
  showOnHover?: boolean;
}) {
  const Comp = asChild ? Slot : "button";

  return (
    &lt;Comp
      data-slot="sidebar-menu-action"
      data-sidebar="menu-action"
      className={cn(
        "text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent
     hover:text-sidebar-accent-foreground peer-hover/menu-button:text-sidebar-accent-foreg
    round absolute top-1.5 right-1 flex aspect-square w-5 items-center justify-center
     rounded-md p-0 outline-hidden transition-transform focus-visible:ring-2
     [&amp;&gt;svg]:size-4 [&amp;&gt;svg]:shrink-0",
        // Increases the hit area of the button on mobile.
        "after:absolute after:-inset-2 md:after:hidden",
        "peer-data-[size=sm]/menu-button:top-1",
        "peer-data-[size=default]/menu-button:top-1.5",
        "peer-data-[size=lg]/menu-button:top-2.5",
        "group-data-[collapsible=icon]:hidden",
        showOnHover &amp;&amp;
          "peer-data-[active=true]/menu-button:text-sidebar-accent-foreground
     group-focus-within/menu-item:opacity-100 group-hover/menu-item:opacity-100
     data-[state=open]:opacity-100 md:opacity-0",
        className,
      )}
      {...props}
    /&gt;
  );
}

function SidebarMenuBadge({
  className,
  ...props
}: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="sidebar-menu-badge"
      data-sidebar="menu-badge"
      className={cn(
        "text-sidebar-foreground pointer-events-none absolute right-1 flex h-5 min-w-5
     items-center justify-center rounded-md px-1 text-xs font-medium tabular-nums
     select-none",
        "peer-hover/menu-button:text-sidebar-accent-foreground
     peer-data-[active=true]/menu-button:text-sidebar-accent-foreground",
        "peer-data-[size=sm]/menu-button:top-1",
        "peer-data-[size=default]/menu-button:top-1.5",
        "peer-data-[size=lg]/menu-button:top-2.5",
        "group-data-[collapsible=icon]:hidden",
        className,
      )}
      {...props}
    /&gt;
  );
}

function SidebarMenuSkeleton({
  className,
  showIcon = false,
  ...props
```

```
}: React.ComponentProps&lt;"div"&gt; &amp; {
  showIcon?: boolean;
}) {
  // Random width between 50 to 90%.
  const width = React.useMemo(() =&gt; {
    return `${Math.floor(Math.random() * 40) + 50}%`;
  }, []);

  return (
    &lt;div
      data-slot="sidebar-menu-skeleton"
      data-sidebar="menu-skeleton"
      className={cn("flex h-8 items-center gap-2 rounded-md px-2", className)}
      {...props}
    &gt;
      {showIcon &amp;&amp; (
        &lt;Skeleton
          className="size-4 rounded-md"
          data-sidebar="menu-skeleton-icon"
        /&gt;
      )}
      &lt;Skeleton
        className="h-4 max-w-(--skeleton-width) flex-1"
        data-sidebar="menu-skeleton-text"
        style={
          {
            "--skeleton-width": width,
          } as React.CSSProperties
        }
      /&gt;
    &lt;/div&gt;
  );
}

function SidebarMenuSub({ className, ...props }: React.ComponentProps&lt;"ul"&gt;) {
  return (
    &lt;ul
      data-slot="sidebar-menu-sub"
      data-sidebar="menu-sub"
      className={cn(
        "border-sidebar-border mx-3.5 flex min-w-0 translate-x-px flex-col gap-1 border-l
     px-2.5 py-0.5",
        "group-data-[collapsible=icon]:hidden",
        className,
      )}
      {...props}
    /&gt;
  );
}

function SidebarMenuSubItem({
  className,
  ...props
}: React.ComponentProps&lt;"li"&gt;) {
  return (
    &lt;li
      data-slot="sidebar-menu-sub-item"
      data-sidebar="menu-sub-item"
      className={cn("group/menu-sub-item relative", className)}
      {...props}
    /&gt;
  );
}

function SidebarMenuSubButton({
  asChild = false,
  size = "md",
  isActive = false,
  className,
  ...props
}: React.ComponentProps&lt;"a"&gt; &amp; {
  asChild?: boolean;
  size?: "sm" | "md";
  isActive?: boolean;
}) {
  const Comp = asChild ? Slot : "a";

  return (
    &lt;Comp
      data-slot="sidebar-menu-sub-button"
      data-sidebar="menu-sub-button"
      data-size={size}
```

```
      data-active={isActive}
      className={cn(
        "text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent
     hover:text-sidebar-accent-foreground active:bg-sidebar-accent
     active:text-sidebar-accent-foreground [&amp;&gt;svg]:text-sidebar-accent-foreground flex
     h-7 min-w-0 -translate-x-px items-center gap-2 overflow-hidden rounded-md px-2
     outline-hidden focus-visible:ring-2 disabled:pointer-events-none disabled:opacity-50
     aria-disabled:pointer-events-none aria-disabled:opacity-50
     [&amp;&gt;span:last-child]:truncate [&amp;&gt;svg]:size-4 [&amp;&gt;svg]:shrink-0",
        "data-[active=true]:bg-sidebar-accent
     data-[active=true]:text-sidebar-accent-foreground",
        size === "sm" &amp;&amp; "text-xs",
        size === "md" &amp;&amp; "text-sm",
        "group-data-[collapsible=icon]:hidden",
        className,
      )}
      {...props}
    /&gt;
  );
}

export {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarGroup,
  SidebarGroupAction,
  SidebarGroupContent,
  SidebarGroupLabel,
  SidebarHeader,
  SidebarInput,
  SidebarInset,
  SidebarMenu,
  SidebarMenuAction,
  SidebarMenuBadge,
  SidebarMenuButton,
  SidebarMenuItem,
  SidebarMenuSkeleton,
  SidebarMenuSub,
  SidebarMenuSubButton,
  SidebarMenuSubItem,
  SidebarProvider,
  SidebarRail,
  SidebarSeparator,
  SidebarTrigger,
  useSidebar,
};
```

■ File: src\components\ui\skeleton.tsx

```
import { cn } from "./utils";

function Skeleton({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="skeleton"
      className={cn("bg-accent animate-pulse rounded-md", className)}
      {...props}
    /&gt;
  );
}

export { Skeleton };
```

■ File: src\components\ui\slider.tsx

```
"use client";

import * as React from "react";
import * as SliderPrimitive from "@radix-ui/react-slider@1.2.3";

import { cn } from "./utils";

function Slider({
  className,
  defaultValue,
  value,
  min = 0,
  max = 100,
  ...props
}: React.ComponentProps&lt;typeof SliderPrimitive.Root&gt;) {
  const _values = React.useMemo(
    () =&gt;
      Array.isArray(value)
        ? value
        : Array.isArray(defaultValue)
          ? defaultValue
          : [min, max],
    [value, defaultValue, min, max],
  );

  return (
    &lt;SliderPrimitive.Root
      data-slot="slider"
      defaultValue={defaultValue}
      value={value}
      min={min}
      max={max}
      className={cn(
        "relative flex w-full touch-none items-center select-none
     data-[disabled]:opacity-50 data-[orientation=vertical]:h-full
     data-[orientation=vertical]:min-h-44 data-[orientation=vertical]:w-auto
     data-[orientation=vertical]:flex-col",
        className,
      )}
      {...props}
    &gt;
      &lt;SliderPrimitive.Track
        data-slot="slider-track"
        className={cn(
          "bg-muted relative grow overflow-hidden rounded-full
     data-[orientation=horizontal]:h-4 data-[orientation=horizontal]:w-full
     data-[orientation=vertical]:h-full data-[orientation=vertical]:w-1.5",
        )}
      &gt;
        &lt;SliderPrimitive.Range
          data-slot="slider-range"
          className={cn(
            "bg-primary absolute data-[orientation=horizontal]:h-full
     data-[orientation=vertical]:w-full",
          )}
        /&gt;
      &lt;/SliderPrimitive.Track&gt;
      {Array.from({ length: _values.length }, (_, index) =&gt; (
        &lt;SliderPrimitive.Thumb
          data-slot="slider-thumb"
          key={index}
          className="border-primary bg-background ring-ring/50 block size-4 shrink-0
     rounded-full border shadow-sm transition-[color,box-shadow] hover:ring-4
     focus-visible:ring-4 focus-visible:outline-hidden disabled:pointer-events-none
     disabled:opacity-50"
        /&gt;
      ))}
    &lt;/SliderPrimitive.Root&gt;
  );
}

export { Slider };
```

■ File: src\components\ui\sonner.tsx

```
"use client";

import { useTheme } from "next-themes@0.4.6";
import { Toaster as Sonner, ToasterProps } from "sonner@2.0.3";

const Toaster = ({ ...props }: ToasterProps) =&gt; {
  const { theme = "system" } = useTheme();

  return (
    &lt;Sonner
      theme={theme as ToasterProps["theme"]}
      className="toaster group"
      style={
        {
          "--normal-bg": "var(--popover)",
          "--normal-text": "var(--popover-foreground)",
          "--normal-border": "var(--border)",
        } as React.CSSProperties
      }
      {...props}
    /&gt;
  );
};

export { Toaster };
```

■ File: src\components\ui\switch.tsx

```
"use client";

import * as React from "react";
import * as SwitchPrimitive from "@radix-ui/react-switch@1.1.3";

import { cn } from "./utils";

function Switch({
  className,
  ...props
}: React.ComponentProps&lt;typeof SwitchPrimitive.Root&gt;) {
  return (
    &lt;SwitchPrimitive.Root
      data-slot="switch"
      className={cn(
        "peer data-[state=checked]:bg-primary data-[state=unchecked]:bg-switch-background
     focus-visible:border-ring focus-visible:ring-ring/50
     dark:data-[state=unchecked]:bg-input/80 inline-flex h-[1.15rem] w-8 shrink-0
     items-center rounded-full border border-transparent transition-all outline-none
     focus-visible:ring-[3px] disabled:cursor-not-allowed disabled:opacity-50",
        className,
      )}
      {...props}
    &gt;
      &lt;SwitchPrimitive.Thumb
        data-slot="switch-thumb"
        className={cn(
          "bg-card dark:data-[state=unchecked]:bg-card-foreground
     dark:data-[state=checked]:bg-primary-foreground pointer-events-none block size-4
     rounded-full ring-0 transition-transform
     data-[state=checked]:translate-x-[calc(100%-2px)]
     data-[state=unchecked]:translate-x-0",
        )}
      /&gt;
    &lt;/SwitchPrimitive.Root&gt;
  );
}

export { Switch };
```

■ File: src\components\ui\table.tsx

```
"use client";

import * as React from "react";

import { cn } from "./utils";

function Table({ className, ...props }: React.ComponentProps&lt;"table"&gt;) {
  return (
    &lt;div
      data-slot="table-container"
      className="relative w-full overflow-x-auto"
    &gt;
      &lt;table
        data-slot="table"
        className={cn("w-full caption-bottom text-sm", className)}
        {...props}
      /&gt;
    &lt;/div&gt;
  );
}

function TableHeader({ className, ...props }: React.ComponentProps&lt;"thead"&gt;) {
  return (
    &lt;thead
      data-slot="table-header"
      className={cn("[&amp;_tr]:border-b", className)}
      {...props}
    /&gt;
  );
}

function TableBody({ className, ...props }: React.ComponentProps&lt;"tbody"&gt;) {
  return (
    &lt;tbody
      data-slot="table-body"
      className={cn("[&amp;_tr:last-child]:border-0", className)}
      {...props}
    /&gt;
  );
}

function TableFooter({ className, ...props }: React.ComponentProps&lt;"tfoot"&gt;) {
  return (
    &lt;tfoot
      data-slot="table-footer"
      className={cn(
        "bg-muted/50 border-t font-medium [&amp;&gt;tr]:last:border-b-0",
        className,
      )}
      {...props}
    /&gt;
  );
}

function TableRow({ className, ...props }: React.ComponentProps&lt;"tr"&gt;) {
  return (
    &lt;tr
      data-slot="table-row"
      className={cn(
        "hover:bg-muted/50 data-[state=selected]:bg-muted border-b transition-colors",
        className,
      )}
      {...props}
    /&gt;
  );
}

function TableHead({ className, ...props }: React.ComponentProps&lt;"th"&gt;) {
  return (
    &lt;th
      data-slot="table-head"
      className={cn(
        "text-foreground h-10 px-2 text-left align-middle font-medium whitespace-nowrap
    [&amp;:has([role=checkbox])]:pr-0 [&amp;&gt;[role=checkbox]]:translate-y-[2px]",
        className,
      )}
      {...props}
    /&gt;
  );
```

```
}

function TableCell({ className, ...props }: React.ComponentProps&lt;"td"&gt;) {
  return (
    &lt;td
      data-slot="table-cell"
      className={cn(
        "p-2 align-middle whitespace-nowrap [&amp;:has([role=checkbox])]:pr-0
     [&amp;&gt;[role=checkbox]]:translate-y-[2px]",
        className,
      )}
      {...props}
    /&gt;
  );
}

function TableCaption({
  className,
  ...props
}: React.ComponentProps&lt;"caption"&gt;) {
  return (
    &lt;caption
      data-slot="table-caption"
      className={cn("text-muted-foreground mt-4 text-sm", className)}
      {...props}
    /&gt;
  );
}

export {
  Table,
  TableHeader,
  TableBody,
  TableFooter,
  TableHead,
  TableRow,
  TableCell,
  TableCaption,
};
```

■ File: src\components\ui\tabs.tsx

```
"use client";

import * as React from "react";
import * as TabsPrimitive from "@radix-ui/react-tabs@1.1.3";

import { cn } from "./utils";

function Tabs({
  className,
  ...props
}: React.ComponentProps&lt;typeof TabsPrimitive.Root&gt;) {
  return (
    &lt;TabsPrimitive.Root
      data-slot="tabs"
      className={cn("flex flex-col gap-2", className)}
      {...props}
    /&gt;
  );
}

function TabsList({
  className,
  ...props
}: React.ComponentProps&lt;typeof TabsPrimitive.List&gt;) {
  return (
    &lt;TabsPrimitive.List
      data-slot="tabs-list"
      className={cn(
        "bg-muted text-muted-foreground inline-flex h-9 w-fit items-center justify-center
     rounded-xl p-[3px] flex",
        className,
      )}
      {...props}
    /&gt;
  );
}

function TabsTrigger({
  className,
  ...props
}: React.ComponentProps&lt;typeof TabsPrimitive.Trigger&gt;) {
  return (
    &lt;TabsPrimitive.Trigger
      data-slot="tabs-trigger"
      className={cn(
        "data-[state=active]:bg-card dark:data-[state=active]:text-foreground
     focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:outline-ring
     dark:data-[state=active]:border-input dark:data-[state=active]:bg-input/30
     text-foreground dark:text-muted-foreground inline-flex h-[calc(100%-1px)] flex-1
     items-center justify-center gap-1.5 rounded-xl border border-transparent px-2 py-1
     text-sm font-medium whitespace-nowrap transition-[color,box-shadow]
     focus-visible:ring-[3px] focus-visible:outline-1 disabled:pointer-events-none
     disabled:opacity-50 [&amp;_svg]:pointer-events-none [&amp;_svg]:shrink-0
     [&amp;_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    /&gt;
  );
}

function TabsContent({
  className,
  ...props
}: React.ComponentProps&lt;typeof TabsPrimitive.Content&gt;) {
  return (
    &lt;TabsPrimitive.Content
      data-slot="tabs-content"
      className={cn("flex-1 outline-none", className)}
      {...props}
    /&gt;
  );
}

export { Tabs, TabsList, TabsTrigger, TabsContent };
```

■ File: src\components\ui\textarea.tsx

```
import * as React from "react";

import { cn } from "./utils";

function Textarea({ className, ...props }: React.ComponentProps&lt;"textarea"&gt;) {
  return (
    &lt;textarea
      data-slot="textarea"
      className={cn(
        "resize-none border-input placeholder:text-muted-foreground
     focus-visible:border-ring focus-visible:ring-ring/50
     aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40
     aria-invalid:border-destructive dark:bg-input/30 flex field-sizing-content min-h-16
     w-full rounded-md border bg-input-background px-3 py-2 text-base
     transition-[color,box-shadow] outline-none focus-visible:ring-[3px]
     disabled:cursor-not-allowed disabled:opacity-50 md:text-sm",
        className,
      )}
      {...props}
    /&gt;
  );
}

export { Textarea };
```

■ File: src\components\ui\toggle-group.tsx

```
"use client";

import * as React from "react";
import * as ToggleGroupPrimitive from "@radix-ui/react-toggle-group@1.1.2";
import { type VariantProps } from "class-variance-authority@0.7.1";

import { cn } from "./utils";
import { toggleVariants } from "./toggle";

const ToggleGroupContext = React.createContext&lt;
  VariantProps&lt;typeof toggleVariants&gt;
&gt;({
  size: "default",
  variant: "default",
});

function ToggleGroup({
  className,
  variant,
  size,
  children,
  ...props
}: React.ComponentProps&lt;typeof ToggleGroupPrimitive.Root&gt; &amp;
  VariantProps&lt;typeof toggleVariants&gt;) {
  return (
    &lt;ToggleGroupPrimitive.Root
      data-slot="toggle-group"
      data-variant={variant}
      data-size={size}
      className={cn(
        "group/toggle-group flex w-fit items-center rounded-md
     data-[variant=outline]:shadow-xs",
        className,
      )}
      {...props}
    &gt;
      &lt;ToggleGroupContext.Provider value={{ variant, size }}&gt;
        {children}
      &lt;/ToggleGroupContext.Provider&gt;
    &lt;/ToggleGroupPrimitive.Root&gt;
  );
}

function ToggleGroupItem({
  className,
  children,
  variant,
  size,
  ...props
}: React.ComponentProps&lt;typeof ToggleGroupPrimitive.Item&gt; &amp;
  VariantProps&lt;typeof toggleVariants&gt;) {
  const context = React.useContext(ToggleGroupContext);

  return (
    &lt;ToggleGroupPrimitive.Item
      data-slot="toggle-group-item"
      data-variant={context.variant || variant}
      data-size={context.size || size}
      className={cn(
        toggleVariants({
          variant: context.variant || variant,
          size: context.size || size,
        }),
        "min-w-0 flex-1 shrink-0 rounded-none shadow-none first:rounded-l-md
     last:rounded-r-md focus:z-10 focus-visible:z-10 data-[variant=outline]:border-l-0
     data-[variant=outline]:first:border-l",
        className,
      )}
      {...props}
    &gt;
      {children}
    &lt;/ToggleGroupPrimitive.Item&gt;
  );
}

export { ToggleGroup, ToggleGroupItem };
```

■ File: src\components\ui\toggle.tsx

```
"use client";

import * as React from "react";
import * as TogglePrimitive from "@radix-ui/react-toggle@1.1.2";
import { cva, type VariantProps } from "class-variance-authority@0.7.1";

import { cn } from "./utils";

const toggleVariants = cva(
  "inline-flex items-center justify-center gap-2 rounded-md text-sm font-medium
     hover:bg-muted hover:text-muted-foreground disabled:pointer-events-none
     disabled:opacity-50 data-[state=on]:bg-accent data-[state=on]:text-accent-foreground
     [&amp;_svg]:pointer-events-none [&amp;_svg:not([class*='size-'])]:size-4 [&amp;_svg]:shrink-0
     focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-[3px]
     outline-none transition-[color,box-shadow] aria-invalid:ring-destructive/20
     dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructive
     whitespace-nowrap",
  {
    variants: {
      variant: {
        default: "bg-transparent",
        outline:
          "border border-input bg-transparent hover:bg-accent
    hover:text-accent-foreground",
      },
      size: {
        default: "h-9 px-2 min-w-9",
        sm: "h-8 px-1.5 min-w-8",
        lg: "h-10 px-2.5 min-w-10",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
);

function Toggle({
  className,
  variant,
  size,
  ...props
}: React.ComponentProps&lt;typeof TogglePrimitive.Root&gt; &amp;
  VariantProps&lt;typeof toggleVariants&gt;) {
  return (
    &lt;TogglePrimitive.Root
      data-slot="toggle"
      className={cn(toggleVariants({ variant, size, className }))}
      {...props}
    /&gt;
  );
}

export { Toggle, toggleVariants };
```

■ File: src\components\ui\tooltip.tsx

```
"use client";

import * as React from "react";
import * as TooltipPrimitive from "@radix-ui/react-tooltip@1.1.8";

import { cn } from "./utils";

function TooltipProvider({
  delayDuration = 0,
  ...props
}: React.ComponentProps&lt;typeof TooltipPrimitive.Provider&gt;) {
  return (
    &lt;TooltipPrimitive.Provider
      data-slot="tooltip-provider"
      delayDuration={delayDuration}
      {...props}
    /&gt;
  );
}

function Tooltip({
  ...props
}: React.ComponentProps&lt;typeof TooltipPrimitive.Root&gt;) {
  return (
    &lt;TooltipProvider&gt;
      &lt;TooltipPrimitive.Root data-slot="tooltip" {...props} /&gt;
    &lt;/TooltipProvider&gt;
  );
}

function TooltipTrigger({
  ...props
}: React.ComponentProps&lt;typeof TooltipPrimitive.Trigger&gt;) {
  return &lt;TooltipPrimitive.Trigger data-slot="tooltip-trigger" {...props} /&gt;;
}

function TooltipContent({
  className,
  sideOffset = 0,
  children,
  ...props
}: React.ComponentProps&lt;typeof TooltipPrimitive.Content&gt;) {
  return (
    &lt;TooltipPrimitive.Portal&gt;
      &lt;TooltipPrimitive.Content
        data-slot="tooltip-content"
        sideOffset={sideOffset}
        className={cn(
          "bg-primary text-primary-foreground animate-in fade-in-0 zoom-in-95
     data-[state=closed]:animate-out data-[state=closed]:fade-out-0
     data-[state=closed]:zoom-out-95 data-[side=bottom]:slide-in-from-top-2
     data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
     data-[side=top]:slide-in-from-bottom-2 z-50 w-fit
     origin-(--radix-tooltip-content-transform-origin) rounded-md px-3 py-1.5 text-xs
     text-balance",
          className,
        )}
        {...props}
      &gt;
        {children}
        &lt;TooltipPrimitive.Arrow className="bg-primary fill-primary z-50 size-2.5
     translate-y-[calc(-50%_-_2px)] rotate-45 rounded-[2px]" /&gt;
      &lt;/TooltipPrimitive.Content&gt;
    &lt;/TooltipPrimitive.Portal&gt;
  );
}

export { Tooltip, TooltipTrigger, TooltipContent, TooltipProvider };
```

■ File: src\components\ui\use-mobile.ts

```
import * as React from "react";

const MOBILE_BREAKPOINT = 768;

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState&lt;boolean | undefined&gt;(
    undefined,
  );

  React.useEffect(() =&gt; {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`);
    const onChange = () =&gt; {
      setIsMobile(window.innerWidth &lt; MOBILE_BREAKPOINT);
    };
    mql.addEventListener("change", onChange);
    setIsMobile(window.innerWidth &lt; MOBILE_BREAKPOINT);
    return () =&gt; mql.removeEventListener("change", onChange);
  }, []);

  return !!isMobile;
}
```

■ File: src\components\ui\utils.ts

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

■ File: src\index.css

```
/*! tailwindcss v4.1.3 | MIT License | https://tailwindcss.com */
@layer properties {
  @supports (((-webkit-hyphens: none)) and (not (margin-trim: inline))) or ((-moz-orient:
     inline) and (not (color: rgb(from red r g b)))) {
    *, :before, :after, ::backdrop {
      --tw-translate-x: 0;
      --tw-translate-y: 0;
      --tw-translate-z: 0;
      --tw-space-y-reverse: 0;
      --tw-border-style: solid;
      --tw-gradient-position: initial;
      --tw-gradient-from: #0000;
      --tw-gradient-via: #0000;
      --tw-gradient-to: #0000;
      --tw-gradient-stops: initial;
      --tw-gradient-via-stops: initial;
      --tw-gradient-from-position: 0%;
      --tw-gradient-via-position: 50%;
      --tw-gradient-to-position: 100%;
      --tw-shadow: 0 0 #0000;
      --tw-shadow-color: initial;
      --tw-shadow-alpha: 100%;
      --tw-inset-shadow: 0 0 #0000;
      --tw-inset-shadow-color: initial;
      --tw-inset-shadow-alpha: 100%;
      --tw-ring-color: initial;
      --tw-ring-shadow: 0 0 #0000;
      --tw-inset-ring-color: initial;
      --tw-inset-ring-shadow: 0 0 #0000;
      --tw-ring-inset: initial;
      --tw-ring-offset-width: 0px;
      --tw-ring-offset-color: #fff;
      --tw-ring-offset-shadow: 0 0 #0000;
      --tw-blur: initial;
      --tw-brightness: initial;
      --tw-contrast: initial;
      --tw-grayscale: initial;
      --tw-hue-rotate: initial;
      --tw-invert: initial;
      --tw-opacity: initial;
      --tw-saturate: initial;
      --tw-sepia: initial;
      --tw-drop-shadow: initial;
      --tw-drop-shadow-color: initial;
      --tw-drop-shadow-alpha: 100%;
      --tw-drop-shadow-size: initial;
    }
  }
}

@layer theme {
  :root, :host {
    --font-sans: ui-sans-serif, system-ui, sans-serif;
    --font-mono: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, "Liberation
     Mono", "Courier New", monospace;
    --color-red-50: oklch(.971 .013 17.38);
    --color-red-100: oklch(.936 .032 17.717);
    --color-orange-100: oklch(.954 .038 75.164);
    --color-yellow-50: oklch(.987 .026 102.212);
    --color-yellow-100: oklch(.973 .071 103.193);
    --color-green-100: oklch(.962 .044 156.743);
    --color-green-600: oklch(.627 .194 149.214);
    --color-cyan-50: oklch(.984 .019 200.873);
    --color-cyan-100: oklch(.956 .045 203.388);
    --color-blue-50: oklch(.97 .014 254.604);
    --color-blue-100: oklch(.932 .032 255.585);
    --color-blue-200: oklch(.882 .059 254.128);
    --color-purple-100: oklch(.946 .033 307.174);
    --color-purple-600: oklch(.558 .288 302.321);
    --color-gray-50: oklch(.985 .002 247.839);
    --color-gray-100: oklch(.967 .003 264.542);
    --color-gray-200: oklch(.928 .006 264.531);
    --color-black: #000;
    --color-white: #fff;
    --spacing: .25rem;
    --container-md: 28rem;
    --container-3xl: 48rem;
    --container-4xl: 56rem;
    --text-sm: .875rem;
```

```
    --text-sm--line-height: calc(1.25 / .875);
    --radius-2xl: 1rem;
    --default-transition-duration: .15s;
    --default-transition-timing-function: cubic-bezier(.4, 0, .2, 1);
    --default-font-family: var(--font-sans);
    --default-font-feature-settings: var(--font-sans--font-feature-settings);
    --default-font-variation-settings: var(--font-sans--font-variation-settings);
    --default-mono-font-family: var(--font-mono);
    --default-mono-font-feature-settings: var(--font-mono--font-feature-settings);
    --default-mono-font-variation-settings: var(--font-mono--font-variation-settings);
    --color-background: #f8fafc;
    --color-primary: #3b82f6;
    --color-secondary: #06b6d4;
    --color-border: #e2e8f0;
    --color-primary-dark: #2563eb;
    --color-medical-blue: #e8f4f8;
    --color-success: #10b981;
    --color-warning: #f59e0b;
    --color-danger: #ef4444;
    --color-text-primary: #1e293b;
    --color-text-secondary: #64748b;
  }
}

@layer base {
  *, :after, :before, ::backdrop {
    box-sizing: border-box;
    border: 0 solid;
    margin: 0;
    padding: 0;
  }

  ::file-selector-button {
    box-sizing: border-box;
    border: 0 solid;
    margin: 0;
    padding: 0;
  }

  html, :host {
    -webkit-text-size-adjust: 100%;
    tab-size: 4;
    line-height: 1.5;
    font-family: var(--default-font-family, ui-sans-serif, system-ui, sans-serif, "Apple
     Color Emoji", "Segoe UI Emoji", "Segoe UI Symbol", "Noto Color Emoji");
    font-feature-settings: var(--default-font-feature-settings, normal);
    font-variation-settings: var(--default-font-variation-settings, normal);
    -webkit-tap-highlight-color: transparent;
  }

  body {
    line-height: inherit;
  }

  hr {
    height: 0;
    color: inherit;
    border-top-width: 1px;
  }

  abbr:where([title]) {
    -webkit-text-decoration: underline dotted;
    text-decoration: underline dotted;
  }

  h1, h2, h3, h4, h5, h6 {
    font-size: inherit;
    font-weight: inherit;
  }

  a {
    color: inherit;
    -webkit-text-decoration: inherit;
    -webkit-text-decoration: inherit;
    text-decoration: inherit;
  }

  b, strong {
    font-weight: bolder;
  }

  code, kbd, samp, pre {
```

```
  font-family: var(--default-mono-font-family, ui-monospace, SFMono-Regular, Menlo,
   Monaco, Consolas, "Liberation Mono", "Courier New", monospace);
  font-feature-settings: var(--default-mono-font-feature-settings, normal);
  font-variation-settings: var(--default-mono-font-variation-settings, normal);
  font-size: 1em;
}

small {
  font-size: 80%;
}

sub, sup {
  vertical-align: baseline;
  font-size: 75%;
  line-height: 0;
  position: relative;
}

sub {
  bottom: -.25em;
}

sup {
  top: -.5em;
}

table {
  text-indent: 0;
  border-color: inherit;
  border-collapse: collapse;
}

:-moz-focusring {
  outline: auto;
}

progress {
  vertical-align: baseline;
}

summary {
  display: list-item;
}

ol, ul, menu {
  list-style: none;
}

img, svg, video, canvas, audio, iframe, embed, object {
  vertical-align: middle;
  display: block;
}

img, video {
  max-width: 100%;
  height: auto;
}

button, input, select, optgroup, textarea {
  font: inherit;
  font-feature-settings: inherit;
  font-variation-settings: inherit;
  letter-spacing: inherit;
  color: inherit;
  opacity: 1;
  background-color: #0000;
  border-radius: 0;
}

::file-selector-button {
  font: inherit;
  font-feature-settings: inherit;
  font-variation-settings: inherit;
  letter-spacing: inherit;
  color: inherit;
  opacity: 1;
  background-color: #0000;
  border-radius: 0;
}

:where(select:is([multiple], [size])) optgroup {
  font-weight: bolder;
```

```
}

:where(select:is([multiple], [size])) optgroup option {
  padding-inline-start: 20px;
}

::file-selector-button {
  margin-inline-end: 4px;
}

::placeholder {
  opacity: 1;
  color: currentColor;
}

@supports (color: color-mix(in lab, red, red)) {
  ::placeholder {
    color: color-mix(in oklab, currentColor 50%, transparent);
  }
}

textarea {
  resize: vertical;
}

::-webkit-search-decoration {
  -webkit-appearance: none;
}

::-webkit-date-and-time-value {
  min-height: 1lh;
  text-align: inherit;
}

::-webkit-datetime-edit {
  display: inline-flex;
}

::-webkit-datetime-edit-fields-wrapper {
  padding: 0;
}

::-webkit-datetime-edit {
  padding-block: 0;
}

::-webkit-datetime-edit-year-field {
  padding-block: 0;
}

::-webkit-datetime-edit-month-field {
  padding-block: 0;
}

::-webkit-datetime-edit-day-field {
  padding-block: 0;
}

::-webkit-datetime-edit-hour-field {
  padding-block: 0;
}

::-webkit-datetime-edit-minute-field {
  padding-block: 0;
}

::-webkit-datetime-edit-second-field {
  padding-block: 0;
}

::-webkit-datetime-edit-millisecond-field {
  padding-block: 0;
}

::-webkit-datetime-edit-meridiem-field {
  padding-block: 0;
}

:-moz-ui-invalid {
  box-shadow: none;
}
```

```
  button, input:where([type="button"], [type="reset"], [type="submit"]) {
    appearance: button;
  }

  ::file-selector-button {
    appearance: button;
  }

  ::-webkit-inner-spin-button {
    height: auto;
  }

  ::-webkit-outer-spin-button {
    height: auto;
  }

  [hidden]:where(:not([hidden="until-found"])) {
    display: none !important;
  }

  * {
    border-color: var(--color-border);
    outline-color: color-mix(in oklab, oklch(.708 0 0) 50%, transparent);
  }

  body {
    background-color: var(--color-background);
    color: oklch(.145 0 0);
  }
}

@layer utilities {
  .absolute {
    position: absolute;
  }

  .fixed {
    position: fixed;
  }

  .relative {
    position: relative;
  }

  .sticky {
    position: sticky;
  }

  .inset-0 {
    inset: calc(var(--spacing) * 0);
  }

  .top-0 {
    top: calc(var(--spacing) * 0);
  }

  .top-1\/2 {
    top: 50%;
  }

  .top-24 {
    top: calc(var(--spacing) * 24);
  }

  .bottom-0 {
    bottom: calc(var(--spacing) * 0);
  }

  .left-3 {
    left: calc(var(--spacing) * 3);
  }

  .z-50 {
    z-index: 50;
  }

  .col-span-2 {
    grid-column: span 2 / span 2;
  }

  .mx-auto {
    margin-inline: auto;
```

```
}

.mt-0\.5 {
  margin-top: calc(var(--spacing) * .5);
}

.mt-1 {
  margin-top: calc(var(--spacing) * 1);
}

.mt-2 {
  margin-top: calc(var(--spacing) * 2);
}

.mt-3 {
  margin-top: calc(var(--spacing) * 3);
}

.mt-4 {
  margin-top: calc(var(--spacing) * 4);
}

.mt-6 {
  margin-top: calc(var(--spacing) * 6);
}

.mb-1 {
  margin-bottom: calc(var(--spacing) * 1);
}

.mb-2 {
  margin-bottom: calc(var(--spacing) * 2);
}

.mb-3 {
  margin-bottom: calc(var(--spacing) * 3);
}

.mb-4 {
  margin-bottom: calc(var(--spacing) * 4);
}

.mb-6 {
  margin-bottom: calc(var(--spacing) * 6);
}

.mb-8 {
  margin-bottom: calc(var(--spacing) * 8);
}

.mb-16 {
  margin-bottom: calc(var(--spacing) * 16);
}

.ml-2 {
  margin-left: calc(var(--spacing) * 2);
}

.block {
  display: block;
}

.flex {
  display: flex;
}

.grid {
  display: grid;
}

.hidden {
  display: none;
}

.inline-flex {
  display: inline-flex;
}

.h-2 {
  height: calc(var(--spacing) * 2);
}
```

```
.h-3 {
  height: calc(var(--spacing) * 3);
}

.h-4 {
  height: calc(var(--spacing) * 4);
}

.h-5 {
  height: calc(var(--spacing) * 5);
}

.h-6 {
  height: calc(var(--spacing) * 6);
}

.h-8 {
  height: calc(var(--spacing) * 8);
}

.h-10 {
  height: calc(var(--spacing) * 10);
}

.h-12 {
  height: calc(var(--spacing) * 12);
}

.h-16 {
  height: calc(var(--spacing) * 16);
}

.h-20 {
  height: calc(var(--spacing) * 20);
}

.h-96 {
  height: calc(var(--spacing) * 96);
}

.max-h-96 {
  max-height: calc(var(--spacing) * 96);
}

.max-h-\[90vh\] {
  max-height: 90vh;
}

.min-h-screen {
  min-height: 100vh;
}

.w-2 {
  width: calc(var(--spacing) * 2);
}

.w-4 {
  width: calc(var(--spacing) * 4);
}

.w-5 {
  width: calc(var(--spacing) * 5);
}

.w-6 {
  width: calc(var(--spacing) * 6);
}

.w-8 {
  width: calc(var(--spacing) * 8);
}

.w-10 {
  width: calc(var(--spacing) * 10);
}

.w-12 {
  width: calc(var(--spacing) * 12);
}

.w-16 {
  width: calc(var(--spacing) * 16);
```

```
}

.w-20 {
  width: calc(var(--spacing) * 20);
}

.w-full {
  width: 100%;
}

.max-w-3xl {
  max-width: var(--container-3xl);
}

.max-w-4xl {
  max-width: var(--container-4xl);
}

.max-w-full {
  max-width: 100%;
}

.max-w-md {
  max-width: var(--container-md);
}

.flex-1 {
  flex: 1;
}

.flex-shrink-0 {
  flex-shrink: 0;
}

.-translate-y-1\/2 {
  --tw-translate-y: calc(calc(1 / 2 * 100%) * -1);
  translate: var(--tw-translate-x) var(--tw-translate-y);
}

.cursor-pointer {
  cursor: pointer;
}

.resize-none {
  resize: none;
}

.list-inside {
  list-style-position: inside;
}

.list-disc {
  list-style-type: disc;
}

.appearance-none {
  appearance: none;
}

.grid-cols-1 {
  grid-template-columns: repeat(1, minmax(0, 1fr));
}

.grid-cols-2 {
  grid-template-columns: repeat(2, minmax(0, 1fr));
}

.grid-cols-5 {
  grid-template-columns: repeat(5, minmax(0, 1fr));
}

.grid-cols-7 {
  grid-template-columns: repeat(7, minmax(0, 1fr));
}

.flex-col {
  flex-direction: column;
}

.flex-wrap {
  flex-wrap: wrap;
}
```

```
.items-center {
  align-items: center;
}

.items-start {
  align-items: flex-start;
}

.justify-between {
  justify-content: space-between;
}

.justify-center {
  justify-content: center;
}

.justify-end {
  justify-content: flex-end;
}

.gap-2 {
  gap: calc(var(--spacing) * 2);
}

.gap-3 {
  gap: calc(var(--spacing) * 3);
}

.gap-4 {
  gap: calc(var(--spacing) * 4);
}

.gap-6 {
  gap: calc(var(--spacing) * 6);
}

.gap-8 {
  gap: calc(var(--spacing) * 8);
}

:where(.space-y-1 &gt; :not(:last-child)) {
  --tw-space-y-reverse: 0;
  margin-block-start: calc(calc(var(--spacing) * 1) * var(--tw-space-y-reverse));
  margin-block-end: calc(calc(var(--spacing) * 1) * calc(1 -
   var(--tw-space-y-reverse)));
}

:where(.space-y-2 &gt; :not(:last-child)) {
  --tw-space-y-reverse: 0;
  margin-block-start: calc(calc(var(--spacing) * 2) * var(--tw-space-y-reverse));
  margin-block-end: calc(calc(var(--spacing) * 2) * calc(1 -
   var(--tw-space-y-reverse)));
}

:where(.space-y-3 &gt; :not(:last-child)) {
  --tw-space-y-reverse: 0;
  margin-block-start: calc(calc(var(--spacing) * 3) * var(--tw-space-y-reverse));
  margin-block-end: calc(calc(var(--spacing) * 3) * calc(1 -
   var(--tw-space-y-reverse)));
}

:where(.space-y-4 &gt; :not(:last-child)) {
  --tw-space-y-reverse: 0;
  margin-block-start: calc(calc(var(--spacing) * 4) * var(--tw-space-y-reverse));
  margin-block-end: calc(calc(var(--spacing) * 4) * calc(1 -
   var(--tw-space-y-reverse)));
}

:where(.space-y-5 &gt; :not(:last-child)) {
  --tw-space-y-reverse: 0;
  margin-block-start: calc(calc(var(--spacing) * 5) * var(--tw-space-y-reverse));
  margin-block-end: calc(calc(var(--spacing) * 5) * calc(1 -
   var(--tw-space-y-reverse)));
}

:where(.space-y-6 &gt; :not(:last-child)) {
  --tw-space-y-reverse: 0;
  margin-block-start: calc(calc(var(--spacing) * 6) * var(--tw-space-y-reverse));
  margin-block-end: calc(calc(var(--spacing) * 6) * calc(1 -
   var(--tw-space-y-reverse)));
}
```

```
.self-start {
  align-self: flex-start;
}

.overflow-hidden {
  overflow: hidden;
}

.overflow-x-auto {
  overflow-x: auto;
}

.overflow-y-auto {
  overflow-y: auto;
}

.rounded {
  border-radius: .25rem;
}

.rounded-2xl {
  border-radius: var(--radius-2xl);
}

.rounded-full {
  border-radius: 3.40282e38px;
}

.rounded-lg {
  border-radius: .625rem;
}

.rounded-xl {
  border-radius: 1.025rem;
}

.border {
  border-style: var(--tw-border-style);
  border-width: 1px;
}

.border-2 {
  border-style: var(--tw-border-style);
  border-width: 2px;
}

.border-t {
  border-top-style: var(--tw-border-style);
  border-top-width: 1px;
}

.border-b {
  border-bottom-style: var(--tw-border-style);
  border-bottom-width: 1px;
}

.border-b-2 {
  border-bottom-style: var(--tw-border-style);
  border-bottom-width: 2px;
}

.border-blue-200 {
  border-color: var(--color-blue-200);
}

.border-border {
  border-color: var(--color-border);
}

.border-primary {
  border-color: var(--color-primary);
}

.border-warning\/30 {
  border-color: #f59e0b4d;
}

@supports (color: color-mix(in lab, red, red)) {
  .border-warning\/30 {
    border-color: color-mix(in oklab, var(--color-warning) 30%, transparent);
  }
}
```

```css
.bg-background {
  background-color: var(--color-background);
}

.bg-black\/50 {
  background-color: #00000080;
}

@supports (color: color-mix(in lab, red, red)) {
  .bg-black\/50 {
    background-color: color-mix(in oklab, var(--color-black) 50%, transparent);
  }
}

.bg-blue-50 {
  background-color: var(--color-blue-50);
}

.bg-blue-100 {
  background-color: var(--color-blue-100);
}

.bg-cyan-100 {
  background-color: var(--color-cyan-100);
}

.bg-gray-50 {
  background-color: var(--color-gray-50);
}

.bg-gray-100 {
  background-color: var(--color-gray-100);
}

.bg-gray-200 {
  background-color: var(--color-gray-200);
}

.bg-green-100 {
  background-color: var(--color-green-100);
}

.bg-medical-blue {
  background-color: var(--color-medical-blue);
}

.bg-orange-100 {
  background-color: var(--color-orange-100);
}

.bg-primary {
  background-color: var(--color-primary);
}

.bg-purple-100 {
  background-color: var(--color-purple-100);
}

.bg-red-50 {
  background-color: var(--color-red-50);
}

.bg-red-100 {
  background-color: var(--color-red-100);
}

.bg-secondary {
  background-color: var(--color-secondary);
}

.bg-success {
  background-color: var(--color-success);
}

.bg-white {
  background-color: var(--color-white);
}

.bg-yellow-50 {
  background-color: var(--color-yellow-50);
}
```

```
.bg-yellow-100 {
  background-color: var(--color-yellow-100);
}

.bg-gradient-to-br {
  --tw-gradient-position: to bottom right in oklab;
  background-image: linear-gradient(var(--tw-gradient-stops));
}

.bg-gradient-to-r {
  --tw-gradient-position: to right in oklab;
  background-image: linear-gradient(var(--tw-gradient-stops));
}

.from-blue-50 {
  --tw-gradient-from: var(--color-blue-50);
  --tw-gradient-stops: var(--tw-gradient-via-stops, var(--tw-gradient-position),
   var(--tw-gradient-from) var(--tw-gradient-from-position), var(--tw-gradient-to)
   var(--tw-gradient-to-position));
}

.via-white {
  --tw-gradient-via: var(--color-white);
  --tw-gradient-via-stops: var(--tw-gradient-position), var(--tw-gradient-from)
   var(--tw-gradient-from-position), var(--tw-gradient-via)
   var(--tw-gradient-via-position), var(--tw-gradient-to)
   var(--tw-gradient-to-position);
  --tw-gradient-stops: var(--tw-gradient-via-stops);
}

.to-blue-50 {
  --tw-gradient-to: var(--color-blue-50);
  --tw-gradient-stops: var(--tw-gradient-via-stops, var(--tw-gradient-position),
   var(--tw-gradient-from) var(--tw-gradient-from-position), var(--tw-gradient-to)
   var(--tw-gradient-to-position));
}

.to-cyan-50 {
  --tw-gradient-to: var(--color-cyan-50);
  --tw-gradient-stops: var(--tw-gradient-via-stops, var(--tw-gradient-position),
   var(--tw-gradient-from) var(--tw-gradient-from-position), var(--tw-gradient-to)
   var(--tw-gradient-to-position));
}

.p-2 {
  padding: calc(var(--spacing) * 2);
}

.p-3 {
  padding: calc(var(--spacing) * 3);
}

.p-4 {
  padding: calc(var(--spacing) * 4);
}

.p-6 {
  padding: calc(var(--spacing) * 6);
}

.p-8 {
  padding: calc(var(--spacing) * 8);
}

.px-2 {
  padding-inline: calc(var(--spacing) * 2);
}

.px-3 {
  padding-inline: calc(var(--spacing) * 3);
}

.px-4 {
  padding-inline: calc(var(--spacing) * 4);
}

.px-6 {
  padding-inline: calc(var(--spacing) * 6);
}

.py-1 {
  padding-block: calc(var(--spacing) * 1);
```

```
}

.py-2 {
  padding-block: calc(var(--spacing) * 2);
}

.py-3 {
  padding-block: calc(var(--spacing) * 3);
}

.py-4 {
  padding-block: calc(var(--spacing) * 4);
}

.py-8 {
  padding-block: calc(var(--spacing) * 8);
}

.py-12 {
  padding-block: calc(var(--spacing) * 12);
}

.pt-4 {
  padding-top: calc(var(--spacing) * 4);
}

.pr-4 {
  padding-right: calc(var(--spacing) * 4);
}

.pb-4 {
  padding-bottom: calc(var(--spacing) * 4);
}

.pl-10 {
  padding-left: calc(var(--spacing) * 10);
}

.text-center {
  text-align: center;
}

.text-left {
  text-align: left;
}

.text-sm {
  font-size: var(--text-sm);
  line-height: var(--tw-leading, var(--text-sm--line-height));
}

.whitespace-nowrap {
  white-space: nowrap;
}

.text-danger {
  color: var(--color-danger);
}

.text-primary {
  color: var(--color-primary);
}

.text-purple-600 {
  color: var(--color-purple-600);
}

.text-secondary {
  color: var(--color-secondary);
}

.text-success {
  color: var(--color-success);
}

.text-text-primary {
  color: var(--color-text-primary);
}

.text-text-secondary {
  color: var(--color-text-secondary);
}
```

```
.text-warning {
  color: var(--color-warning);
}

.text-white {
  color: var(--color-white);
}

.accent-primary {
  accent-color: var(--color-primary);
}

.opacity-50 {
  opacity: .5;
}

.shadow-2xl {
  --tw-shadow: 0 25px 50px -12px var(--tw-shadow-color, #00000040);
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow),
   var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
}

.shadow-lg {
  --tw-shadow: 0 10px 15px -3px var(--tw-shadow-color, #0000001a), 0 4px 6px -4px
   var(--tw-shadow-color, #0000001a);
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow),
   var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
}

.shadow-md {
  --tw-shadow: 0 4px 6px -1px var(--tw-shadow-color, #0000001a), 0 2px 4px -2px
   var(--tw-shadow-color, #0000001a);
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow),
   var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
}

.shadow-sm {
  --tw-shadow: 0 1px 3px 0 var(--tw-shadow-color, #0000001a), 0 1px 2px -1px
   var(--tw-shadow-color, #0000001a);
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow),
   var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
}

.shadow-xl {
  --tw-shadow: 0 20px 25px -5px var(--tw-shadow-color, #0000001a), 0 8px 10px -6px
   var(--tw-shadow-color, #0000001a);
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow),
   var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
}

.filter {
  filter: var(--tw-blur, ) var(--tw-brightness, ) var(--tw-contrast, )
   var(--tw-grayscale, ) var(--tw-hue-rotate, ) var(--tw-invert, ) var(--tw-saturate, )
   var(--tw-sepia, ) var(--tw-drop-shadow, );
}

.transition-all {
  transition-property: all;
  transition-timing-function: var(--tw-ease, var(--default-transition-timing-function));
  transition-duration: var(--tw-duration, var(--default-transition-duration));
}

.transition-colors {
  transition-property: color, background-color, border-color, outline-color,
   text-decoration-color, fill, stroke, --tw-gradient-from, --tw-gradient-via,
   --tw-gradient-to;
  transition-timing-function: var(--tw-ease, var(--default-transition-timing-function));
  transition-duration: var(--tw-duration, var(--default-transition-duration));
}

.transition-shadow {
  transition-property: box-shadow;
  transition-timing-function: var(--tw-ease, var(--default-transition-timing-function));
  transition-duration: var(--tw-duration, var(--default-transition-duration));
}

@media (hover: hover) {
  .group-hover\:text-primary:is(:where(.group):hover *) {
    color: var(--color-primary);
  }
}
```

```
.last\:border-b-0:last-child {
  border-bottom-style: var(--tw-border-style);
  border-bottom-width: 0;
}

@media (hover: hover) {
  .hover\:border-primary:hover {
    border-color: var(--color-primary);
  }
}

@media (hover: hover) {
  .hover\:bg-gray-50:hover {
    background-color: var(--color-gray-50);
  }
}

@media (hover: hover) {
  .hover\:bg-gray-100:hover {
    background-color: var(--color-gray-100);
  }
}

@media (hover: hover) {
  .hover\:bg-green-600:hover {
    background-color: var(--color-green-600);
  }
}

@media (hover: hover) {
  .hover\:bg-medical-blue:hover {
    background-color: var(--color-medical-blue);
  }
}

@media (hover: hover) {
  .hover\:bg-primary-dark:hover {
    background-color: var(--color-primary-dark);
  }
}

@media (hover: hover) {
  .hover\:bg-red-50:hover {
    background-color: var(--color-red-50);
  }
}

@media (hover: hover) {
  .hover\:text-primary:hover {
    color: var(--color-primary);
  }
}

@media (hover: hover) {
  .hover\:shadow-lg:hover {
    --tw-shadow: 0 10px 15px -3px var(--tw-shadow-color, #0000001a), 0 4px 6px -4px
   var(--tw-shadow-color, #0000001a);
    box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow),
   var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
  }
}

.focus\:border-transparent:focus {
  border-color: #0000;
}

.focus\:ring-2:focus {
  --tw-ring-shadow: var(--tw-ring-inset, ) 0 0 0 calc(2px +
   var(--tw-ring-offset-width)) var(--tw-ring-color, currentcolor);
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow),
   var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
}

.focus\:ring-primary:focus {
  --tw-ring-color: var(--color-primary);
}

.focus\:outline-none:focus {
  --tw-outline-style: none;
  outline-style: none;
}
```

```
  @media (width &gt;= 40rem) {
    .sm\:block {
      display: block;
    }
  }

  @media (width &gt;= 48rem) {
    .md\:block {
      display: block;
    }
  }

  @media (width &gt;= 48rem) {
    .md\:flex {
      display: flex;
    }
  }

  @media (width &gt;= 48rem) {
    .md\:hidden {
      display: none;
    }
  }

  @media (width &gt;= 48rem) {
    .md\:w-48 {
      width: calc(var(--spacing) * 48);
    }
  }

  @media (width &gt;= 48rem) {
    .md\:grid-cols-2 {
      grid-template-columns: repeat(2, minmax(0, 1fr));
    }
  }

  @media (width &gt;= 48rem) {
    .md\:grid-cols-3 {
      grid-template-columns: repeat(3, minmax(0, 1fr));
    }
  }

  @media (width &gt;= 48rem) {
    .md\:grid-cols-4 {
      grid-template-columns: repeat(4, minmax(0, 1fr));
    }
  }

  @media (width &gt;= 48rem) {
    .md\:flex-row {
      flex-direction: row;
    }
  }

  @media (width &gt;= 48rem) {
    .md\:items-center {
      align-items: center;
    }
  }

  @media (width &gt;= 64rem) {
    .lg\:grid-cols-2 {
      grid-template-columns: repeat(2, minmax(0, 1fr));
    }
  }

  @media (width &gt;= 64rem) {
    .lg\:grid-cols-4 {
      grid-template-columns: repeat(4, minmax(0, 1fr));
    }
  }
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  background-color: var(--color-background);
  color: oklch(.145 0 0);
```

```
  width: 100%;
  min-height: 100vh;
  overflow-x: hidden;
}

#root {
  width: 100%;
  min-height: 100vh;
}

h1 {
  color: var(--color-text-primary);
  font-size: 2rem;
  font-weight: 700;
  line-height: 1.2;
}

h2 {
  color: var(--color-text-primary);
  font-size: 1.5rem;
  font-weight: 600;
  line-height: 1.3;
}

h3 {
  color: var(--color-text-primary);
  font-size: 1.25rem;
  font-weight: 600;
  line-height: 1.4;
}

h4 {
  color: var(--color-text-primary);
  font-size: 1.125rem;
  font-weight: 600;
  line-height: 1.4;
}

p {
  font-size: 1rem;
  line-height: 1.6;
}

button {
  font-size: 1rem;
  font-weight: 500;
  line-height: 1.5;
}

input, textarea, select {
  font-size: 1rem;
  line-height: 1.5;
}

label {
  color: var(--color-text-primary);
  font-size: .875rem;
  font-weight: 500;
  line-height: 1.5;
}

@property --tw-translate-x {
  syntax: "*";
  inherits: false;
  initial-value: 0;
}

@property --tw-translate-y {
  syntax: "*";
  inherits: false;
  initial-value: 0;
}

@property --tw-translate-z {
  syntax: "*";
  inherits: false;
  initial-value: 0;
}

@property --tw-space-y-reverse {
  syntax: "*";
  inherits: false;
```

```
  initial-value: 0;
}

@property --tw-border-style {
  syntax: "*";
  inherits: false;
  initial-value: solid;
}

@property --tw-gradient-position {
  syntax: "*";
  inherits: false
}

@property --tw-gradient-from {
  syntax: "&lt;color&gt;";
  inherits: false;
  initial-value: #0000;
}

@property --tw-gradient-via {
  syntax: "&lt;color&gt;";
  inherits: false;
  initial-value: #0000;
}

@property --tw-gradient-to {
  syntax: "&lt;color&gt;";
  inherits: false;
  initial-value: #0000;
}

@property --tw-gradient-stops {
  syntax: "*";
  inherits: false
}

@property --tw-gradient-via-stops {
  syntax: "*";
  inherits: false
}

@property --tw-gradient-from-position {
  syntax: "&lt;length-percentage&gt;";
  inherits: false;
  initial-value: 0%;
}

@property --tw-gradient-via-position {
  syntax: "&lt;length-percentage&gt;";
  inherits: false;
  initial-value: 50%;
}

@property --tw-gradient-to-position {
  syntax: "&lt;length-percentage&gt;";
  inherits: false;
  initial-value: 100%;
}

@property --tw-shadow {
  syntax: "*";
  inherits: false;
  initial-value: 0 0 #0000;
}

@property --tw-shadow-color {
  syntax: "*";
  inherits: false
}

@property --tw-shadow-alpha {
  syntax: "&lt;percentage&gt;";
  inherits: false;
  initial-value: 100%;
}

@property --tw-inset-shadow {
  syntax: "*";
  inherits: false;
  initial-value: 0 0 #0000;
}
```

```
@property --tw-inset-shadow-color {
  syntax: "*";
  inherits: false
}

@property --tw-inset-shadow-alpha {
  syntax: "&lt;percentage&gt;";
  inherits: false;
  initial-value: 100%;
}

@property --tw-ring-color {
  syntax: "*";
  inherits: false
}

@property --tw-ring-shadow {
  syntax: "*";
  inherits: false;
  initial-value: 0 0 #0000;
}

@property --tw-inset-ring-color {
  syntax: "*";
  inherits: false
}

@property --tw-inset-ring-shadow {
  syntax: "*";
  inherits: false;
  initial-value: 0 0 #0000;
}

@property --tw-ring-inset {
  syntax: "*";
  inherits: false
}

@property --tw-ring-offset-width {
  syntax: "&lt;length&gt;";
  inherits: false;
  initial-value: 0;
}

@property --tw-ring-offset-color {
  syntax: "*";
  inherits: false;
  initial-value: #fff;
}

@property --tw-ring-offset-shadow {
  syntax: "*";
  inherits: false;
  initial-value: 0 0 #0000;
}

@property --tw-blur {
  syntax: "*";
  inherits: false
}

@property --tw-brightness {
  syntax: "*";
  inherits: false
}

@property --tw-contrast {
  syntax: "*";
  inherits: false
}

@property --tw-grayscale {
  syntax: "*";
  inherits: false
}

@property --tw-hue-rotate {
  syntax: "*";
  inherits: false
}

@property --tw-invert {
```

```
  syntax: "*";
  inherits: false
}

@property --tw-opacity {
  syntax: "*";
  inherits: false
}

@property --tw-saturate {
  syntax: "*";
  inherits: false
}

@property --tw-sepia {
  syntax: "*";
  inherits: false
}

@property --tw-drop-shadow {
  syntax: "*";
  inherits: false
}

@property --tw-drop-shadow-color {
  syntax: "*";
  inherits: false
}

@property --tw-drop-shadow-alpha {
  syntax: "&lt;percentage&gt;";
  inherits: false;
  initial-value: 100%;
}

@property --tw-drop-shadow-size {
  syntax: "*";
  inherits: false
}
```

■ File: src\index.html

```
&lt;!DOCTYPE html&gt;
&lt;html lang="zh-TW"&gt;
  &lt;head&gt;
    &lt;meta charset="UTF-8" /&gt;
    &lt;meta name="viewport" content="width=device-width, initial-scale=1.0" /&gt;
    &lt;title&gt;■■■■■■■■■■&lt;/title&gt;
  &lt;/head&gt;
  &lt;body&gt;
    &lt;div id="root"&gt;&lt;/div&gt;
    &lt;script type="module" src="/main.tsx"&gt;&lt;/script&gt;
  &lt;/body&gt;
&lt;/html&gt;
```

■ File: src\main.tsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import './index.css'; // ■ ■■■■■■■ CSS ■■

ReactDOM.createRoot(document.getElementById('root')!).render(
  &lt;React.StrictMode&gt;
    &lt;BrowserRouter&gt;
      &lt;App /&gt;
    &lt;/BrowserRouter&gt;
  &lt;/React.StrictMode&gt;
);
```

■ File: src\store\useAuthStore.ts

```
import { create } from 'zustand';

export type UserRole = 'patient' | 'doctor' | 'caseManager' | null;

export interface User {
  id: string;
  name: string;
  role: UserRole;
}

interface AuthState {
  currentUser: User | null;
  login: (user: User) =&gt; void;
  logout: () =&gt; void;
}

export const useAuthStore = create&lt;AuthState&gt;((set) =&gt; ({
  currentUser: null,
  login: (user) =&gt; set({ currentUser: user }),
  logout: () =&gt; set({ currentUser: null }),
}));
```

■ File: src\styles\globals.css

```
@import "tailwindcss";

@theme {
  /* Color Palette - Medical Theme */
  --color-primary: #3b82f6;
  --color-primary-light: #60a5fa;
  --color-primary-dark: #2563eb;
  --color-secondary: #06b6d4;
  --color-background: #f8fafc;
  --color-surface: #ffffff;
  --color-medical-blue: #e8f4f8;
  --color-success: #10b981;
  --color-warning: #f59e0b;
  --color-danger: #ef4444;
  --color-text-primary: #1e293b;
  --color-text-secondary: #64748b;
  --color-border: #e2e8f0;

  /* Typography */
  --font-sans: ui-sans-serif, system-ui, sans-serif;
}

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  background-color: var(--color-background);
  color: oklch(.145 0 0);

  /* ■■■■■■■■■■ */
  min-height: 100vh;
  width: 100%;
  overflow-x: hidden;
}

/* ■■■■■■ # ■■■■■ React ■■■■■■ */
#root {
  width: 100%;
  min-height: 100vh;
}

h1 {
  font-size: 2rem;
  font-weight: 700;
  line-height: 1.2;
  color: var(--color-text-primary);
}

h2 {
  font-size: 1.5rem;
  font-weight: 600;
  line-height: 1.3;
  color: var(--color-text-primary);
}

h3 {
  font-size: 1.25rem;
  font-weight: 600;
  line-height: 1.4;
  color: var(--color-text-primary);
}

h4 {
  font-size: 1.125rem;
  font-weight: 600;
  line-height: 1.4;
  color: var(--color-text-primary);
}

p {
  font-size: 1rem;
  line-height: 1.6;
}

button {
  font-size: 1rem;
```

```
  font-weight: 500;
  line-height: 1.5;
}

input, textarea, select {
  font-size: 1rem;
  line-height: 1.5;
}

label {
  font-size: 0.875rem;
  font-weight: 500;
  line-height: 1.5;
  color: var(--color-text-primary);
}
```

■ File: src\supabase\functions\server\index.tsx

```
import { Hono } from "npm:hono";
import { cors } from "npm:hono/cors";
import { logger } from "npm:hono/logger";
import * as kv from "./kv_store.tsx";
const app = new Hono();

// Enable logger
app.use('*', logger(console.log));

// Enable CORS for all routes and methods
app.use(
  "/*",
  cors({
    origin: "*",
    allowHeaders: ["Content-Type", "Authorization"],
    allowMethods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    exposeHeaders: ["Content-Length"],
    maxAge: 600,
  }),
);

// Health check endpoint
app.get("/make-server-826d6f62/health", (c) =&gt; {
  return c.json({ status: "ok" });
});

Deno.serve(app.fetch);
```

■ File: src\supabase\functions\server\kv_store.tsx

```
/* AUTOGENERATED FILE - DO NOT EDIT CONTENTS */

/* Table schema:
CREATE TABLE kv_store_826d6f62 (
  key TEXT NOT NULL PRIMARY KEY,
  value JSONB NOT NULL
);
*/

// View at https://supabase.com/dashboard/project/hkwvplgzliyvrpoqpksh/database/tables

// This file provides a simple key-value interface for storing Figma Make data. It should
     be adequate for most small-scale use cases.
import { createClient } from "jsr:@supabase/supabase-js@2.49.8";

const client = () =&gt; createClient(
  Deno.env.get("SUPABASE_URL"),
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY"),
);

// Set stores a key-value pair in the database.
export const set = async (key: string, value: any): Promise&lt;void&gt; =&gt; {
  const supabase = client()
  const { error } = await supabase.from("kv_store_826d6f62").upsert({
    key,
    value
  });
  if (error) {
    throw new Error(error.message);
  }
};

// Get retrieves a key-value pair from the database.
export const get = async (key: string): Promise&lt;any&gt; =&gt; {
  const supabase = client()
  const { data, error } = await supabase.from("kv_store_826d6f62").select("value").eq("key
    ", key).maybeSingle();
  if (error) {
    throw new Error(error.message);
  }
  return data?.value;
};

// Delete deletes a key-value pair from the database.
export const del = async (key: string): Promise&lt;void&gt; =&gt; {
  const supabase = client()
  const { error } = await supabase.from("kv_store_826d6f62").delete().eq("key", key);
  if (error) {
    throw new Error(error.message);
  }
};

// Sets multiple key-value pairs in the database.
export const mset = async (keys: string[], values: any[]): Promise&lt;void&gt; =&gt; {
  const supabase = client()
  const { error } = await supabase.from("kv_store_826d6f62").upsert(keys.map((k, i) =&gt; ({
     key: k, value: values[i] })));
  if (error) {
    throw new Error(error.message);
  }
};

// Gets multiple key-value pairs from the database.
export const mget = async (keys: string[]): Promise&lt;any[]&gt; =&gt; {
  const supabase = client()
  const { data, error } = await supabase.from("kv_store_826d6f62").select("value").in("key
    ", keys);
  if (error) {
    throw new Error(error.message);
  }
  return data?.map((d) =&gt; d.value) ?? [];
};

// Deletes multiple key-value pairs from the database.
export const mdel = async (keys: string[]): Promise&lt;void&gt; =&gt; {
  const supabase = client()
  const { error } = await supabase.from("kv_store_826d6f62").delete().in("key", keys);
  if (error) {
    throw new Error(error.message);
```

```
  }
};

// Search for key-value pairs by prefix.
export const getByPrefix = async (prefix: string): Promise&lt;any[]&gt; =&gt; {
  const supabase = client()
  const { data, error } = await supabase.from("kv_store_826d6f62").select("key,
     value").like("key", prefix + "%");
  if (error) {
    throw new Error(error.message);
  }
  return data?.map((d) =&gt; d.value) ?? [];
};
```

■ File: src\utils\supabase\info.tsx

```
/* AUTOGENERATED FILE - DO NOT EDIT CONTENTS */

export const projectId = "hkwvplgzliyvrpoqpksh"
export const publicAnonKey = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSI
    sInJlZiI6Imhrd3ZwbGd6bGl5dnJwb3Fwa3NoIiwicm9sZSI6ImFub24iLCJpYXQiOjE3NjQwNzExMDAsImV4c
    CI6MjA3OTY0NzEwMH0.eGqkO0OOhORuxuCul4YFc5xv_BClURVznV5x06msX5g"
```

■ File: src\vite.config.ts

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import path from 'path';
import tailwindcss from '@tailwindcss/vite'; // ■■■■ Tailwind ■■

export default defineConfig({
  plugins: [
    react(),
    tailwindcss(), // ■■ Tailwind ■■
  ],
  resolve: {
    extensions: ['.js', '.jsx', '.ts', '.tsx', '.json'],
    alias: {
      'vaul@1.1.2': 'vaul',
      'sonner@2.0.3': 'sonner',
      'recharts@2.15.2': 'recharts',
      'react-resizable-panels@2.1.7': 'react-resizable-panels',
      'react-hook-form@7.55.0': 'react-hook-form',
      'react-day-picker@8.10.1': 'react-day-picker',
      'next-themes@0.4.6': 'next-themes',
      'lucide-react@0.487.0': 'lucide-react',
      'input-otp@1.4.2': 'input-otp',
      'embla-carousel-react@8.6.0': 'embla-carousel-react',
      'cmdk@1.1.1': 'cmdk',
      'class-variance-authority@0.7.1': 'class-variance-authority',
      '@radix-ui/react-tooltip@1.1.8': '@radix-ui/react-tooltip',
      '@radix-ui/react-toggle@1.1.2': '@radix-ui/react-toggle',
      '@radix-ui/react-toggle-group@1.1.2': '@radix-ui/react-toggle-group',
      '@radix-ui/react-tabs@1.1.3': '@radix-ui/react-tabs',
      '@radix-ui/react-switch@1.1.3': '@radix-ui/react-switch',
      '@radix-ui/react-slot@1.1.2': '@radix-ui/react-slot',
      '@radix-ui/react-slider@1.2.3': '@radix-ui/react-slider',
      '@radix-ui/react-separator@1.1.2': '@radix-ui/react-separator',
      '@radix-ui/react-select@2.1.6': '@radix-ui/react-select',
      '@radix-ui/react-scroll-area@1.2.3': '@radix-ui/react-scroll-area',
      '@radix-ui/react-radio-group@1.2.3': '@radix-ui/react-radio-group',
      '@radix-ui/react-progress@1.1.2': '@radix-ui/react-progress',
      '@radix-ui/react-popover@1.1.6': '@radix-ui/react-popover',
      '@radix-ui/react-navigation-menu@1.2.5': '@radix-ui/react-navigation-menu',
      '@radix-ui/react-menubar@1.1.6': '@radix-ui/react-menubar',
      '@radix-ui/react-label@2.1.2': '@radix-ui/react-label',
      '@radix-ui/react-hover-card@1.1.6': '@radix-ui/react-hover-card',
      '@radix-ui/react-dropdown-menu@2.1.6': '@radix-ui/react-dropdown-menu',
      '@radix-ui/react-dialog@1.1.6': '@radix-ui/react-dialog',
      '@radix-ui/react-context-menu@2.2.6': '@radix-ui/react-context-menu',
      '@radix-ui/react-collapsible@1.1.3': '@radix-ui/react-collapsible',
      '@radix-ui/react-checkbox@1.1.4': '@radix-ui/react-checkbox',
      '@radix-ui/react-avatar@1.1.3': '@radix-ui/react-avatar',
      '@radix-ui/react-aspect-ratio@1.1.2': '@radix-ui/react-aspect-ratio',
      '@radix-ui/react-alert-dialog@1.1.6': '@radix-ui/react-alert-dialog',
      '@radix-ui/react-accordion@1.2.3': '@radix-ui/react-accordion',
      '@': path.resolve(__dirname, './src'),
    },
  },
  build: {
    target: 'esnext',
    outDir: 'dist', // ■■■■■■■■■ 'dist'
  },
  server: {
    port: 3000,
    open: true,
  },
});
```

■ File: thomas-backend\build.gradle

```
plugins {
■id 'java'
■id 'org.springframework.boot' version '4.0.5'
■id 'io.spring.dependency-management' version '1.1.7'
}

group = 'com.thomas'
version = '0.0.1-SNAPSHOT'

java {
■toolchain {
■■languageVersion = JavaLanguageVersion.of(21)
■}
}

repositories {
■mavenCentral()
}

dependencies {
■implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
■implementation 'org.springframework.boot:spring-boot-starter-webmvc'
■compileOnly 'org.projectlombok:lombok'
■runtimeOnly 'com.mysql:mysql-connector-j'
■annotationProcessor 'org.projectlombok:lombok'
■testImplementation 'org.springframework.boot:spring-boot-starter-data-jpa-test'
■testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'
■testCompileOnly 'org.projectlombok:lombok'
■testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
■testAnnotationProcessor 'org.projectlombok:lombok'
}

tasks.named('test') {
■useJUnitPlatform()
}
```

■ File: thomas-backend\gradle\wrapper\gradle-wrapper.properties

```
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-9.4.1-bin.zip
networkTimeout=10000
validateDistributionUrl=true
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
```

■ File: thomas-backend\settings.gradle

```
rootProject.name = 'backend'
```

■ File: thomas-backend\src\main\java\com\thomas\backend\BackendApplication.java

```
package com.thomas.backend;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BackendApplication {

■public static void main(String[] args) {
■■SpringApplication.run(BackendApplication.class, args);
■}

}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\controller\AuthController.java

```
package com.thomas.backend.controller;

import com.thomas.backend.model.User;
import com.thomas.backend.repository.UserRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;
import java.util.Optional;

@RestController
@RequestMapping("/api/auth")
@CrossOrigin(origins = "*")
public class AuthController {

    private final UserRepository userRepository;

    public AuthController(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    // A helper route to init some default users if needed for testing
    @GetMapping("/init")
    public ResponseEntity&lt;?&gt; initUsers() {
        if (userRepository.count() == 0) {
            User patient = new User();
            patient.setName("patient1");
            patient.setPassword("1234");
            patient.setRole("patient");
            userRepository.save(patient);

            User doctor = new User();
            doctor.setName("doctor1");
            doctor.setPassword("1234");
            doctor.setRole("doctor");
            userRepository.save(doctor);

            return ResponseEntity.ok("Dummy initialized");
        }
        return ResponseEntity.ok("Already initialized");
    }

    @PostMapping("/register")
    public ResponseEntity&lt;?&gt; register(@RequestBody User user) {
        if (userRepository.findByName(user.getName()).isPresent()) {
            return ResponseEntity.status(400).body(Map.of("message", "■■■■■■■■■"));
        }
        User saved = userRepository.save(user);
        return ResponseEntity.ok(saved);
    }

    @PostMapping("/login")
    public ResponseEntity&lt;?&gt; login(@RequestBody Map&lt;String, String&gt; creds) {
        String name = creds.get("name");
        String password = creds.get("password");

        Optional&lt;User&gt; userOpt = userRepository.findByName(name);
        if (userOpt.isPresent() &amp;&amp; userOpt.get().getPassword().equals(password)) {
            return ResponseEntity.ok(userOpt.get());
        }
        return ResponseEntity.status(401).body(Map.of("message", "Invalid credentials"));
    }
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\controller\CaseManagerController.java

```
package com.thomas.backend.controller;

import com.thomas.backend.model.User;
import com.thomas.backend.model.PatientTracking;
import com.thomas.backend.repository.UserRepository;
import com.thomas.backend.repository.DiaryEntryRepository;
import com.thomas.backend.repository.PatientTrackingRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.*;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api/casemanager")
@CrossOrigin(origins = "*")
public class CaseManagerController {

    private final UserRepository userRepository;
    private final DiaryEntryRepository diaryRepository;
    private final PatientTrackingRepository trackingRepository;

    public CaseManagerController(UserRepository userRepository, DiaryEntryRepository
     diaryRepository, PatientTrackingRepository trackingRepository) {
        this.userRepository = userRepository;
        this.diaryRepository = diaryRepository;
        this.trackingRepository = trackingRepository;
    }

    @GetMapping("/patients")
    public ResponseEntity&lt;List&lt;Map&lt;String, Object&gt;&gt;&gt; getHighRiskPatients() {
        List&lt;User&gt; patients = userRepository.findAll().stream()
            .filter(u -&gt; "patient".equals(u.getRole()))
            .collect(Collectors.toList());

        List&lt;Map&lt;String, Object&gt;&gt; response = patients.stream().map(p -&gt; {
            long headacheDays = diaryRepository.findByUserIdOrderByDateDesc(p.getId()).siz
    e();

            PatientTracking tracking = trackingRepository.findById(p.getId()).orElseGet(()
     -&gt; {
                PatientTracking t = new PatientTracking();
                t.setPatientId(p.getId());
                t.setStatus("pending");
                t.setNotes("");
                return trackingRepository.save(t);
            });

            List&lt;String&gt; riskFactors = new ArrayList&lt;&gt;();
            if (headacheDays &gt;= 5) riskFactors.add("■■■■■■");

            Map&lt;String, Object&gt; map = new HashMap&lt;&gt;();
            map.put("id", p.getId());
            map.put("name", p.getName());
            map.put("age", 42); // Placeholder
            map.put("phone", "■■■");
            map.put("email", p.getName() + "@test.com");
            map.put("midasScore", 0); // Placeholder
            map.put("headacheDays", headacheDays);
            map.put("lastVisit", "■■");
            map.put("nextAppointment", tracking.getNextAppointment());
            map.put("riskFactors", riskFactors);
            map.put("status", tracking.getStatus() != null ? tracking.getStatus() :
     "pending");
            map.put("notes", tracking.getNotes() == null ? "" : tracking.getNotes());

            return map;
        }).collect(Collectors.toList());

        return ResponseEntity.ok(response);
    }

    @PutMapping("/tracking/{patientId}")
    public ResponseEntity&lt;?&gt; updateTracking(@PathVariable String patientId, @RequestBody
     Map&lt;String, String&gt; payload) {
        PatientTracking t = trackingRepository.findById(patientId).orElseGet(PatientTracki
    ng::new);
        t.setPatientId(patientId);
```

```
        if (payload.containsKey("status")) {
            t.setStatus(payload.get("status"));
        }
        if (payload.containsKey("nextAppointment")) {
            t.setNextAppointment(payload.get("nextAppointment"));
        }
        if (payload.containsKey("notes")) {
            t.setNotes(payload.get("notes"));
        }

        trackingRepository.save(t);
        return ResponseEntity.ok(Map.of("message", "Updated successfully"));
    }
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\controller\DiaryController.java

```
package com.thomas.backend.controller;

import com.thomas.backend.model.DiaryEntry;
import com.thomas.backend.repository.DiaryEntryRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/diaries")
@CrossOrigin(origins = "*")
public class DiaryController {

    private final DiaryEntryRepository repository;

    public DiaryController(DiaryEntryRepository repository) {
        this.repository = repository;
    }

    @GetMapping("/user/{userId}")
    public ResponseEntity&lt;List&lt;DiaryEntry&gt;&gt; getByUserId(@PathVariable String userId) {
        return ResponseEntity.ok(repository.findByUserIdOrderByDateDesc(userId));
    }

    @PostMapping
    public ResponseEntity&lt;DiaryEntry&gt; save(@RequestBody DiaryEntry entry) {
        return ResponseEntity.ok(repository.save(entry));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity&lt;?&gt; delete(@PathVariable String id) {
        repository.deleteById(id);
        return ResponseEntity.ok().build();
    }
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\controller\PrescriptionController.java

```
package com.thomas.backend.controller;

import com.thomas.backend.model.Prescription;
import com.thomas.backend.repository.PrescriptionRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/prescriptions")
@CrossOrigin(origins = "*")
public class PrescriptionController {

    private final PrescriptionRepository repository;

    public PrescriptionController(PrescriptionRepository repository) {
        this.repository = repository;
    }

    @GetMapping("/patient/{patientId}")
    public ResponseEntity&lt;List&lt;Prescription&gt;&gt; getByPatientId(@PathVariable String
     patientId) {
        return ResponseEntity.ok(repository.findByPatientIdOrderByCreatedAtDesc(patientId)
    );
    }

    @PostMapping
    public ResponseEntity&lt;Prescription&gt; save(@RequestBody Prescription prescription) {
        return ResponseEntity.ok(repository.save(prescription));
    }
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\controller\QuestionnaireController.java

```
package com.thomas.backend.controller;

import com.thomas.backend.model.QuestionnaireResponse;
import com.thomas.backend.repository.QuestionnaireResponseRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/questionnaires")
@CrossOrigin(origins = "*")
public class QuestionnaireController {

    private final QuestionnaireResponseRepository repository;

    public QuestionnaireController(QuestionnaireResponseRepository repository) {
        this.repository = repository;
    }

    @GetMapping("/patient/{patientId}")
    public ResponseEntity&lt;List&lt;QuestionnaireResponse&gt;&gt; getByPatientId(@PathVariable
     String patientId) {
        return ResponseEntity.ok(repository.findByPatientId(patientId));
    }

    @PostMapping
    public ResponseEntity&lt;QuestionnaireResponse&gt; save(@RequestBody QuestionnaireResponse
     response) {
        return ResponseEntity.ok(repository.save(response));
    }
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\controller\UserController.java

```
package com.thomas.backend.controller;

import com.thomas.backend.model.User;
import com.thomas.backend.repository.UserRepository;
import com.thomas.backend.repository.DiaryEntryRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.*;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api/users")
@CrossOrigin(origins = "*")
public class UserController {

    private final UserRepository userRepository;
    private final DiaryEntryRepository diaryRepository;

    public UserController(UserRepository userRepository, DiaryEntryRepository
     diaryRepository) {
        this.userRepository = userRepository;
        this.diaryRepository = diaryRepository;
    }

    @GetMapping("/patients")
    public ResponseEntity&lt;List&lt;Map&lt;String, Object&gt;&gt;&gt; getPatients() {
        List&lt;User&gt; patients = userRepository.findAll().stream()
            .filter(u -&gt; "patient".equals(u.getRole()))
            .collect(Collectors.toList());

        List&lt;Map&lt;String, Object&gt;&gt; response = patients.stream().map(p -&gt; {
            Map&lt;String, Object&gt; map = new HashMap&lt;&gt;();
            map.put("id", p.getId());
            map.put("name", p.getName());
            map.put("age", 40); // Base placeholder
            map.put("gender", "■■■");
            map.put("lastVisit", "■■");
            map.put("midasScore", 0); // Need questionnaire logic to replace this

            long headacheDays = diaryRepository.findByUserIdOrderByDateDesc(p.getId()).siz
    e();
            map.put("headacheDays", headacheDays);

            map.put("riskLevel", headacheDays &gt;= 10 ? "high" : headacheDays &gt;= 5 ?
     "medium" : "low");
            return map;
        }).collect(Collectors.toList());

        return ResponseEntity.ok(response);
    }
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\model\DiaryEntry.java

```
package com.thomas.backend.model;

import jakarta.persistence.*;
import lombok.Data;
import java.util.List;

@Entity
@Data
@Table(name = "diary_entries")
public class DiaryEntry {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    private String userId; // foreign key mapping to the User

    private String date;
    private String time;
    private Integer intensity;

    @ElementCollection
    private List&lt;String&gt; symptoms;

    private String medication;
    private String notes;
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\model\PatientTracking.java

```
package com.thomas.backend.model;

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDateTime;

@Entity
@Data
@Table(name = "patient_tracking")
public class PatientTracking {
    @Id
    private String patientId;

    private String status = "pending";
    private String nextAppointment;

    @Column(columnDefinition = "TEXT")
    private String notes;

    private LocalDateTime updatedAt = LocalDateTime.now();
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\model\Prescription.java

```
package com.thomas.backend.model;

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDateTime;

@Entity
@Data
@Table(name = "prescriptions")
public class Prescription {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    private String doctorId;
    private String patientId;

    @Column(columnDefinition = "TEXT")
    private String medicationDetails;

    @Column(columnDefinition = "TEXT")
    private String instructions;

    private LocalDateTime createdAt = LocalDateTime.now();
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\model\QuestionnaireResponse.java

```
package com.thomas.backend.model;

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDateTime;

@Entity
@Data
@Table(name = "questionnaire_responses")
public class QuestionnaireResponse {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    private String patientId;
    private String questionnaireId; // e.g. "midas"
    private Integer score;

    @Column(columnDefinition = "TEXT")
    private String details;

    private LocalDateTime completedAt = LocalDateTime.now();
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\model\User.java

```
package com.thomas.backend.model;

import jakarta.persistence.*;
import lombok.Data;

@Entity
@Data
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    private String name;

    private String password;

    private String role; // 'patient', 'doctor', 'caseManager'
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\repository\DiaryEntryRepository.java

```
package com.thomas.backend.repository;

import com.thomas.backend.model.DiaryEntry;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface DiaryEntryRepository extends JpaRepository&lt;DiaryEntry, String&gt; {
    List&lt;DiaryEntry&gt; findByUserIdOrderByDateDesc(String userId);
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\repository\PatientTrackingRepository.java

```
package com.thomas.backend.repository;

import com.thomas.backend.model.PatientTracking;
import org.springframework.data.jpa.repository.JpaRepository;

public interface PatientTrackingRepository extends JpaRepository&lt;PatientTracking, String&gt;
     {
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\repository\PrescriptionRepository.java

```
package com.thomas.backend.repository;

import com.thomas.backend.model.Prescription;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface PrescriptionRepository extends JpaRepository&lt;Prescription, String&gt; {
    List&lt;Prescription&gt; findByPatientIdOrderByCreatedAtDesc(String patientId);
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\repository\QuestionnaireResponseRepository.java

```
package com.thomas.backend.repository;

import com.thomas.backend.model.QuestionnaireResponse;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;
import java.util.Optional;

public interface QuestionnaireResponseRepository extends
     JpaRepository&lt;QuestionnaireResponse, String&gt; {
    List&lt;QuestionnaireResponse&gt; findByPatientId(String patientId);
    Optional&lt;QuestionnaireResponse&gt; findTopByPatientIdAndQuestionnaireIdOrderByCompletedAt
    Desc(String patientId, String questionnaireId);
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\repository\UserRepository.java

```
package com.thomas.backend.repository;

import com.thomas.backend.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository&lt;User, String&gt; {
    Optional&lt;User&gt; findByName(String name);
}
```

■ File: thomas-backend\src\main\resources\application.properties

```
spring.application.name=thomas-backend

# MySQL Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/thomas_db?useSSL=false&amp;serverTimezone=UT
    C&amp;allowPublicKeyRetrieval=true
spring.datasource.username=root
spring.datasource.password=root

# Hibernate ORM Settings
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Server Configuration
server.port=8080
```

■ File: thomas-backend\src\test\java\com\thomas\backend\BackendApplicationTests.java

```
package com.thomas.backend;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class BackendApplicationTests {

■@Test
■void contextLoads() {
■}

}
```

■ File: vite.config.ts

```
// Thomas-main/vite.config.ts
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import path from 'path';
import tailwindcss from '@tailwindcss/vite'; // ■■■■ Tailwind ■■

export default defineConfig({
  plugins: [
    react(),
    tailwindcss(), // ■■ Tailwind ■■
  ],
  resolve: {
    extensions: ['.js', '.jsx', '.ts', '.tsx', '.json'],
    alias: {
      'vaul@1.1.2': 'vaul',
      'sonner@2.0.3': 'sonner',
      'recharts@2.15.2': 'recharts',
      'react-resizable-panels@2.1.7': 'react-resizable-panels',
      'react-hook-form@7.55.0': 'react-hook-form',
      'react-day-picker@8.10.1': 'react-day-picker',
      'next-themes@0.4.6': 'next-themes',
      'lucide-react@0.487.0': 'lucide-react',
      'input-otp@1.4.2': 'input-otp',
      'embla-carousel-react@8.6.0': 'embla-carousel-react',
      'cmdk@1.1.1': 'cmdk',
      'class-variance-authority@0.7.1': 'class-variance-authority',
      '@radix-ui/react-tooltip@1.1.8': '@radix-ui/react-tooltip',
      '@radix-ui/react-toggle@1.1.2': '@radix-ui/react-toggle',
      '@radix-ui/react-toggle-group@1.1.2': '@radix-ui/react-toggle-group',
      '@radix-ui/react-tabs@1.1.3': '@radix-ui/react-tabs',
      '@radix-ui/react-switch@1.1.3': '@radix-ui/react-switch',
      '@radix-ui/react-slot@1.1.2': '@radix-ui/react-slot',
      '@radix-ui/react-slider@1.2.3': '@radix-ui/react-slider',
      '@radix-ui/react-separator@1.1.2': '@radix-ui/react-separator',
      '@radix-ui/react-select@2.1.6': '@radix-ui/react-select',
      '@radix-ui/react-scroll-area@1.2.3': '@radix-ui/react-scroll-area',
      '@radix-ui/react-radio-group@1.2.3': '@radix-ui/react-radio-group',
      '@radix-ui/react-progress@1.1.2': '@radix-ui/react-progress',
      '@radix-ui/react-popover@1.1.6': '@radix-ui/react-popover',
      '@radix-ui/react-navigation-menu@1.2.5': '@radix-ui/react-navigation-menu',
      '@radix-ui/react-menubar@1.1.6': '@radix-ui/react-menubar',
      '@radix-ui/react-label@2.1.2': '@radix-ui/react-label',
      '@radix-ui/react-hover-card@1.1.6': '@radix-ui/react-hover-card',
      '@radix-ui/react-dropdown-menu@2.1.6': '@radix-ui/react-dropdown-menu',
      '@radix-ui/react-dialog@1.1.6': '@radix-ui/react-dialog',
      '@radix-ui/react-context-menu@2.2.6': '@radix-ui/react-context-menu',
      '@radix-ui/react-collapsible@1.1.3': '@radix-ui/react-collapsible',
      '@radix-ui/react-checkbox@1.1.4': '@radix-ui/react-checkbox',
      '@radix-ui/react-avatar@1.1.3': '@radix-ui/react-avatar',
      '@radix-ui/react-aspect-ratio@1.1.2': '@radix-ui/react-aspect-ratio',
      '@radix-ui/react-alert-dialog@1.1.6': '@radix-ui/react-alert-dialog',
      '@radix-ui/react-accordion@1.2.3': '@radix-ui/react-accordion',
      '@jsr/supabase__supabase-js@2.49.8': '@jsr/supabase__supabase-js',
      '@': path.resolve(__dirname, './src'),
    },
  },
  build: {
    target: 'esnext',
    outDir: 'dist', // ■■: ■■■■■■■■ 'dist'
  },
  server: {
    port: 3000,
    open: true,
  },
});
```